

Tài liệu chuẩn bị bảo vệ ĐATN — PM QLKT

Tài liệu trả lời các câu hỏi hay gặp khi bảo vệ đồ án về ứng dụng web. Mỗi câu hỏi có (1) câu trả lời ngắn 20–30 giây để đáp ngay, (2) phần chi tiết kỹ thuật để trả lời tiếp khi bị truy vấn sâu, (3) đoạn code thật trong project khi có liên quan, và (4) câu hỏi phản biện thường đi kèm.

Mục lục

- 0. Chiến thuật trả lời hội đồng
 - A. Công nghệ và lý do chọn
 - B. Kiến trúc và design pattern
 - C. Bảo mật ứng dụng web
 - D. Race condition và concurrency
 - E. Logic chuỗi danh hiệu
 - F. Cú pháp Prisma đối chiếu SQL
 - G. Hiệu năng và mở rộng
 - H. Kiểm thử
 - I. Triển khai và vận hành
 - J. Câu hỏi khoai và edge case
 - K. Câu hỏi nghiệp vụ quân đội
 - L. Khi không biết câu trả lời
 - M. Khả năng bảo trì và mở rộng kiến trúc
 - N. Tổng hợp chống tấn công và đánh giá an toàn
 - O. Truy vấn nâng cao và tối ưu cơ sở dữ liệu
 - P. Phạm vi đề tài — đã làm và hướng phát triển
-

0. Chiến thuật trả lời hội đồng

Cấu trúc một lần trả lời chuẩn: 1. **Khẳng định mệnh đề chính** (1 câu): “Em xử lý X bằng cách Y.” 2. **Lý do chọn** (1 câu): “Em chọn Y vì Z (so với phương án W).” 3. **Bằng chứng cụ thể** (1 câu, tùy chọn): “Trong file ..., đoạn em viết... cho ra kết quả... Em có viết test case ... kiểm tra điều này.” 4. **Hạn chế còn tồn tại** (nửa câu): “Hiện tại em chưa làm được... nhưng đã đề xuất ở Chương 6.”

Nguyên tắc vàng: - Đừng nói “em không biết” trống không. Nói: “Em chưa kiểm chứng phần đó nhưng theo em hiểu thì... nếu sai mong thầy/cô chỉ giáo.” - Khi bí, kéo hội đồng về phần em làm tốt: “Để em lấy ví dụ cụ thể từ module X trong project...” - Đừng cãi quá 1 vòng. Hội đồng đúng → ghi nhận luôn: “Đúng ạ, em sẽ ghi vào hướng phát triển.” - Câu hỏi mở rộng không có trong đồ án → trả lời “trên lý thuyết” rồi chốt “em có thể bổ sung sau khi có thời gian thử nghiệm.”

A. Công nghệ và lý do chọn

A.1 — Tại sao chọn Next.js 14 App Router thay vì Pages Router hay React thuần?

Ngắn: App Router cho phép Server Components giảm bundle JS phía client, và cấu trúc route lồng theo thư mục để tổ chức cho project có nhiều cấp menu như em.

Chi tiết: - Next.js 14 ổn định App Router từ tháng 10/2023, hỗ trợ tốt React Server Components — phần render trên server không cần gửi JS xuống client, giảm bundle ~30 % cho trang chỉ đọc dữ liệu (vd: trang danh sách quân nhân). - Layout lồng (app/layout.tsx → app/admin/layout.tsx → app/admin/personnel/page.tsx) giúp tách thành điều hướng theo vai trò mà không cần higher-order component. - File-based routing tránh phải duy trì thủ công bảng route map. - So với React + Vite thuần: không có SSR/SSG sẵn → SEO (nếu sau này public ra Internet) sẽ kém hơn; phải tự config router. - So với Pages Router: API mới hơn (loading.tsx, error.tsx, parallel routes) thuận tiện cho UI phức tạp.

Hạn chế nói trước: “Em chưa khai thác hết Server Components — phần lớn trang vẫn là Client Component vì cần Ant Design và state.”

Phản biện thường gặp: “Tại sao không dùng Nuxt/Remix/SvelteKit?” → “Em đã quen React từ trước, nhân lực sau này tiếp nhận project cũng dễ tìm hơn so với Vue/Svelte.”

A.2 — Tại sao Express thay vì NestJS hoặc Fastify?

Ngắn: Express vẫn là framework Node.js có cộng đồng lớn nhất, em đã đủ kinh nghiệm áp đặt kiến trúc layered + Repository lên Express, không cần dependency-injection container của Nest cho quy mô project hiện tại.

Chi tiết: - Nest mạnh ở DI và decorator nhưng kéo theo TypeScript decorator + reflect-metadata + module system riêng — tốn thời gian học cho người maintain sau. - Fastify nhanh hơn ~30 % nhưng ecosystem plugin ít hơn (vd: multer tương đương fastify-multipart không hoàn toàn 1-1). - Em tự áp đặt convention Route → Middleware → Controller → Service → Repository → Prisma qua quy ước file đặt tên — đạt mục tiêu tách lớp mà không cần framework ép buộc.

Phản biện: “Sao không dùng Hono/Bun cho hiện đại hơn?” → “Bun chưa đạt 1.0 ở thời điểm em bắt đầu (đầu 2025), em ưu tiên ổn định hơn.”

A.3 — Tại sao Prisma thay vì TypeORM, Sequelize, hay raw SQL?

Ngắn: Prisma có schema-first (1 file schema.prisma là nguồn duy nhất), client tự sinh type-safe, migration tự sinh từ diff schema — giảm 80 % bug runtime do typo tên cột.

Chi tiết so sánh:

Tiêu chí	Prisma	TypeORM	Sequelize	Raw SQL (pg)
Type-safe	Tự sinh từ schema	Decorator + entity	Yếu	Không
Migration	Auto từ diff	Manual hoặc auto	Manual	Manual
Truy vấn lồng nhau	include/select rõ ràng	Lazy/eager phức tạp	Tương đối	Toàn quyền nhưng dài
Raw escape hatch	<code>\$queryRaw</code> parameterized	<code>query()</code>	<code>query()</code>	Mặc định
Performance	Tốt, có connection pool	Tốt	Tương đương	Nhanh nhất
Học	Dễ nhất	Khó nhất (nhiều khái niệm)	Trung bình	Phụ thuộc người dùng

- Prisma sinh client tự động sau `npx prisma generate` → IDE autocomplete cho tên model, tên cột, kiểu dữ liệu.
- `Migration prisma migrate dev` tạo file SQL có thể commit, đảm bảo môi trường dev/staging/prod cùng schema.
- Hạn chế Prisma: query phức tạp dạng window function hoặc CTE phải fallback `$queryRaw` (em có sẵn vài chỗ trong dashboard service).

Phản biện: “Prisma có overhead không?” → “Có ~10–15 % cho query đơn giản so với raw pg, nhưng đổi lại type-safe và DX. Khi có endpoint chậm, em fallback `$queryRaw`.”

A.4 — Tại sao PostgreSQL thay vì MySQL hoặc MongoDB?

Ngắn: PostgreSQL hỗ trợ JSONB, CTE, window function, partial index — phù hợp cho dữ liệu nửa-cấu-trúc như cờ thành tích trong DanhHieuHangNam và truy vấn phân tích dashboard.

Chi tiết: - Schema khen thưởng có 23 model có quan hệ chặt → cần quan hệ 1-N, M-N nguyên gốc → relational DB phù hợp hơn MongoDB. - MongoDB không có transaction multi-document mặc định trong môi trường standalone → khó dùng cho luồng phê duyệt cần ACID. - MySQL cũng tốt nhưng PostgreSQL có: - JSONB index (em dùng cho `co_quan_don_vi` trong `QuanNhan` và `payload` trong `SystemLog`). - RETURNING clause sau INSERT/UPDATE/DELETE (Prisma đã tự dùng). - tự sinh chuỗi `INSERT INTO ... VALUES` trong `backup.service.ts` (không dùng `pg_dump`). - Foreign key ON UPDATE CASCADE chặt chẽ hơn MySQL.

Phản biện: “PostgreSQL có nặng cho LAN nội bộ?” → “Một instance Postgres ăn ~150 MB RAM ở idle, hoàn toàn chạy được trên server 4 GB như em đề xuất.”

A.5 — Tại sao chọn Zod cho cả backend và frontend?

Ngắn: Zod cho phép suy luận kiểu TypeScript trực tiếp từ schema (`z.infer<typeof schema>`), nên dùng cùng một thư viện ở cả hai phía giúp lập trình viên không phải học hai cú pháp khác nhau cho cùng một mục đích và mở đường để chia sẻ schema giữa BE/FE.

Chi tiết: - **Zod ở BE:** dùng trong middleware `validate.ts` để kiểm tra `req.body`, `req.query`, `req.params`; mỗi endpoint có schema riêng (vd: `accountValidation.create`). `z.object()` mặc định strip các field ngoài schema (chống mass assignment) mà không cần option như `stripUnknown` của Joi. - **Zod ở FE:** infer kiểu trực tiếp từ schema → form data type-safe khi submit. Schema dùng trong Ant Design Form qua validator tùy chỉnh hoặc trong React Hook Form qua `zodResolver`. - Lý do **chọn Zod thay vì Joi** (lựa chọn ban đầu của em): - Joi không có khả năng suy luận kiểu TypeScript — kết quả validate trả về `unknown`, ép phải cast hoặc khai báo type song song. - Joi và Zod có cú pháp hoàn toàn khác nhau; nếu dùng Joi BE + Zod FE thì lập trình viên phải nhớ hai API cho cùng một thao tác (`validate string`, `refine`, `transform...`). - Zod hỗ trợ async validation, refinement và transform đầy đủ, đủ dùng cho mọi rule nghiệp vụ của PM QLKT. - **Hạn chế hiện tại:** Schema vẫn được khai báo riêng ở BE và FE (chưa shared qua workspace package). Trong tương lai có thể tách thành package `shared/` để loại bỏ phần lặp này.

Phản biện: “Sao không dùng class-validator như NestJS?” → “Em không dùng decorator để giữ tương thích với TS không bật experimental flag, đồng thời để cú pháp schema BE giống hết FE.”

A.6 — Tại sao JWT (access + refresh) chứ không phải session-based?

Ngắn: Server stateless dễ scale ngang khi sau này deploy nhiều instance; refresh token rotation cho phép thu hồi session từ server mà không cần Redis.

Chi tiết theo code thật (auth.service.ts): - **Access token:** 30 phút, ký bằng `JWT_SECRET`, payload chứa `id`, `username`, `role`, `quan_nhan_id`. - **Refresh token:** 7 ngày, ký bằng `JWT_REFRESH_SECRET` riêng, payload chỉ `id` và `username` để giảm bề mặt rò rỉ. - **Lưu trữ refresh:** lưu `refreshToken` trong cột `TaiKhoan.refreshToken` → mỗi tài khoản chỉ có 1 phiên hoạt động cùng lúc. Đăng nhập mới sẽ ghi đè và `emitToUser('force_logout')` cho session cũ. - **Verify access:** middleware `verifyToken` check JWT chữ ký + so sánh có `refreshToken` trong DB → nếu admin xoá `refreshToken` thì access token đang còn hạn cũng bị từ chối.

Yếu điểm trung thực: - JWT không thể thu hồi chỉ access token — em mitigate bằng cách check DB mỗi request (đã đánh đổi 1 query mỗi request lấy khả năng revoke). - Nếu access token bị lộ trong 30 phút → kẻ tấn công vẫn dùng được. Cách giảm: rút expire xuống 15 phút.

Token leak qua đâu — phòng thể nào: - **Log:** Express middleware chặn log header `Authorization` trong access log (winston format custom). - **Network:** LAN nội bộ → traffic không qua Internet → MITM khó. Khi nâng HTTPS với cert nội bộ, JWT trên đường dây được mã hoá TLS. - **Browser DevTools:** User vẫn thấy JWT trong Network tab — đây là rủi ro của mọi app dùng JWT. Mitigate: short expire + refresh rotation. - **localStorage XSS:** XSS đọc được localStorage. Em chống bằng React tự escape + không dùng eval — XSS gần như không thực hiện được.

Phản biện: “Lưu refreshToken trong DB là làm mất ưu điểm stateless?” → “Đúng, đây là đánh đổi giữa scaling và khả năng revoke. Em chọn revoke vì project chạy LAN, không cần scale ngang.”

A.7 — Tại sao Socket.IO mà không phải WebSocket native hay SSE?

Ngắn: Socket.IO có fallback HTTP long-polling khi mạng nội bộ chặn WebSocket, room/namespace sẵn để gửi notification cho 1 user, và reconnect tự động.

Chi tiết: - WebSocket native không có fallback — nếu firewall LAN chặn, người dùng mất thông báo. - SSE chỉ 1 chiều server → client, không gửi ngược được. Em cần ngược lại cho ack. - Socket.IO room: `socket.join(userId)` → `io.to(userId).emit(...)` gửi đúng người. - Em dùng cho 3 luồng: - Notification đề xuất mới gửi cho ADMIN. - Force logout khi đăng nhập từ nơi khác. - Tiến trình import Excel lớn (push từng dòng).

Phản biện: “Sao không Pusher/Ably?” → “Phải gửi data ra Internet, vi phạm chính sách bảo mật LAN nội bộ.”

A.8 — Ant Design + Tailwind CSS — tại sao kết hợp 2 thư viện UI?

Ngắn: Hai thư viện phục vụ hai mục đích khác nhau, không trùng lặp: Ant Design cho component nghiệp vụ phức tạp (Form, Table, Modal, Dropdown), Tailwind CSS cho spacing/layout/responsive — không động đến logic component.

Chi tiết: - **Ant Design:** form validation tích hợp, table có pagination/sort/filter sẵn, locale tiếng Việt — rút ngắn ~50 % code so với tự build component. - **Tailwind:** dùng cho layout grid, spacing margin/padding, flex/responsive — chỉnh các vị trí mà Ant Design chưa đáp ứng được (vd: bố cục thẻ huy chương 2 cột trên desktop, 1 cột trên mobile).

Hạn chế: Bundle CSS có thể overlap nhẹ. Em đã purge Tailwind theo content và import từng component AntD theo nhu cầu (import { Table } from 'antd').

Phản biện: “Có thể chỉ dùng Tailwind + Headless UI?” → “Có, nhưng phải tự xây Form, Table — tốn 4–6 tuần thêm.”

A.9 — Tại sao Jest mà không phải Vitest hay Mocha?

Ngắn: Jest tích hợp ts-jest chạy file .ts không cần build, mocking sẵn, cộng đồng lớn nhất cho Node.js backend. Vitest mới hơn, tốt cho FE Vite nhưng chưa cần đổi.

Chi tiết: - 937 ca kiểm thử / 80 file hiện chạy trong ~7 giây — chấp nhận được. - Jest snapshot testing chưa dùng nhiều, chủ yếu unit test pure function. - jest --coverage sinh báo cáo HTML tại coverage/lcov-report/index.html, đạt > 85 % cho services/profile, services/eligibility, services/proposal.

A.10 — ExcelJS, multer, bcrypt, nodemailer — vai trò?

Thư viện	Vai trò trong project	Tại sao chọn
ExcelJS	Đọc/ghi .xlsx cho import danh hiệu hằng năm và xuất danh sách	Xử lý formula, style, merged cell tốt hơn xlsx thuần
multer	Nhận file PDF quyết định và file Excel upload	Streaming, không nuốt RAM với file lớn
bcrypt	Hash password tài khoản	Adaptive cost (em dùng cost 10 = ~100 ms/hash)
nodemailer	(Hiện chưa kích hoạt) gửi email reset password	Có sẵn để bật khi cần

A.11 — Next.js 14 build sinh ra những file gì? Khác gì so với React + Vite?

Ngắn: Next.js 14 chia ra 3 nhóm artifact: file dev sinh khi next dev, file build sinh khi next build, file type sinh tự động cho TypeScript. React + Vite chỉ có nhóm dev và build, không có generation cho route file-based.

Khi gõ next dev lần đầu: - next-env.d.ts — file tự sinh ở root, khai báo type cho *.module.css, image import, env vars. **Đừng commit edit thủ công** — Next overwrite mỗi lần chạy. - .next/cache/ — cache webpack/SWC để dev start nhanh hơn lần 2. - .next/types/ (Next 13.2+) — type cho route handler, link href, dynamic param. Bật bằng experimental.typeRoutes để type-check tên route.

Khi gõ next build: - .next/server/ — code render trên server (RSC + route handler). - .next/static/ — JS/CSS bundle gửi xuống browser, có content hash trong tên file. - .next/standalone/ (nếu output: 'standalone' trong next.config.js) — bundle tự chứa Node modules, có thể node server.js chạy luôn. - .next/build-manifest.json + app-build-manifest.json — map route → JS chunk.

Em không có config standalone ở project này → deploy qua PM2 với npm start (alias next start). .next/ toàn bộ phải được copy lên server.

Khác React + Vite thuần: | Việc | Next.js 14 | React + Vite | |—|—|—| | Routing | File-based (folder = route) | Phải tự setup react-router || SSR/SSG/ISR | Tự động theo file page.tsx | Không có sẵn || API endpoint | app/api/.../route.ts cùng repo | Phải BE riêng || Image optimization | next/image resize on-demand | Phải tự xử lý || Bundle analyzer | @next/bundle-analyzer | Phải tự setup || Dev server | next dev (port 3000) | vite (port 5173) |

Em chọn Next.js dù không cần SSR vì: routing file-based + middleware tự động + hỗ trợ Server Components về sau khi cần tối ưu bundle.

Phản biện: “Em có dùng SSR/SSG không?” → “Hệ thống nội bộ cần đăng nhập, không có trang public → em dùng Client Components là chính. SSR chỉ giá trị khi có SEO public.”

A.12 — App Router file conventions: page, layout, loading, error, ... — kể chi tiết

Ngắn: Mỗi tên file đặc biệt trong app/ có ngữ nghĩa cố định, Next.js tự render đúng vị trí. Em dùng 5 trong số đó cho project: layout.tsx, page.tsx, error.tsx, not-found.tsx, loading.tsx.

Bảng đầy đủ Next.js 14 hỗ trợ:

File	Vai trò	Project em có
layout.tsx	UI bao quanh nhiều page con, không re-render khi navigate giữa con	☐ app/layout.tsx (root) + app/admin/layout.tsx (sidebar admin)
page.tsx	UI cho route đó, làm route public	☐ tất cả route
loading.tsx	UI hiển thị khi page con đang fetch (Suspense boundary tự động)	Chưa dùng
error.tsx	Error boundary, catch lỗi từ page con	☐ app/error.tsx
not-found.tsx	Render khi notFound() được gọi hoặc route không match	☐ app/not-found.tsx
template.tsx	Như layout nhưng re-mount mỗi navigation	Chưa dùng
route.ts	Route handler (REST endpoint thay vì UI)	Chưa dùng — em có BE Express riêng
default.tsx	Fallback cho parallel route	Chưa dùng

Quy ước folder: - app/(auth)/login/page.tsx — group route, dấu () không vào URL → URL là /login. Em dùng để gom login, change-password mà không làm tăng path. - app/admin/personnel/[id]/page.tsx — dynamic param, params.id được Next inject vào prop của page. - app/admin/personnel/[id]/edit/page.tsx — nested route, URL là /admin/personnel/abc123/edit.

Render order:

```
RootLayout (app/layout.tsx)
├─ AdminLayout (app/admin/layout.tsx)
│   └─ ErrorBoundary (app/error.tsx)
│       └─ Suspense (loading.tsx nếu có)
│           └─ Page (app/admin/personnel/[id]/page.tsx)
```

Phân biệt: “Em không dùng loading.tsx?” → “Em dùng <LoadingState> shared component bên trong page, kiểm soát chi tiết hơn — nhưng đúng là loading.tsx chuẩn Next hơn, em sẽ chuyển đổi nếu có thời gian.”

A.13 — Server Components vs Client Components — em dùng cái nào?

Ngắn: Mặc định Next 14 App Router coi mọi component là Server Component. Em phải gắn 'use client' ở đầu file để chuyển sang Client Component khi cần state/effect/event listener. Project em hầu hết là Client Component vì dùng Ant Design.

Server Component (mặc định): - Render trên server, kết quả là HTML + RSC payload (không gửi JS xuống). - KHÔNG dùng được: useState, useEffect, onClick, browser API (window, localStorage). - Dùng được: async/await trực tiếp trong body, gọi DB/API ngay trong component. - Lợi ích: giảm bundle JS, fetch song song trên server, không leak secret xuống client.

Client Component ('use client'): - Render bootstrap trên server (HTML đầu) + hydrate trên browser. - Dùng được hooks, event handler, browser API. - Bắt buộc cho: form, modal, animation, AntD component (vì AntD dùng useContext).

Project em: - 'use client' ở đa số page (do AntD Form, Table, Modal cần context). - Server Component **chỉ** dùng cho 2 layout root đơn giản (app/layout.tsx, app/admin/layout.tsx). - Trade-off: bundle to hơn nhưng DX (developer experience) đơn giản — không phải nhớ ranh giới.

Quy tắc thực dụng: - Component có useState/useEffect/onClick → 'use client'. - Component import AntD/Tailwind plugin có hook → 'use client'. - Component chỉ render markup tĩnh + fetch dữ liệu → có thể Server Component.

Hạn chế trung thực: “Em chưa khai thác hết Server Components — nếu chuyển 1 nửa số page sang RSC, bundle sẽ giảm thêm ~25 %.”

Phản biện: “Component cha là Server, con là Client truyền props — props phải serializable?” → “Đúng. Em không truyền function/JSEX qua ranh giới này, chỉ truyền data thuần.”

A.14 — Tailwind CSS + PostCSS — config file gì, hoạt động ra sao?

Ngắn: Tailwind sinh CSS theo class trong code (JIT — Just-In-Time). PostCSS là pipeline xử lý plugin CSS.

File cấu hình project em có: - `tailwind.config.js` — khai báo `content: ['./src/**/*.{ts,tsx}']` để Tailwind scan class từ code, `theme.extend` thêm color palette tùy biến, `darkMode: 'class'` bật dark mode qua class trên `<html>`. - `postcss.config.js` — chạy `tailwindcss + autoprefixer` plugin. Next.js đọc file này tự động khi build. - `src/app/globals.css` — import 3 directive @tailwind base/components/utilities. File này được import 1 lần ở `app/layout.tsx`. - `src/lib/utils.ts` — chứa `cn()` helper (`clsx + tailwind-merge`) để merge các class Tailwind có thể conflict (vd: `cn('p-4', isActive && 'p-2')` ra `p-2` đúng).

Tailwind hoạt động: 1. `next dev` → PostCSS chạy. 2. Tailwind plugin scan content glob, tìm class string (`text-red-500, flex, ...`) trong file `.tsx`. 3. Sinh CSS chỉ chứa class được dùng → bundle CSS final ~20-30 KB cho project em (so với 3 MB nếu include hết Tailwind).

Hạn chế: Bundle CSS có overlap nhẹ giữa AntD reset và Tailwind preflight. Em đã thử disable preflight (`corePlugins.preflight: false`) — không đáng kể về kích thước nhưng AntD style ưu tiên hơn nếu xảy ra xung đột.

Phản biện: “Sao không dùng styled-components hoặc emotion?” → “CSS-in-JS overhead runtime (~10-20 KB). Tailwind biên dịch lúc build, runtime cost = 0.”

A.15 — TypeScript config: BE strict: false, FE strict — tại sao khác?

Ngắn: BE em đặt `strict: false` để giảm friction khi viết Prisma query lồng và cast Prisma JSON column. FE bật strict đầy đủ vì component cần type-safe để IDE refactor an toàn.

File config: - BE-QLKT/`tsconfig.json`: `strict: false, strictNullChecks: false, target: ES2020, module: CommonJS`. Output không phải `.js` build (em dùng `tsx watch` ở dev và `tsc` ở production build vào `dist/`). - FE-QLKT/`tsconfig.json`: `strict: true, target: ES2017, module: esnext, moduleResolution: bundler, paths: { "@/*": ["./src/*"] }` cho path alias.

Lý do BE relax: - Prisma `findUnique` trả `T | null`, đôi khi em chắc chắn record tồn tại (vừa create xong) → cast ! hoặc destructure không null check là acceptable. - Prisma JSON column (vd: `data_danh_hieu`) có kiểu `Prisma.JsonValue` — `strictNullChecks` ép thêm narrow rườm rà cho mỗi lần đọc field bên trong. - BE đã có Zod validation ở route → input đã được làm sạch, runtime safety không phụ thuộc TS strict.

Lý do FE strict: - Component nhận props nhiều cấp lồng — strict null check cứu khỏi `Cannot read property 'x' of undefined` khi render. - Refactor tên field DB → IDE báo đỏ ngay nơi sai.

Trade-off em chấp nhận: BE có ~5 chỗ `as any` (đã loại hết trong commit gần đây), FE chỉ có 0 sau khi cleanup.

Phản biện: “Sao không bật strict cả 2?” → “Bật strict BE phải sửa ~80 chỗ liên quan đến Prisma null. Em ưu tiên tốc độ phát triển. Có thể bật dần qua flag `noUncheckedIndexedAccess` rồi mới đến full strict.”

A.16 — Prisma CLI: migrate dev vs db push vs generate vs migrate deploy — khác gì?

Ngắn: Bốn lệnh phục vụ vòng đời khác nhau: `generate` sinh client TS, `migrate dev` tạo migration file ở dev, `db push` đồng bộ schema không tạo migration, `migrate deploy` áp migration đã có ở production.

Vòng đời em đang dùng:

Lệnh	Khi nào dùng	Hậu quả
<code>npx prisma generate</code>	Sau khi đổi <code>schema.prisma</code> , trước khi gõ code	Sinh <code>src/generated/prisma/</code> (em config custom output, không dùng <code>node_modules/.prisma</code> mặc định)

Lệnh	Khi nào dùng	Hậu quả
<code>npx prisma migrate dev</code>	Dev: thêm/sửa cột	Sinh file <code>prisma/migrations/<timestamp>_<name>.sql</code> + auto chạy + auto generate
<code>npx prisma db push</code>	Dev: prototype nhanh, không có data quan trọng	Sync schema vào DB, KHÔNG tạo migration file — chỉ dùng nháp
<code>npx prisma migrate deploy</code>	Production: áp tất cả migration chưa chạy	Đọc folder <code>prisma/migrations/</code> , chạy theo thứ tự, không tương tác
<code>npx prisma migrate reset</code>	Dev: reset DB sạch	Drop + recreate + chạy lại tất cả migration + seed
<code>npx prisma studio</code>	Dev: GUI xem/edit data	Mở web UI port 5555

Điểm em đã trả giá học: - **Đổi tên cột có data → KHÔNG dùng `db push`** — `db push` sẽ DROP cột cũ + CREATE cột mới → mất hết data. Phải viết script `prisma.$executeRawUnsafe('ALTER TABLE x RENAME COLUMN old TO new')` trong `src/scripts/` trước, rồi mới `db push` để Prisma sync schema. - Em đã ghi rule này vào `BE-QLKT/CLAUDE.md` (AP-8) sau khi suýt mất data lần đầu.

Custom output: `schema.prisma` của em có:

```
generator client {
  provider = "prisma-client-js"
  output   = "../src/generated/prisma"
}
```

→ Client sinh vào `src/generated/prisma/` thay vì `node_modules/@prisma/client`. Lý do: control version đi kèm code, không phụ thuộc reinstall.

Phản biện: “Sinh client vào `src/generated/` thì có nên commit không?” → “Có, vì TS strict ở FE cần type, và CI nhanh hơn (không phải `prisma generate` lại). Trade-off là repo to hơn ~5 MB.”

A.17 — ESLint, Prettier, husky/lint-staged — em setup thế nào?

Ngắn: FE có ESLint + Prettier, BE chỉ có Prettier. Cả hai bên không có pre-commit hook (`husky/lint-staged`) — em phải nhớ chạy `npm run format` thủ công.

FE config: - `.eslintrc.json` — ESLint legacy config, extends `next/core-web-vitals + prettier`, plugin `unused-imports` rule `no-unused-imports: error` để chặn import thừa. - `.prettierrc` — `semi: true, singleQuote: true, tabWidth: 2, printWidth: 100, trailingComma: 'es5', arrowParens: 'avoid'`. - Script: `npm run lint` (gọi next lint), `npm run format` (gọi prettier --write).

BE config: - `.prettierrc` cùng convention với FE. - **KHÔNG** có `.eslintrc` — em rely vào `tsc --noEmit` (`npm run typecheck`) và Prettier để giữ chuẩn. - Script: `npm run typecheck`, `npm run format`.

Hạn chế trung thực: - **Không có pre-commit hook** — nếu em quên chạy `format`, code messy có thể commit. Đây là tech debt em đã ghi vào `PROJECT_REVIEW.md` §LOW. - Có thể thêm `husky + lint-staged` chạy `prettier --write + tsc --noEmit` trên file staged để chặn commit lỗi format/type.

Phản biện: “Sao không dùng Biome thay ESLint + Prettier?” → “Biome mới (1.0 cuối 2023), em chưa migrate vì project đã ổn định. Để hướng phát triển.”

A.18 — PM2 ecosystem + Nginx reverse proxy — config thế nào?

Ngắn: PM2 chạy BE Node.js (port 4000) và FE Next.js production server (port 3000) với auto-restart. Mỗi app có `ecosystem.config.js` riêng. Nginx đặt trước, terminate TLS, reverse proxy `/api/*` về BE và `/*` về FE.

BE-QLKT/ecosystem.config.js (rút gọn):

```

module.exports = {
  apps: [{
    name: 'be-qlkt',
    script: 'dist/index.js',          // compiled từ tsc, KHÔNG dùng tsx production
    instances: 1,
    exec_mode: 'fork',
    autorestart: true,
    max_memory_restart: '500M',      // restart nếu RSS > 500 MB (chống memory leak)
    env_file: '.env',
    log_date_format: 'YYYY-MM-DD HH:mm:ss',
    out_file: 'logs/out.log',
    error_file: 'logs/error.log',
  }],
};

```

FE-QLKT/ecosystem.config.js: tương tự, script: 'node_modules/.bin/next'+args: 'start'.

Khởi động: pm2 start BE-QLKT/ecosystem.config.js && pm2 start FE-QLKT/ecosystem.config.js.
 Lưu state: pm2 save && pm2 startup để tự khởi động lại sau reboot server.

Nginx (rút gọn):

```

server {
  listen 80;
  server_name qlkt.local;

  client_max_body_size 50M; # cho upload Excel/PDF lớn

  location /api/ {
    proxy_pass http://localhost:4000;
    proxy_set_header Upgrade $http_upgrade;      # Socket.IO
    proxy_set_header Connection "upgrade";
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
  }

  location /socket.io/ {
    proxy_pass http://localhost:4000;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
  }

  location / {
    proxy_pass http://localhost:3000;
  }
}

```

Vai trò Nginx: - TLS termination (chứng chỉ self-signed cho LAN). - Static caching cho /_next/static/* (tăng tốc tải lần 2). - Buffer body upload — protect Node khỏi slowloris. - Single entry point → user chỉ thấy 1 origin, không gặp CORS.

Express phải `app.set('trust proxy', 1)` để req.ip lấy đúng IP client từ header X-Forwarded-For.

Phản biện: “Sao không dùng Caddy thay Nginx?” → “Caddy auto-HTTPS rất tiện cho public Internet, nhưng LAN nội bộ em đã có cert nội bộ, Nginx ổn định và phổ biến hơn ở Việt Nam.”

A.19 — Logging, helmet, rate-limit, dotenv, cors — middleware Express còn lại

Ngắn: 5 middleware chuẩn cho Express production. Em dùng tất cả ngoại trừ logging file thì rely vào console.error + system_logs DB thay vì winston.

Thư viện	Mục đích	Config trong code
helmet	Set 14 security header (CSP, HSTS, X-Frame, ...)	<code>app.use(helmet())</code> ở <code>index.ts</code>
cors	Whitelist origin	<code>app.use(cors({ origin: ALLOWED_ORIGINS, credentials: true })))</code> import 'dotenv/config' ở đầu <code>index.ts</code>
dotenv	Load .env vào <code>process.env</code>	<code>authLimiter 30 req/5min, writeLimiter 30 req/15min — file configs/rateLimiter.ts</code>
express-rate-limit	Chặn DoS / brute force	Em không dùng — log qua <code>system_logs</code> DB cho audit, console output đủ ở dev
morgan	Log HTTP request	

Vì sao không dùng winston riêng? - System log đã ghi vào DB (`system_logs` table) qua `writeSystemLog()` — có thể query/filter/visualize qua trang Admin. - PM2 đã tự ghi `logs/out.log` (stdout) và `logs/error.log` (stderr) — em chỉ cần `console.log/console.error` ở app code, PM2 capture lại. - Trade-off: PM2 không tự rotate file log → file có thể to dần. Cần thêm plugin `pm2-logrotate` (`pm2 install pm2-logrotate`) để rotate theo size hoặc thời gian. Đã ghi vào hướng phát triển.

Helmet tính chỉnh (`src/index.ts:40-44`):

```
app.use(helmet({ crossOriginResourcePolicy: { policy: 'cross-origin' } }));
```

- Đặt `crossOriginResourcePolicy: cross-origin` để FE (port 3000) load được file PDF/Excel served từ BE (port 4000) — mặc định helmet là `same-origin`.
- Các header khác giữ default: HSTS (chỉ effect khi HTTPS), X-Frame-Options DENY, X-Content-Type-Options nosniff, X-XSS-Protection, ...
- Đáng lẽ nên thêm `contentSecurityPolicy` nhưng AntD inject inline style → cần allow 'unsafe-inline' cho `style-src`. Em chưa setup → ghi vào hướng phát triển.

Phản biện: “Sao không bật full CSP?” → “AntD chưa hỗ trợ nonce-based CSP. Khi nào AntD v6 ra (đã có roadmap), em sẽ migrate. Hiện LAN nội bộ rủi ro XSS thấp.”

A.20 — Thư viện FE phụ: dayjs, axios, chart.js, react-pdf-viewer

Ngắn: 4 thư viện FE phụ trợ. Mỗi cái thay thế phương án “to” hơn để giữ bundle nhỏ.

Thư viện	Mục đích	Thay cho
dayjs (~7 KB)	Format/parse date, locale tiếng Việt	<code>moment.js</code> (~70 KB), <code>date-fns</code> (~13 KB <code>tree-shakable</code>)
axios	HTTP client với interceptor	<code>fetch</code> (phải tự wrap), <code>TanStack Query</code> (overkill cho CRUD đơn giản)
chart.js + react-chartjs-2	Biểu đồ dashboard	<code>recharts</code> (phình bundle), <code>apache echarts</code> (overkill)
@react-pdf-viewer/core	Xem PDF quyết định inline	<code>iframe src=...</code> (không có UI điều khiển), <code>pdf.js</code> thuần (phải tự build UI)

Form state dùng `Form.useForm()` của Ant Design (built-in, không cần thư viện ngoài), validation gọi `zod-Schema.safeParse()` trong handler rồi map lỗi qua `form.setFields()`.

Axios interceptor (src/lib/axiosInstance.ts): - Request: tự gắn `Authorization: Bearer <accessToken>` từ `localStorage`. - Response: nếu 401 → tự gọi `/api/auth/refresh` → retry request gốc 1 lần. Nếu

refresh cũng 401 → redirect /login. - Lý do dùng axios thay fetch: interceptor pattern cleaner, retry logic ngắn hơn 50 % so với fetch wrapper.

dayjs locale:

```
import dayjs from 'dayjs';
import 'dayjs/locale/vi';
dayjs.locale('vi'); // → "tháng 5 năm 2026"
```

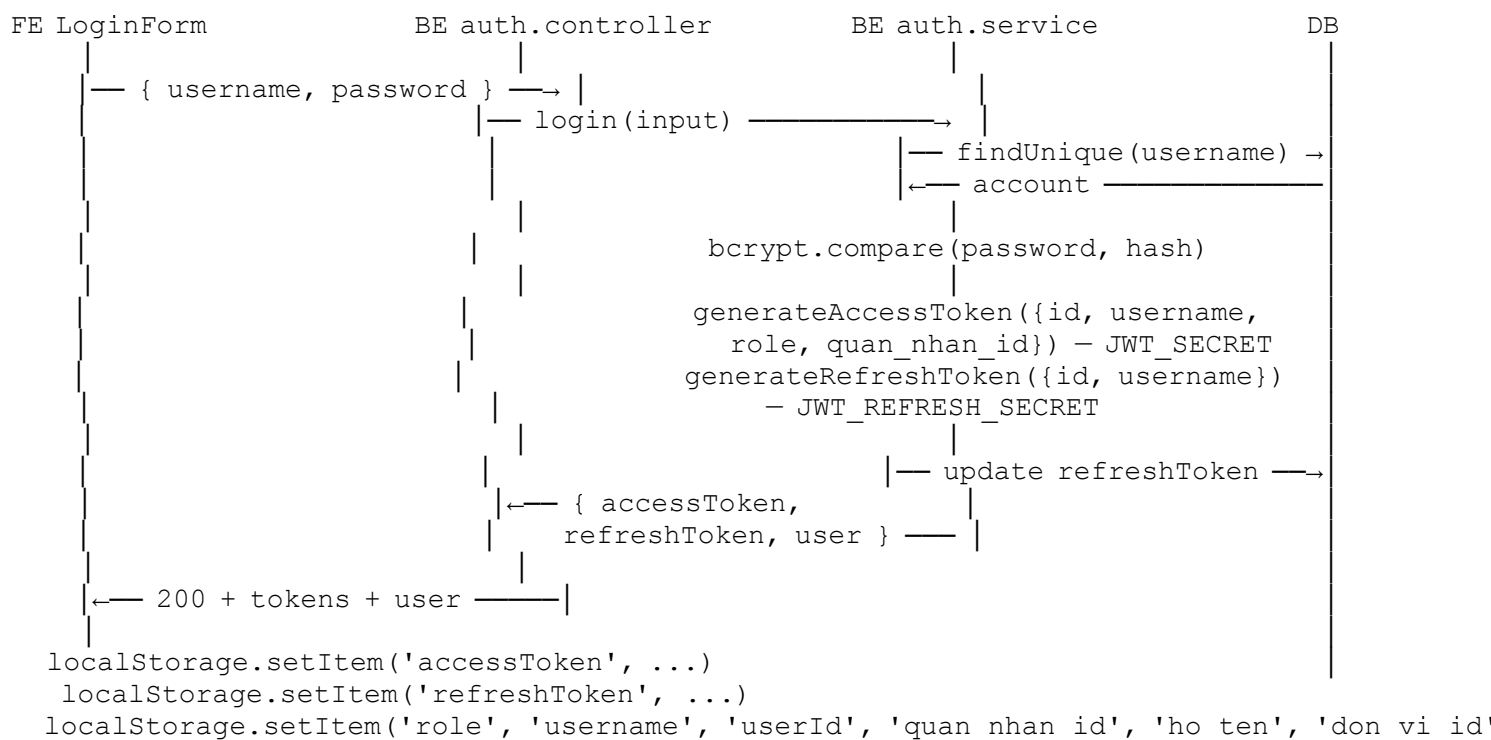
Phản biện: “Sao không dùng TanStack Query cho data fetching?” → “TanStack Query mạnh khi cần cache + invalidation phức tạp. Em chỉ có CRUD + form, custom hook useFetch/useMutation đủ — bundle gọn hơn ~30 KB.”

A.21 — Cơ chế 2 token (access + refresh) end-to-end — login → refresh → logout

Ngắn: Hai token có vai trò khác nhau: access token đóng vai trò “vé vào cửa” mỗi request (ngắn, 30 phút), refresh token đóng vai trò “thẻ thành viên” để xin vé mới (dài, 7 ngày). Mỗi lần refresh, server sinh CẢ access lẫn refresh mới (rotation) → token cũ vô hiệu hoá ngay.

4 giai đoạn:

1. Đăng nhập (POST /api/auth/login)

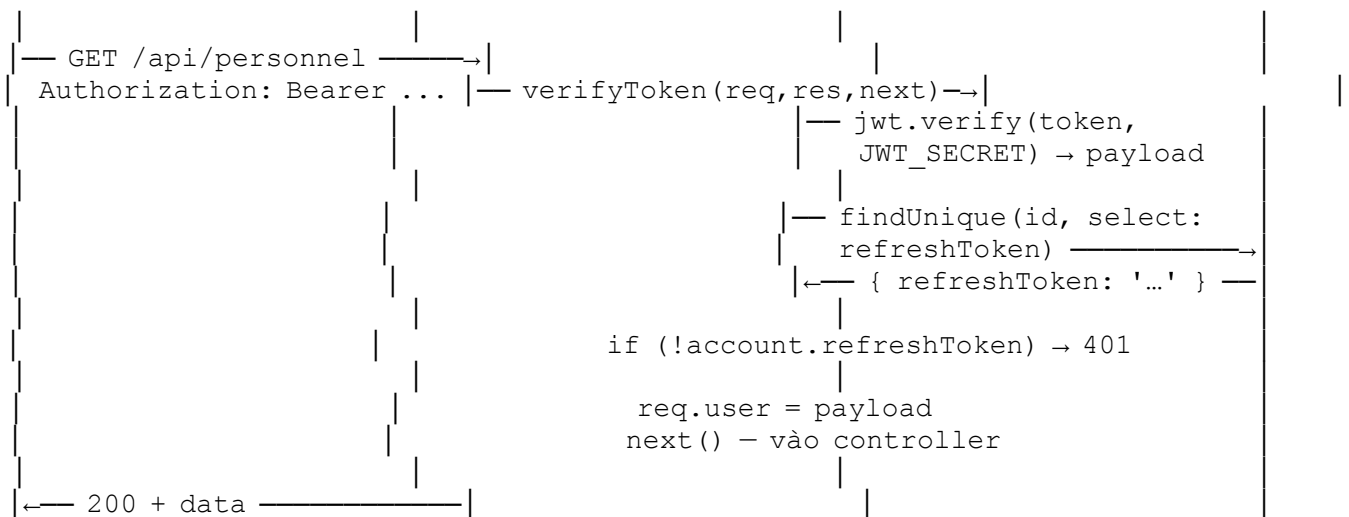


File: BE/src/services/auth.service.ts:35-95, FE/src/contexts/AuthContext.tsx:96-110

Vì sao lưu refresh trong DB? Ý tưởng “stateless” của JWT thuần KHÔNG cho phép thu hồi. Em đánh đổi: thêm 1 cột `TaiKhoan.refreshToken` để có khả năng force-logout. Khi user đăng nhập từ thiết bị khác → ghi đè cột này → thiết bị cũ không refresh được nữa → bị đẩy ra.

2. Request bình thường



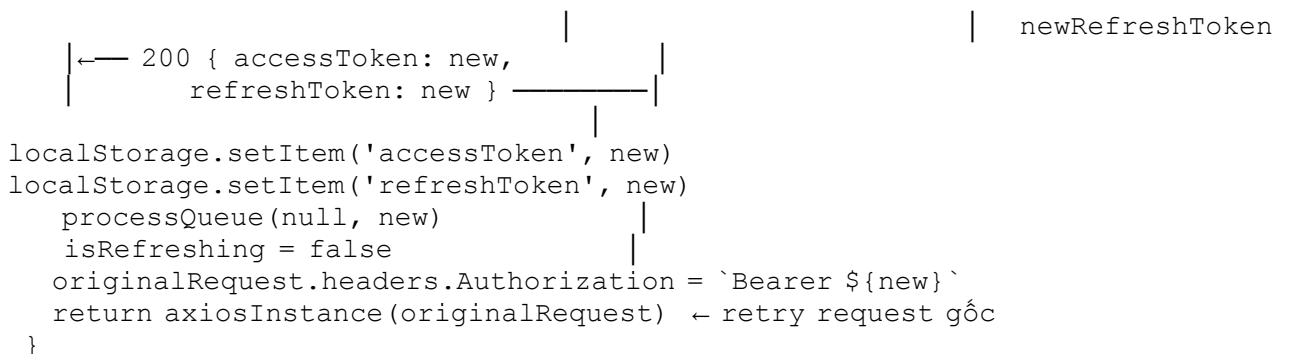


File: BE/src/middlewares/auth.ts:29-40, FE/src/lib/axiosInstance.ts:13-20.

Tại sao check DB mỗi request? Để có thể revoke. Trade-off: 1 query thêm/request (~1ms với index trên id). Đáng cho LAN nội bộ.

3. Access token hết hạn → refresh (POST /api/auth/refresh) — có rotation



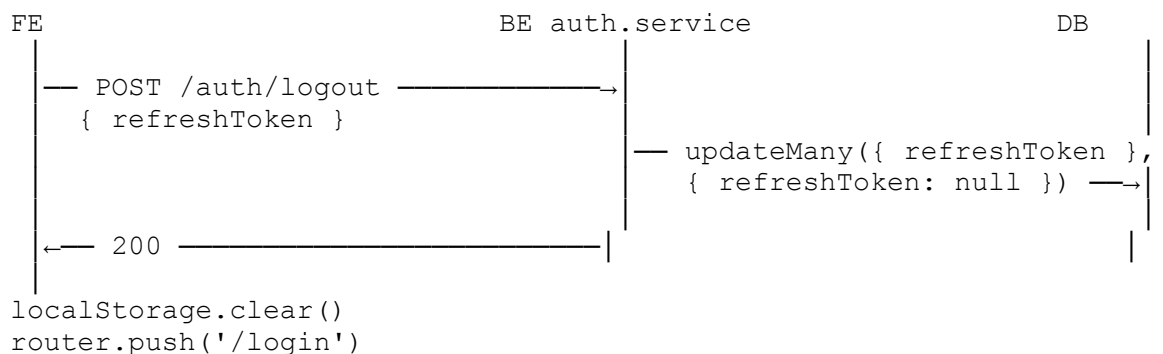


File: BE/src/services/auth.service.ts:111-138, FE/src/lib/axiosInstance.ts:58-180.

Rotation: Mỗi lần refresh, BE sinh CẢ accessToken VÀ refreshToken mới, ghi đè cột DB. Refresh token cũ không còn match → reuse refresh cũ = 401 ngay. Đây là OWASP recommended pattern.

Concurrent refresh: Nếu 5 request 401 cùng lúc, chỉ 1 request đầu thực sự gọi /auth/refresh, 4 cái còn lại đứng failedQueue chờ → tránh sinh 5 cặp token.

4. Logout (POST /api/auth/logout)



File: BE/src/services/auth.service.ts:145-152.

Logout chỉ xoá refresh token trong DB, không invalidate access token đang còn 30 phút. Mitigate: middleware verifyToken check DB phát hiện refreshToken: null → 401 cho mọi request từ device đó NGAY LẬP TỨC.

Force logout cross-device: Đăng nhập máy mới → server ghi đè TaiKhoan.refreshToken. Máy cũ access token expire sau ≤30min → gọi refresh → DB.refreshToken không match → 401 → FE forceLogout. Em còn emit Socket.IO force_logout event để máy cũ logout NGAY thay vì chờ 30 phút.

Bảng tổng kết 2 token:

Tiêu chí	Access Token	Refresh Token
Secret	JWT_SECRET	JWT_REFRESH_SECRET (riêng)
TTL	30 phút	7 ngày
Payload	{id, username, role, quan_nhan_id}	{id, username} (tối giản)
Lưu trong DB	KHÔNG	CÓ (cột TaiKhoan.refreshToken)
Gửi mỗi request	CÓ (header Authorization)	KHÔNG (chỉ khi gọi refresh)
Rotation khi refresh	Sinh mới	Sinh mới + ghi đè DB
Có thể revoke server-side?	Gián tiếp (qua DB check)	Trực tiếp (set null)
Lưu phía client	localStorage	localStorage

Tại sao 2 token mà không phải 1?

với signature gửi lên — KHÁC nhau dù 1 byte → throw `JsonWebTokenError`. 5. Nếu khớp → check `exp > now` — quá hạn → throw `TokenExpiredError`. 6. Nếu cả 2 OK → return decoded payload.

Vì sao user không tự sửa được role thành ADMIN? - User decode payload (base64url, không cần secret) → đổi `role: 'USER'` thành `role: 'ADMIN'` → re-encode → ghép lại signature CŨ. - BE recompute signature từ header+payload mới (đã sửa) → ra signature MỚI khác signature cũ user gửi → throw error. - Để forge thành công, user phải biết `JWT_SECRET` để compute signature đúng — bất khả thi nếu secret 256-bit random và lưu trong `.env` chmod 600.

HS256 vs RS256 — tại sao em chọn HS256?

Tiêu chí	HS256 (em dùng)	RS256
Loại key	Symmetric — 1 secret	Asymmetric — private key (sign) + public key (verify)
Secret	256-bit random string	RSA keypair 2048+ bit
Tốc độ sign	~0.05 ms	~5 ms (chậm hơn ~100×)
Tốc độ verify	~0.05 ms	~0.2 ms
Phù hợp	Sign và verify ở cùng 1 service	Nhiều service verify, 1 service sign (microservices)
Phân phối khoá	Chỉ 1 nơi cần secret	Public key có thể chia sẻ rộng

Em chọn HS256 vì: - BE chỉ là 1 monolith Express duy nhất — vừa sign (login/refresh) vừa verify (mỗi request) → không cần tách private/public. - HS256 nhanh hơn ~100× lúc sign → login/refresh response nhanh hơn vài ms. - Setup đơn giản: 1 secret trong `.env`, không cần generate keypair.

Vì sao 2 secret riêng cho access và refresh? - **Defense in depth:** nếu `JWT_SECRET` lộ qua log/git accidentally → attacker forge access token, NHƯNG không forge được refresh token (cần `JWT_REFRESH_SECRET` khác). Khi user logout, refresh bị xoá DB → access forge cũng vô hiệu sau ≤30 phút (vì middleware check DB). - **Domain separation:** access dùng cho mọi request (bề mặt rộng), refresh chỉ dùng cho 1 endpoint `/auth/refresh` (bề mặt hẹp). Tách secret = nếu compromise xảy ra thì biết chính xác phải rotate secret nào. - **Code rõ ý:** đọc code thấy `JWT_SECRET` vs `JWT_REFRESH_SECRET` biết ngay token nào → tránh nhầm khi refactor.

Sinh secret an toàn:

```
openssl rand -base64 48 # → 64-char base64 string ~ 384 bit entropy
# hoặc:
node -e "console.log(require('crypto').randomBytes(48).toString('base64'))"
```

Em ghi vào `.env.example` hướng dẫn sinh secret, không commit value thật.

Hạn chế trung thực — secret rotation chưa làm: - Nếu `JWT_SECRET` lộ, em chỉ có thể đổi giá trị mới + restart server → MỌI user đang đăng nhập bị 401 ngay (access token cũ ký bằng secret cũ → verify fail) → phải đăng nhập lại. - Lý tưởng: dual-secret window — chấp nhận token ký bằng `JWT_SECRET_OLD` trong 30 phút sau khi rotate, sau đó chỉ chấp nhận `JWT_SECRET_NEW`. Em ghi vào hướng phát triển.

Phản biện thường gặp: - “JWT có vulnerable alg: none không?” → “jsonwebtoken library version 9+ mặc định KHÔNG chấp nhận alg: none. Em đang dùng v9.0.2.” - “Secret 256-bit có đủ không?” → “Đủ. HMAC-SHA256 collision attack cần 2^{128} phép tính → không khả thi với hardware hiện tại.” - “Tại sao không dùng EdDSA (Ed25519) thay HS256?” → “EdDSA mới hơn nhưng jsonwebtoken chưa support trực tiếp, phải dùng jose library — em chưa migrate.”

A.23 — Socket.IO luồng end-to-end — connect, auth, notification, reconnect, force logout

Ngắn: FE dùng Socket.IO client kết nối tới BE qua port 4000 ngay sau khi đăng nhập. Token JWT được gửi qua auth handshake để xác thực 1 lần → mỗi user join vào “phòng riêng” `user_<id>` → BE service emit notification vào phòng đó → chỉ người dùng đó nhận.

Stack thư viện: - BE: `socket.io v4` (file `BE/src/utils/socketService.ts`) - FE: `socket.io-client v4` (file `FE/src/hooks/useSocket.ts`) - Cả 2 nói chuyện qua giao thức Socket.IO (HTTP long-polling + WebSocket upgrade) trên cùng port 4000 với REST API.

Server bootstrap (BE/src/index.ts):

```
import { createServer } from 'http';
import { initSocket } from './utils/socketService';

const httpServer = createServer(app); // Express app + Socket.IO chia chung HTTP
initSocket(httpServer); // attach Socket.IO vào HTTP server
httpServer.listen(PORT); // listen 1 port duy nhất
```

→ Cùng 1 port 4000 phục vụ cả GET /api/personnel (HTTP) lẫn wss://host:4000/socket.io/?EIO=4&... (WebSocket).

Init Socket.IO server (socketService.ts:25-67):

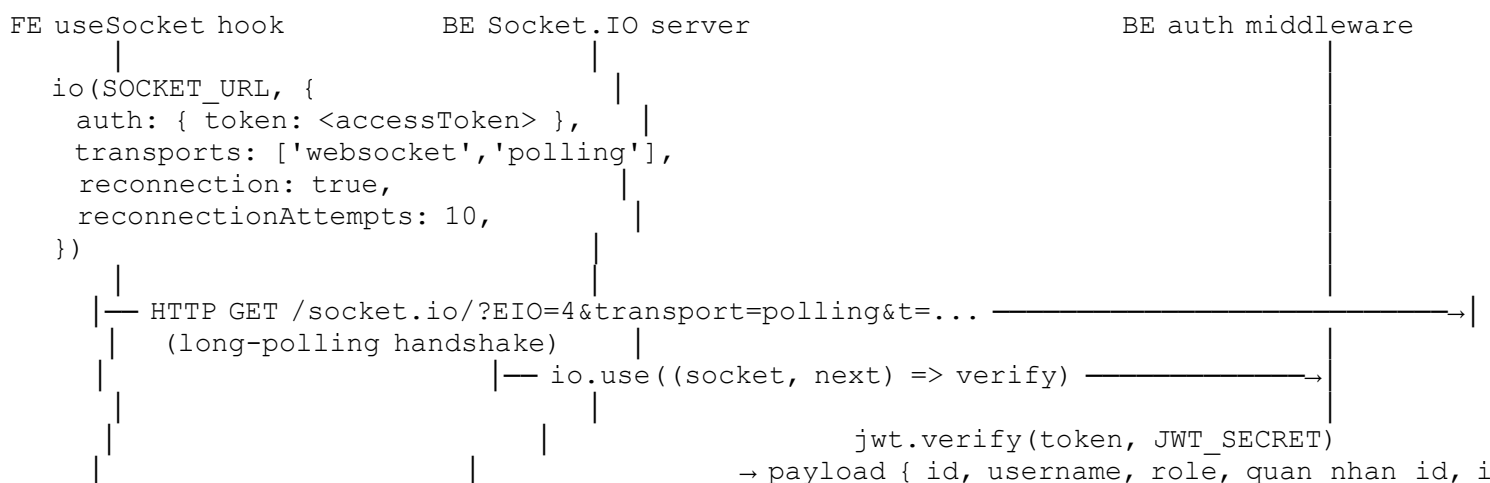
```
io = new Server(httpServer, {
  cors: { origin: allowCorsOrigin, credentials: true },
  pingTimeout: 60000, // sau 60s không nhận pong từ client → kill connection
  pingInterval: 25000, // 25s gửi ping 1 lần để giữ kết nối sống
});

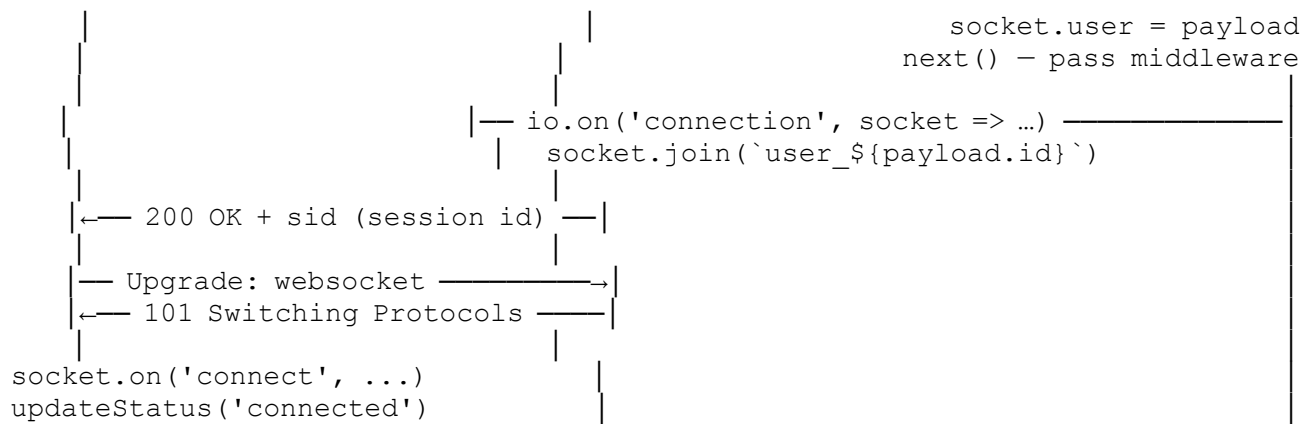
// Middleware xác thực - chạy 1 lần khi handshake, không phải mỗi event
io.use((socket, next) => {
  const token = socket.handshake.auth.token; // lấy token từ FE gửi trong `auth: {`
  if (!token) return next(new Error('Không tìm thấy token'));
  try {
    const decoded = jwt.verify(token, JWT_SECRET); // CÙNG secret như middleware HTTP
    socket.user = decoded; // attach payload vào socket cho dùng
    next();
  } catch (error) {
    if (error instanceof jwt.TokenExpiredError) {
      return next(new Error('TOKEN_EXPIRED')); // signal đặc biệt cho FE biết phải refresh token
    }
    next(new Error('Token không hợp lệ'));
  }
});

io.on('connection', (socket) => {
  const userId = socket.user.id;
  socket.join(`user_${userId}`); // mỗi user 1 room riêng theo id
  socket.on('disconnect', () => socket.leave(`user_${userId}`));
});
```

Sequence diagram chi tiết:

Pha 1 — Connect và join room





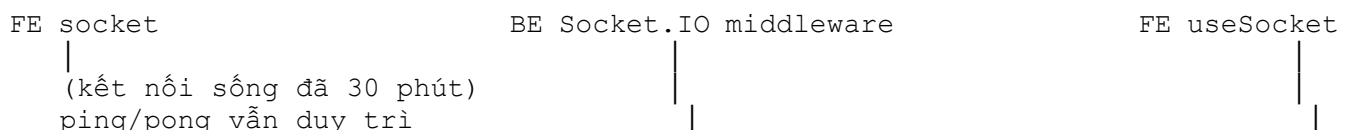
Pha 2 — Server emit notification → FE nhận Ví dụ thực: Manager phê duyệt đề xuất → user người gửi nhận thông báo realtime.

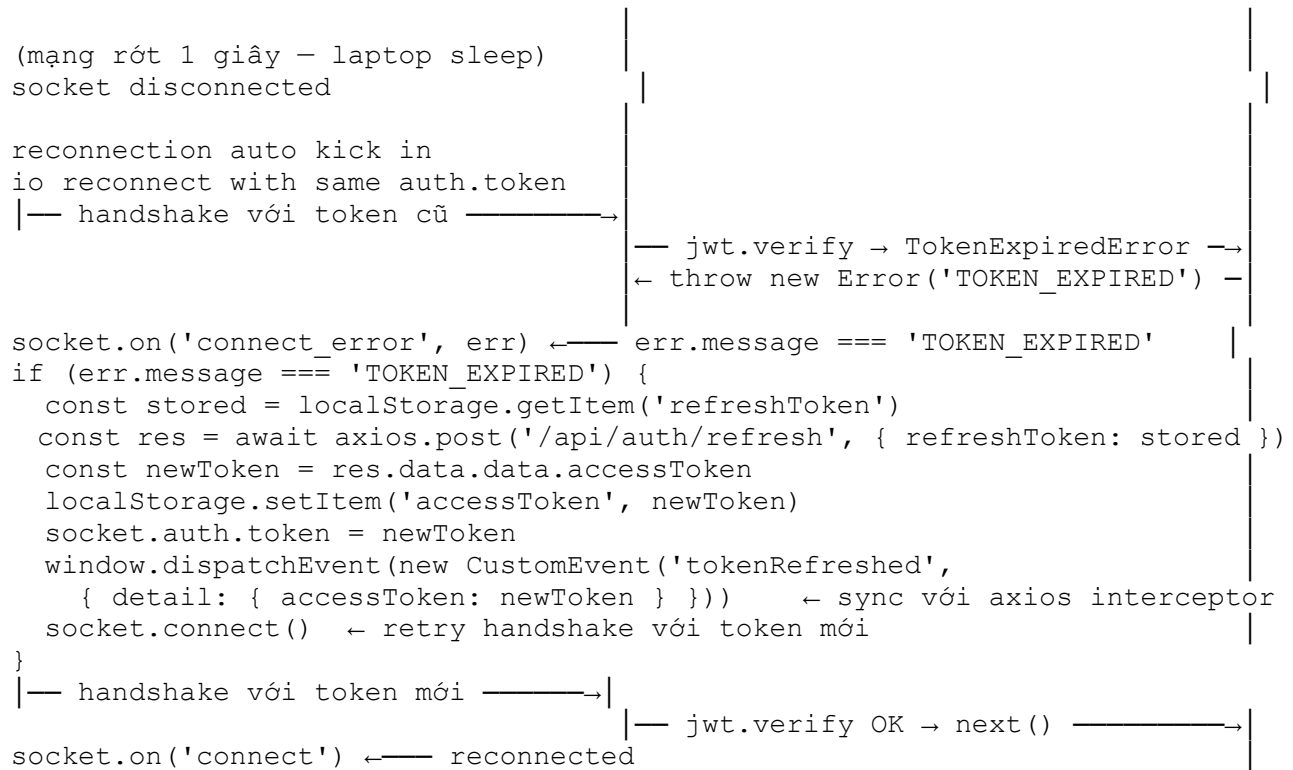


File: BE/src/helpers/notification/helpers.ts:94, 104, BE/src/utils/socketService.ts:73-76 (emitNotificationToUser).

Vì sao dùng room user_<id> thay vì broadcast? - Mỗi notification có người nhận cụ thể (nguoi_nhan_id). Broadcast → mọi user thấy → leak data. - Room cho phép gửi targeted: io.to('user_X').emit(...) chỉ tới socket trong room đó. Nếu user X có 2 tab mở (cùng login) → cả 2 tab đều nhận (vì 2 socket cùng join user_X).

Pha 3 — Access token hết hạn giữa kết nối Socket.IO Đây là edge case quan trọng: socket đang connected, token expire (sau 30 phút) → server reject reconnect.

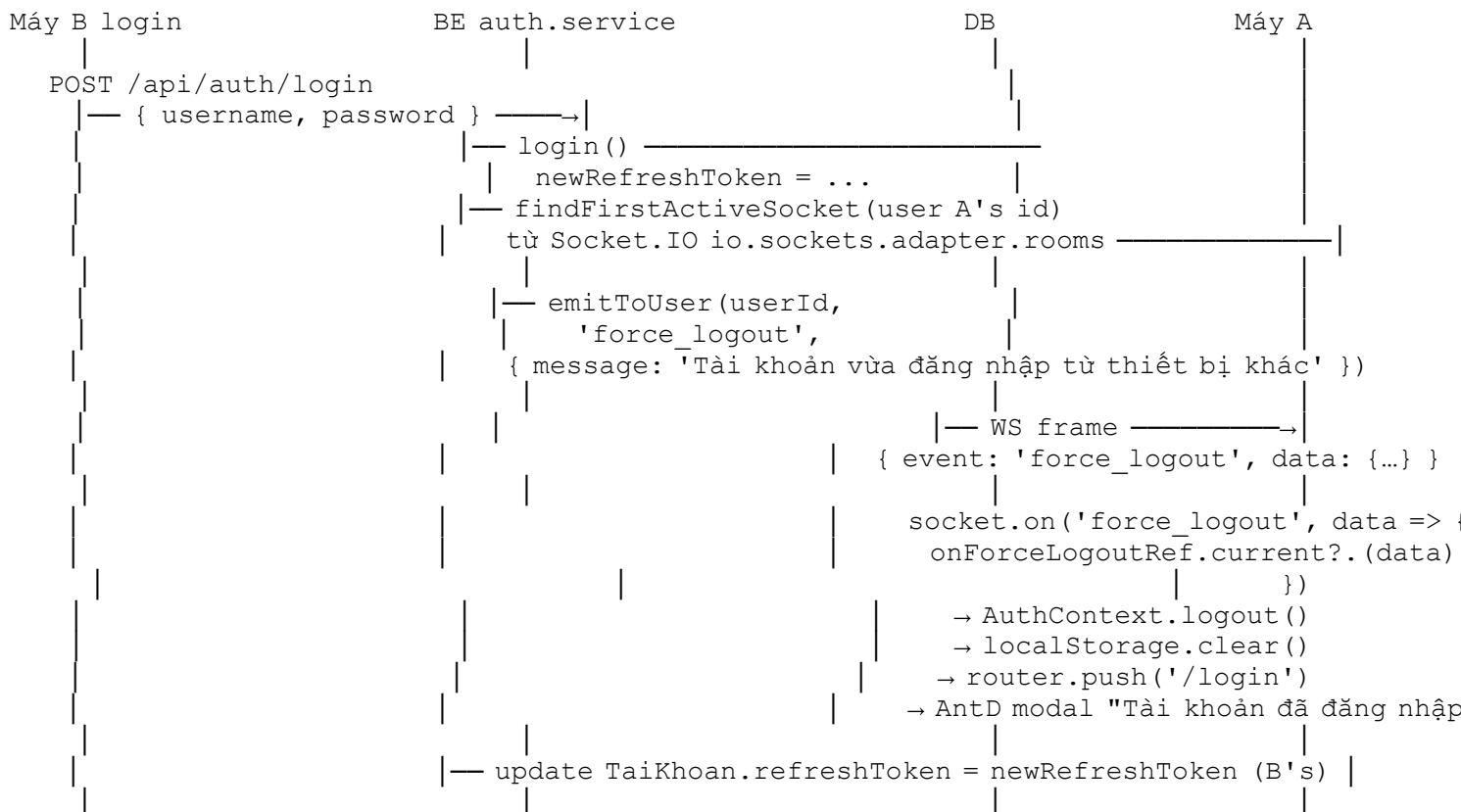




File: FE/src/hooks/useSocket.ts:65-90.

Custom event tokenRefreshed đáng chú ý: Khi axios interceptor (REST flow) refresh token, nó dispatch event này → useSocket lắng nghe → update socket.auth.token luôn → khỏi phải refresh 2 lần (1 bởi axios, 1 bởi socket).

Pha 4 — Force logout cross-device Tình huống: User đăng nhập máy A → đăng nhập máy B → máy A phải bị đẩy ra ngay.

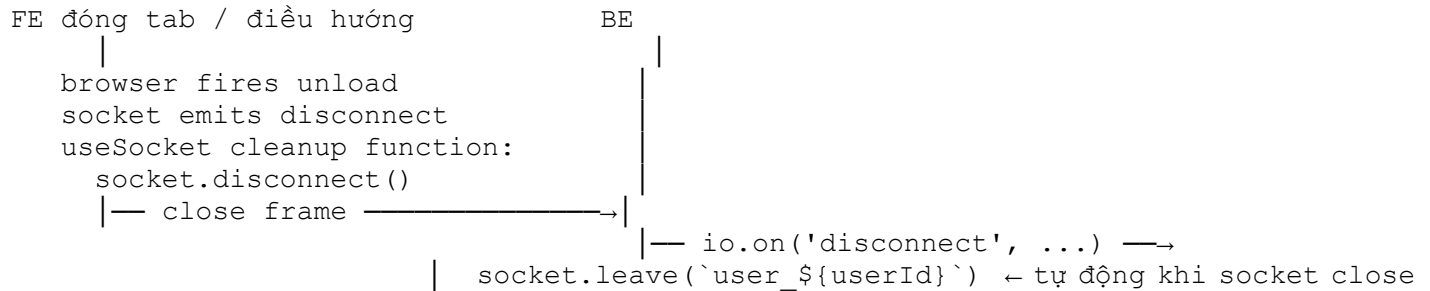


|← 200 + tokens —————|

File: BE/src/services/auth.service.ts(login), FE/src/hooks/useSocket.ts:90(socket.on('force_logout', ...))

Ngay cả khi máy A offline lúc B login (không nhận được force_logout): khi máy A online lại + access token expire + gọi refresh → BE thấy `account.refreshToken !== request.refreshToken` (vì đã bị B ghi đè) → 401 → axios interceptor force logout → cùng kết quả.

Pha 5 — Disconnect



Giao thức transport — vì sao có cả websocket và polling?

Socket.IO mặc định thử upgrade lên WebSocket sau khi handshake bằng HTTP polling. FE em config transports: `['websocket', 'polling']` — thử WebSocket trước, fallback polling nếu LAN/firewall chặn.

Cấu hình reconnection (useSocket.ts:48-54):

```
{
  reconnection: true,
  reconnectionDelay: 1000, // delay đầu tiên 1s
  reconnectionDelayMax: 5000, // backoff tối đa 5s
  reconnectionAttempts: 10, // bỏ cuộc sau 10 lần thử
}
```

→ Wifi rớt 30 giây? Socket.IO tự reconnect 10 lần với delay 1s, 1s, 2s, 4s, 5s, 5s, ... → tổng ~35 giây. Sau đó FE hiển thị badge “Disconnected” — user vẫn dùng REST được (chỉ mất realtime).

Hạn chế trung thực: - Em chưa scale ngang Socket.IO. Nếu deploy 2 instance BE, room `user_X` ở instance 1 không thấy được user emit từ instance 2. Cần `socket.io-redis-adapter` để pub/sub event qua Redis. Đã ghi vào hướng phát triển. - Polling fallback nuốt nhiều CPU/memory hơn WebSocket. Trên LAN nội bộ Học viện, WebSocket luôn work → fallback hiếm khi kích hoạt.

Phản biện: - “Sao không lưu auth token vào cookie httpOnly thay handshake auth: {token}?” → “Cookie tự gửi mỗi request, an toàn hơn localStorage, nhưng cross-origin (FE 3000 ↔ BE 4000) cần config phức tạp + CSRF token. Em chọn handshake auth cho đơn giản, chấp nhận rủi ro XSS.” - “Vì sao verify token ở handshake mà không verify lại mỗi event?” → “Verify mỗi event tốn ~0.5 ms × hàng nghìn event/s → quá tốn. Token chỉ verify lúc connect; nếu user bị revoke giữa session → BE đóng connection bằng `socket.disconnect()` từ event handler `force_logout` flow.” - “Có rate-limit Socket.IO event không?” → “Chưa. Socket.IO có thư viện `socket.io-rate-limiter` nhưng em chưa setup vì user nội bộ. Đã ghi hướng phát triển.”

B. Kiến trúc và design pattern

B.1 — Mô tả kiến trúc tổng thể trong 1 phút

Trả lời mẫu: “Hệ thống chia thành ba tầng. Tầng frontend là Next.js 14 App Router chạy trên cổng 3000. Tầng backend là Express + TypeScript chạy trên cổng 4000, được tổ chức theo 6 lớp: Route → Middleware → Controller → Service → Repository → Prisma. Tầng dữ liệu là PostgreSQL 15. Hai bên FE và BE giao tiếp qua REST API và Socket.IO cho thông báo thời gian thực. Toàn bộ vận hành trên LAN của Học viện qua PM2 và Nginx reverse proxy.”

B.2 — Tại sao layered architecture mà không phải MVC pure?

Ngắn: MVC truyền thống ghép chặt View với Controller. Project em có FE riêng (Next.js), BE thuần là REST API → MVC mất ý nghĩa. Layered cho phép tách rõ trách nhiệm BE.

Chi tiết: - Route chỉ định nghĩa HTTP path + chain middleware — không có logic. - Middleware làm cross-cutting concern: auth, validate, audit log, unit filter. - Controller chỉ parse request → gọi service → format response qua ResponseHelper. Body controller không quá 15 dòng (anti-pattern AP-2). - Service chứa business logic; được chia nhỏ thành sub-folder khi > 800 LOC (vd: services/proposal/approve/). - Repository wrap Prisma, expose method ngữ nghĩa (accountRepository.findUniqueRaw) thay vì prisma.taiKhoan.findUnique rải khắp. - Prisma là tầng cuối, không bị gọi trực tiếp từ Controller (anti-pattern AP-1).

B.3 — Lợi ích của Repository Layer? Có overengineering không?

Ngắn: Repository giúp Service không phụ thuộc vào Prisma cụ thể, dễ thay ORM trong tương lai và dễ mock trong test.

Chi tiết: - Trước khi có Repository (commit 9bd12f6), Service gọi prisma.quanNhan.findMany(...) trực tiếp. Test phải mock prisma global → fragile. - Sau Repository: Service gọi quanNhanRepository.findActiveInUnit(unitId) Test mock quanNhanRepository.findActiveInUnit đơn giản hơn. - **Có overengineering không?** Em thừa nhận với endpoint CRUD đơn giản, Repository chỉ là proxy mỏng. Nhưng với module phức tạp (proposal, profile, eligibility) thì lợi ích rõ.

Phản biện: “Sao không dùng Active Record như Sequelize?” → “Active Record gắn data và behavior chung — service layer mất tác dụng. Em chọn anemic model.”

B.4 — Strategy Pattern cho 7 loại đề xuất — kẻ chỉ tiết

Ngắn: Mỗi loại đề xuất có 1 class implement interface ProposalStrategy với 4 method chuẩn (buildSubmitPayload, validateApprove, importInTransaction, buildSuccessMessage). Một REGISTRY map enum loại → instance. Caller dispatch qua getStrategy(type).method(...).

Chi tiết code:

```
// services/proposal/strategies/proposalStrategy.ts
export interface ProposalStrategy {
  buildSubmitPayload(input: SubmitInput): Promise<SubmitPayload>;
  validateApprove(ctx: ApproveContext): Promise<void>;
  importInTransaction(tx: Prisma.TransactionClient, items: ImportItem[]): Promise<void>;
  buildSuccessMessage(result: ImportResult): string;
}

// services/proposal/strategies/index.ts
const REGISTRY: Record<ProposalType, ProposalStrategy> = {
  CA_NHAN_HANG_NAM: caNhanHangNamStrategy,
  DON_VI_HANG_NAM: donViHangNamStrategy,
  NIEN_HAN: nienHanStrategy,
  HC_QKQT: hcQkqtStrategy,
  KNC: kncStrategy,
  CONG_HIEN: congHienStrategy,
  NCKH: nckhStrategy,
};

export function getStrategy(type: ProposalType): ProposalStrategy {
  const strategy = REGISTRY[type];
  if (!strategy) throw new Error(`Không có strategy cho loại đề xuất ${type}`);
  return strategy;
}
```

Lợi ích so với if/else 7 nhánh: - Thêm loại mới: tạo 1 file <type>Strategy.ts + thêm 1 dòng vào REGISTRY. Không động đến if/else chính. - Test 1 strategy không kéo theo 6 strategy khác. - Hai strategy “single medal” (HC_QKQT,

KNC) chia sẻ qua helper `singleMedalImporter` → DRY mà vẫn rõ ràng.

Phản biện: “Sao không dùng class abstract?” → “Interface đủ ràng buộc; class abstract sẽ kéo theo state mà strategy không cần.”

B.5 — Khi nào tách module? Tiêu chí cụ thể?

Ngắn: File > 500 LOC xem xét tách. File > 800 LOC bắt buộc tách concern. File > 1000 LOC như `approve.ts` (2001 LOC trước refactor) phải tách thành `<feature>.ts` orchestration mỏng + sub-folder cho từng concern.

Pattern áp dụng:

```
services/proposal/
├── approve.ts           # < 500 LOC, public API + flow chính
├── approve/
│   ├── types.ts        # Shared interfaces
│   ├── validation.ts   # Pre-flight checks
│   ├── decisionMappings.ts # Decision metadata + PDF persist
│   └── import.ts       # Transactional import dispatch
```

B.6 — Anti-pattern em đã chủ động tránh

Em viết sẵn 9 anti-pattern trong `BE-QLKT/CLAUDE.md` từ AP-1 đến AP-9: - AP-1: Controller gọi Prisma trực tiếp. - AP-2: Controller chứa business logic > 15 dòng. - AP-3: Helper gọi DB hoặc service (helper phải pure). - AP-4: Duplicate logic ở nhiều service không extract. - AP-5: Service gọi service vòng tròn hoặc chuỗi quá 3 cấp. - AP-6: Hardcoded role/status/danh hiệu thay vì import từ `constants/`. - AP-7: Response không qua `ResponseHelper`. - AP-8: `prisma db push` cho cột có data → mất data. - AP-9: Catch error rồi đẩy detail kỹ thuật vào message cho user.

C. Bảo mật ứng dụng web

C.1 — IDOR (Insecure Direct Object Reference): nếu user đoán URL `/api/personnel/123` của người khác?

Đây là câu hỏi rất hay bị hỏi. Trả lời chuẩn:

Ngắn: Em chống IDOR bằng 3 lớp: (1) middleware `verifyToken` chặn request không có JWT, (2) `requireRole` chặn vai trò thấp truy cập endpoint không được phép, (3) trong service em check ownership: nếu USER thì chỉ trả dữ liệu khớp `req.user.quan_nhan_id`; nếu MANAGER thì lọc theo cây đơn vị qua `unitFilter`.

Chi tiết các trường hợp:

Tình huống	Ai gọi	Có chặn được không	Cơ chế
GET <code>/api/personnel/<other_id></code>	USER	Có — service kiểm if (<code>user.role === USER && id !== user.quan_nhan_id</code>) throw <code>ForbiddenError</code>	Ownership check trong service
GET <code>/api/personnel/<id_outside_unit></code>	MANAGER	Có — <code>unitFilter</code> middleware tính cây đơn vị từ <code>req.user.quan_nhan_id</code> , sau đó service WHERE id IN (<code>personnelIds</code>)	Unit filter
POST <code>/api/proposals</code> (tạo đề xuất)	USER	Có — <code>requireManager</code> reject 403 trước khi vào controller	Role middleware
DELETE <code>/api/personnel/<id></code>	MANAGER	Có — <code>requireAdmin</code> reject	Role middleware

Tình huống	Ai gọi	Có chặn được không	Cơ chế
GET /api/profiles/annual/<other_id>	USER	Có — service check id !== req.user.quan_nhan_id	Ownership

Code thật từ `middlewares/auth.ts`:

```
const checkRole = (allowedRoles: string[]) => {
  return (req: Request, res: Response, next: NextFunction): void => {
    if (!req.user) {
      res.status(401).json({ success: false, message: 'Vui lòng đăng nhập trước.' });
      return;
    }
    if (!allowedRoles.includes(req.user.role)) {
      res.status(403).json({
        success: false,
        message: `Chỉ ${allowedRoles.join(', ')} mới có quyền thực hiện thao tác này.`
      });
      return;
    }
    next();
  };
};
```

```
const requireSuperAdmin = checkRole([ROLES.SUPER_ADMIN]);
const requireAdmin = checkRole([ROLES.SUPER_ADMIN, ROLES.ADMIN]);
const requireManager = checkRole([ROLES.SUPER_ADMIN, ROLES.ADMIN, ROLES.MANAGER]);
```

Code thật từ `middlewares/unitFilter.ts`:

```
const attachUnitFilter = async (req, res, next) => {
  if (req.user?.role !== ROLES.MANAGER) {
    req.unitFilter = null;
    return next();
  }
  const quanNhanId = req.user?.quan_nhan_id;
  // Lấy đơn vị TỪ DB của user, KHÔNG lấy từ query → không bị giả mạo
  const unitInfo = await getUnitInfo(quanNhanId);
  req.unitFilter = unitInfo;
  next();
};
```

Ownership check dùng chung — `personnelService.assertCanViewPersonnel`: Để tránh lặp logic ở nhiều endpoint, em gom kiểm tra quyền xem hồ sơ của một quân nhân vào một hàm dùng chung. ADMIN/SUPER_ADMIN xem tất cả; USER chỉ xem đúng `quan_nhan_id` của mình; MANAGER chỉ xem quân nhân trong cây đơn vị (CQDV cha + các đơn vị trực thuộc con). Hàm này được gọi ở `getPersonnelById`, ở 3 endpoint GET `/api/profiles/{annual,tenure,contribution}/:id`, và ở các nested route GET `/api/personnel/:id/{annual,rewards,position-history,scientific-achievements}` (qua middleware `requireCanViewPersonnel`).

// services/personnel.service.ts

```
async assertCanViewPersonnel(personnelId, userRole, userQuanNhanId, preloadedTarget?) {
  if (userRole === ROLES.SUPER_ADMIN || userRole === ROLES.ADMIN) return;
  if (userRole === ROLES.USER) {
    if (userQuanNhanId !== personnelId) throw new ForbiddenError('Bạn không có quyền xem');
    return;
  }
}
```

```

if (userRole === ROLES.MANAGER && userQuanNhanId) {
  const target = preloadedTarget ?? (await quanNhanRepository.findUnitScope(personnelId));
  const manager = await quanNhanRepository.findUnitScope(userQuanNhanId);
  // ... cho phép nếu target thuộc CQDV của manager HOẶC một đơn vị trực thuộc con
}
}

```

Phản biện thường gặp: - “Nếu MANAGER tự đổi req.unitFilter được không?” → “Không. Middleware tính từ req.user.quan_nhan_id, mà req.user được decode từ JWT chữ ký bằng JWT_SECRET. Sửa JWT phải biết secret server.” - “Tự sửa JWT bằng cách đổi role thành ADMIN?” → “JWT có chữ ký HMAC; sửa payload không update chữ ký → jwt.verify fail.” - “Endpoint hồ sơ /api/profiles/* chỉ có verifyToken, sao đủ?” → “Role guard nằm ở tầng service qua assertCanViewPersonnel chứ không phải ở route, vì cùng một endpoint phục vụ cả USER (xem mình), MANAGER (xem đơn vị) và ADMIN (xem tất cả) — phân quyền theo dữ liệu nên phải check trong service, không thể chặn cứng bằng requireRole ở route.”

C.2 — SQL Injection: làm sao chống?

Ngắn: Prisma tự parameterize mọi truy vấn — prisma.user.findMany({ where: { name: input } }) sinh SELECT ... WHERE name = \$1 chứ không nối chuỗi. Chỗ duy nhất em dùng raw là \$queryRaw template literal — vẫn parameterize tự động vì là tagged template.

Chi tiết:

```

// AN TOÀN — Prisma parameterize tự động
await prisma.quanNhan.findMany({ where: { ho_ten: { contains: userInput } } });

// AN TOÀN — $queryRaw tagged template parameterize $1, $2
const userId = '...';
await prisma.$queryRaw`SELECT * FROM "QuanNhan" WHERE id = ${userId}`;

// NGUY HIỂM — $queryRawUnsafe nối chuỗi
await prisma.$queryRawUnsafe(`SELECT * FROM "QuanNhan" WHERE id = '${userId}'`);

```

Em chỉ dùng \$queryRawUnsafe ở đúng 1 chỗ trong scripts/renameColumn.ts — chạy local một lần để rename column, không nhận input từ user.

Phản biện: “Validate input có cần thiết khi đã có Prisma?” → “Có, vì validation còn để chặn business rule (vd: năm phải 1900-2100), không chỉ chống SQLi.”

C.3 — XSS (Cross-Site Scripting)

Ngắn: React tự escape mọi text khi render qua {value} → script tag bị render thành text. Em không dùng dangerouslySetInnerHTML ở bất cứ đâu trừ chỗ render PDF preview (đã sanitize).

Chi tiết: - **Stored XSS:** ai đó nhập <script>alert(1)</script> vào trường ghi chú → React render thành text literal, không execute. - **Reflected XSS:** error message từ server dạng “Không tìm thấy <input>” → React escape khi render trong <Alert>. - **DOM-based XSS:** em không dùng eval(), new Function(), hay gán innerHTML từ input người dùng. Chỗ duy nhất sinh HTML thủ công là cửa sổ xem trước PDF (lib/file/filePreview.ts) dùng newWindow.document.write(...). Tại đây tên file (lấy từ **số quyết định** do người dùng nhập) được escape trước khi nội suy: hàm escapeHtml() cho phần markup (<title>, tên file), và JSON.stringify + chặn </script> (hàm toJsString()) cho phần nhúng trong <script>.

```

// lib/file/filePreview.ts — escape trước khi document.write
const safeFilename = escapeHtml(filename); // & < > " '
// trong <script>: link.download = ${toJsString(filename)} // JSON.stringify + </>

```

Lưu ý khi bảo vệ: đây là một lỗi DOM-XSS thật em tìm ra khi tự rà soát (số quyết định kiểu </title> sẽ chạy mã trong tab xem PDF) và đã vá. Xem thêm mục C.17.

Header bảo vệ thêm: em đặt helmet() trong index.ts để set X-Content-Type-Options: nosniff, X-Frame-Options: DENY. helmet() bật mặc định nhưng CSP chưa được cấu hình riêng.

C.4 — CSRF (Cross-Site Request Forgery)

Ngắn: Em dùng JWT trong header `Authorization: Bearer <token>` thay vì cookie. Browser không tự gửi header `Authorization` cho cross-origin request → CSRF không lợi dụng được.

Chi tiết: - Nếu lưu JWT trong `localStorage`: kẻ tấn công không đọc được vì same-origin policy. - Nếu lưu trong cookie `HttpOnly`: phải bật `SameSite=Strict` để chống CSRF. - Em chọn header → tránh hoàn toàn CSRF.

Trade-off: XSS sẽ đọc được `localStorage`. Em mitigate bằng React tự escape + không dùng `eval` — XSS gần như không thực hiện được.

C.5 — Brute force password

Ngắn: `authLimiter` chặn sau 30 lần đăng nhập THẤT BẠI trong 5 phút / IP cho `/api/auth/login` (chỉ đếm lần thất bại) → 429 Too Many Requests.

Code thật:

```
// configs/rateLimiter.ts
export const authLimiter = rateLimit({
  windowMs: 5 * 60 * 1000,
  max: 30,
  message: { success: false, message: 'Quá nhiều yêu cầu, thử lại sau ít phút.' },
  skipSuccessfulRequests: true,
});
```

Phản biện: “Kẻ tấn công đổi IP qua proxy thì sao?” → “Đúng, em mới chống brute đơn giản. Nâng cấp sẽ có account lockout sau N lần sai liên tiếp.”

C.6 — Mật khẩu lưu trữ thế nào?

Ngắn: Hash bằng `bcrypt` cost 10 (~100 ms/hash). Không lưu plaintext. Không log password vào audit log.

Cost = 10 nghĩa là gì: $2^{10} = 1024$ vòng key derivation. Mỗi hash mất ~100 ms trên CPU thông thường.

So sánh chi phí vs an toàn:

Cost	Thời gian / hash	Đánh giá
8	~25 ms	Yếu — attacker brute ~1000 password/giây
10	~100 ms	Acceptable — em đang dùng
12	~300 ms	Strong nhưng login chậm
14	~1 s	Quá chậm cho UX

Tự động tăng cost theo thời gian: Mỗi vài năm tăng cost 1 đơn vị khi CPU mạnh hơn. Hash cũ vẫn verify được vì cost được embedded trong hash (`$2b$10$...`).

Phản biện: “Sao không Argon2 mạnh hơn?” → “Argon2 mạnh hơn nhưng `bcrypt` đủ cho ngữ cảnh LAN nội bộ. Đổi `bcrypt` → Argon2 sau để — chỉ thay 1 helper.”

C.7 — File upload: chống upload file độc hại

Ngắn: `multer` config giới hạn (1) MIME type whitelist qua helper `createFileFilter`, (2) size max 10 MB cho phần lớn (50 MB cho ad-hoc award), (3) tách 2 storage strategy: `memoryStorage` cho file xử lý ngắn (parse Excel), `diskStorage` cho file cần lưu lâu dài (PDF quyết định) — lưu vào thư mục riêng ngoài web root để không bị execute như script.

Code thật (configs/multer.ts):

```
const createFileFilter = (allowedMimes: string[], errorMsg: string) =>
  (req, file, cb: FileFilterCallback) => {
    if (allowedMimes.includes(file.mimetype)) cb(null, true);
```

```

    else cb(new Error(errorMsg));
  };

// Excel parse - memoryStorage vì xử lý xong là drop
export const excelUpload = multer({
  storage: multer.memoryStorage(),
  limits: { fileSize: 10 * MB },
  fileFilter: createFileFilter([MIME.XLSX, MIME.XLS], 'Chi chap nhan file Excel'),
});

// PDF quyết định - diskStorage vì cần lưu để download sau
export const pdfDecisionUpload = multer({
  storage: multer.diskStorage({ destination: './uploads/decisions', filename: ... }),
  limits: { fileSize: 10 * MB },
  fileFilter: createFileFilter([MIME.PDF], 'Chi chap nhan file PDF'),
});

```

Dùng helper `createFileFilter` thay vì inline → không lặp logic ở 6 multer config khác nhau (DRY).

Phản biện: “MIME type có thể bị giả?” → “Đúng. Em check thêm magic byte 4 byte đầu cho PDF (%PDF-) ở service. Chưa check magic byte cho .xlsx — sẽ bổ sung.”

C.8 — Path traversal trên API tải file PDF

Ngắn: Endpoint GET `/api/decisions/download/:soQuyetDinh` lookup `FileQuyếtDinh` theo `so_quyet_dinh` lấy `file_path` từ DB → không bao giờ nhận `file_path` trực tiếp từ user.

```

// services/decision.service.ts
const decision = await fileQuyếtDinhRepository.findBySoQuyếtDinh(soQuyếtDinh);
if (!decision?.file_path) throw new NotFoundError('File quyết định không tồn tại');
const safePath = path.join(__dirname, '../..../uploads', path.basename(decision.file_path));
res.download(safePath);

```

`path.basename` strip mọi `../` → không leak file ngoài thư mục `uploads/`.

C.9 — Privilege escalation: USER tự tăng role thành ADMIN

Ngắn: Role nằm trong JWT chữ ký, chỉ server biết secret. User không sửa được payload mà giữ chữ ký valid. Ngay cả khi user gửi field `role` trong body, Zod schema không khai báo field đó nên sẽ bị strip mặc định.

Bonus phòng: - Endpoint update profile cá nhân không cho update field `role` (Zod schema chỉ liệt kê `ho_ten`, `email`).
- Endpoint update tài khoản (PUT `/api/accounts/:id`) chỉ cho `requireSuperAdmin`.

C.10 — Mass assignment

Ngắn: Tất cả Zod schema dùng `z.object()` mặc định strip field ngoài khai báo. Không bao giờ truyền `req.body` thẳng vào `prisma.create()`.

```

// Zod config
z.object({
  ho_ten: z.string(),
  email: z.string().email().optional(),
});

// User gửi { ho_ten: 'A', role: 'ADMIN', is_super: true }
// → sau parse: { ho_ten: 'A' } - role và is_super bị strip

```

C.11 — Thông tin nhạy cảm trong response

- Không trả `password_hash` — Prisma select: `{ id, username, role }` không kèm `password_hash`.

- refreshToken được trả về trong body JSON khi đăng nhập (không dùng HttpOnly cookie); FE lưu ở client và gửi kèm khi gọi /refresh.
- Không trả CCCD đầy đủ cho USER — chỉ ADMIN/MANAGER xem được.

C.12 — DoS attack

Đa lớp: - express-rate-limit: authLimiter (30 request thất bại / 5 phút / IP cho route auth, chỉ đếm request lỗi); writeLimiter (30 request / 15 phút / IP cho write endpoint). - body-parser limit JSON 10 MB → tránh JSON bomb. - File upload limit 10 MB. - Pagination forced: MAX_LIMIT = 100 records/trang → không trả nhầm 100k records. - Slow loris: Nginx có timeout 30s.

Hạn chế: Layer 7 DDoS lớn cần WAF (Cloudflare) — không có vì project chạy LAN.

C.13 — Security headers

helmet() middleware đặt: - X-Content-Type-Options: nosniff - X-Frame-Options: DENY (chống clickjacking) - Strict-Transport-Security (HSTS) — bật khi có HTTPS - X-XSS-Protection: 1; mode=block - CSP: helmet() bật mặc định nhưng CSP chưa được cấu hình riêng.

C.14 — CORS cấu hình thế nào?

```
// configs/cors.ts — đọc whitelist từ process.env.ALLOWED_ORIGINS (comma-separated),
// mặc định 'http://localhost:3000,http://localhost:3001'
const allowedOrigins = process.env.ALLOWED_ORIGINS
  ? process.env.ALLOWED_ORIGINS.split(',').map(o => o.trim())
  : ['http://localhost:3000', 'http://localhost:3001'];

export function allowCorsOrigin(origin, callback) {
  if (!origin) return callback(null, true);
  if (allowedOrigins.includes(origin)) callback(null, true);
  else callback(new Error('Not allowed by CORS'));
}
```

Whitelist nhiều origin qua ALLOWED_ORIGINS → chỉ domain trong whitelist mới gọi được API.

C.15 — Audit log: ghi gì, không ghi gì?

Ghi: - nguoi_thuc_hien_id, actor_role, action (CREATE/UPDATE/DELETE/APPROVE/REJECT/IMPORT/...) - resource (vd: personnel, proposals) - tai_nguyen_id - description tiếng Việt - payload JSON: trạng thái before/after - ip_address, user_agent

Không ghi: - password_hash (kể cả khi UPDATE) - refreshToken - File content đính kèm (chỉ ghi tên file)

Resource backup ẩn với non-SUPER_ADMIN: trong systemLogs.service.ts:

```
if (userRole !== ROLES.SUPER_ADMIN) {
  whereClause.resource = { not: 'backup' };
}
```

C.16 — Reset password / quên mật khẩu

Hiện tại: chưa có flow self-service reset. SUPER_ADMIN/ADMIN dùng endpoint POST /api/accounts/reset-password (gửi username trong body) để reset thủ công.

Lý do: môi trường LAN nội bộ, người dùng có thể đến gặp ADMIN. Tránh phải chạy mail server trong LAN cô lập.

Hướng phát triển: thêm flow OTP qua SMS quân sự nội bộ.

C.17 — Em có tự rà soát bảo mật không? Tìm và sửa được lỗi gì?

Đây là câu rất “ăn điểm” nếu trả lời được cụ thể. Em đã chạy một đợt tự đánh giá code (security self-review) và phát hiện, và 4 nhóm vấn đề. Mỗi vấn đề em nêu được: lỗi là gì, kịch bản khai thác, và cách sửa.

1) DOM-based XSS ở cửa sổ xem trước PDF (lib/file/filePreview.ts). - *Lỗi:* tên file (bắt nguồn từ số quyết định người dùng nhập) được nội suy thẳng vào HTML qua document.write — cả trong <title>, thẻ hiển thị tên, và bên trong <script>(link.download = '...'). - *Khai thác:* tạo quyết định với số kiểu </title> → ai bấm “xem file” sẽ chạy mã lạ trong tab mới, đọc được accessToken ở localStorage. - *Sửa:* thêm escapeHtml() cho phần markup và toJsString() (JSON.stringify + chặn chuỗi </script>) cho phần trong <script>; gộp hàm xem PDF trùng lặp ở downloadDecisionFile.ts về dùng chung một hàm đã escape.

2) IDOR ở các endpoint hồ sơ (/api/profiles/* và nested /api/personnel/:id/*). - *Lỗi:* 3 endpoint GET /api/profiles/{annual,tenure,contribution}/:id và các nested GET (annual-rewards, position-history, scientific-achievements) chỉ có verifyToken, service không kiểm quyền sở hữu. - *Khai thác:* một USER gọi GET /api/profiles/annual/<id_người_khác> đọc được toàn bộ hồ sơ khen thưởng (gồm số quyết định BKBQP/CSTDTQ/BKTTCP) của bất kỳ ai. - *Sửa:* gom kiểm tra quyền vào hàm dùng chung assertCanViewPersonnel (USER xem mình, MANAGER theo cây đơn vị, ADMIN xem tất cả) và gọi ở cả controller hồ sơ lẫn middleware requireCanViewPersonnel cho nested route. Xem C.1.

3) Phân quyền + bỏ qua quy trình duyệt ở khen thưởng đơn vị (/api/unit-annual-awards). - *Lỗi:* endpoint direct-entry upsert (POST /, PUT /:id) ghi thẳng bản ghi status = APPROVED và mở cho cả MANAGER, không validate, không giới hạn đơn_vị_id theo đơn vị của MANAGER → MANAGER có thể tạo khen thưởng đã-duyet cho đơn vị bất kỳ, bỏ qua bước admin duyệt. - *Sửa:* upsert chuyển về ADMIN-only (direct-entry là thao tác của admin) + thêm Zod validation. propose (tạo bản ghi PENDING) giữ cho MANAGER nhưng thêm kiểm tra phạm vi đơn vị (assertCanManageUnit) + Zod. Luồng đề xuất thật của người dùng đi qua /api/proposals nên thay đổi này không ảnh hưởng FE.

4) Xác định đơn vị của quân nhân không nhất quán (DVTT vs CQDV). - *Bối cảnh:* quân nhân ở đơn vị trực thuộc (DVTT) được lưu cả hai khóa co_quan_don_vị_id (đơn vị cha) lẫn don_vị_truc_thuoc_id. Vài chỗ xác định “đơn vị của chính quân nhân” lại ưu tiên CQDV (co_quan_don_vị_id || don_vị_truc_thuoc_id) nên luôn trả về đơn vị cha. - *Sửa có chọn lọc:* ở chỗ nhận diện đơn vị của chính quân nhân (phát hiện chuyển đơn vị personnel/update.ts) đổi sang DVTT-first; ở bộ lọc theo don_vị_id (proposal/awards.ts) đổi sang khớp một trong hai khóa để đúng cho cả lọc theo CQDV lẫn DVTT. **Giữ nguyên** các chỗ xác định phạm vi/thông báo cho MANAGER, vì MANAGER quản ở cấp CQDV (quản cả đơn vị con) nên CQDV-first ở đó là đúng thiết kế — đây là điểm để sửa sai nếu “đào” đồng loạt.

Câu chốt: “Em xem việc tự tìm lỗi của chính mình là một phần của quy trình. 4 vấn đề trên đều đã vá, có kiểm chứng bằng typecheck + bộ test (937 test BE pass) và rà soát thủ công.”

Phản biện thường gặp: - “Còn lỗ hổng nào chưa vá không?” → “Em ghi nhận access token nằm ở localStorage (đánh đổi đã biết khi refresh token ở httpOnly cookie) và mật khẩu DevZone lưu base64 ở sessionStorage — em xếp vào hướng cải thiện, không phải lỗ hổng leo thang quyền vì BE re-validate mỗi request.”

D. Race condition và concurrency

D.1 — Hai user phê duyệt cùng đề xuất cùng lúc

Tình huống: Admin A và Admin B mở tab cùng đề xuất, bấm “Phê duyệt” gần như đồng thời.

Cơ chế chống: 1. **Transaction Prisma:** mở prisma.\$transaction(async tx => { ... }). 2. **Re-fetch trong transaction với lock:** tx.bangDeXuat.findUnique({ where: { id }, ... }) — Postgres mặc định READ COMMITTED, hai transaction đều đọc thấy status = PENDING. 3. **Kiểm tra status:** if (proposal.status !== 'PENDING') throw new ValidationError('Đã được duyệt'). 4. **Update có điều kiện:** tx.bangDeXuat.updateMany({ where: { id, status: 'PENDING' }, data: { status: 'APPROVED' } }) — nếu trả về count: 0 nghĩa là transaction kia đã update trước → throw conflict.

```
await prisma.$transaction(async tx => {
  const proposal = await tx.bangDeXuat.findUniqueOrThrow({ where: { id } });
  if (proposal.status !== PROPOSAL_STATUS.PENDING) {
    throw new ValidationError('Đề xuất đã được xử lý');
  }
});
```

```

}
const updated = await tx.bangDeXuat.updateMany({
  where: { id, status: PROPOSAL_STATUS.PENDING },
  data: { status: PROPOSAL_STATUS.APPROVED, nguoi_duyet_id, ngay_duyet: new Date() },
});
if (updated.count === 0) {
  throw new ValidationError('Đề xuất vừa được người khác xử lý');
}
// ... gán số quyết định, ghi nhật ký
});

```

Phản biện: “Sao không SELECT FOR UPDATE?” → “Có thể, nhưng updateMany có điều kiện đã đủ atomic ở mức row trong Postgres và không cần escape \$queryRaw.”

D.2 — Tạo 2 tài khoản cùng username cùng lúc

Cơ chế: cột TaiKhoan.username có @unique → DB-level unique constraint. Hai INSERT cùng username sẽ có 1 thành công, 1 lỗi P2002 (Unique constraint failed). Em catch và trả 409 Conflict.

```

try {
  return await prisma.taiKhoan.create({ data });
} catch (error) {
  if (error instanceof Prisma.PrismaClientKnownRequestError && error.code === 'P2002') {
    throw new ValidationError('Username đã tồn tại');
  }
  throw error;
}

```

Phản biện: “Sao không check trước rồi mới INSERT?” → “Check-then-insert vẫn race. Để DB unique làm trọng tài là chuẩn.”

D.3 — Import Excel: chèn dòng mới giữa preview và confirm

Tình huống: User upload Excel preview lúc 10:00. Lúc 10:01, có Admin khác tạo bản ghi (quan_nhan_id, nam) = (X, 2025) qua UI. Lúc 10:02, user bấm “Xác nhận import” — gập dòng (X, 2025) trùng.

Cơ chế: - Bước “xác nhận” mở 1 transaction Prisma duy nhất. - Mỗi tx.danhHieuHangNam.create({ data }) sẽ check constraint @unique([quan_nhan_id, nam]). - Nếu trùng → ném exception → toàn bộ transaction rollback. Không bản ghi nào vào DB. - Service trả về danh sách dòng lỗi cho user xem.

```

await prisma.$transaction(async tx => {
  for (const row of rows) {
    try {
      await tx.danhHieuHangNam.create({ data: row });
    } catch (e) {
      if (e.code === 'P2002') {
        throw new Error(`Dòng ${row._index}: trùng (quan_nhan_id=${row.quan_nhan_id}, nam=${row.nam})`);
      }
      throw e;
    }
  }
});

```

D.4 — Đếm so_luong đơn vị khi 2 quân nhân chuyển đơn vị cùng lúc

Tình huống: Quân nhân A chuyển từ Đơn vị X → Y, quân nhân B chuyển từ Y → X cùng thời điểm. Nếu code dùng 2 lần if:

```

if (oldUnit) await unitRepo.decrement(oldUnit, 1);
if (newUnit) await unitRepo.increment(newUnit, 1);

```

Có thể đếm sai khi 2 transaction đan xen.

Cơ chế: - Mỗi update đơn vị trong **một transaction riêng** với atomic increment/decrement của Prisma. - Prisma sinh UPDATE ... SET so_luong = so_luong + 1 — câu này atomic ở mức row trong Postgres.

```
await prisma.$transaction([
  prisma.coQuanDonVi.update({
    where: { id: oldUnit },
    data: { so_luong: { decrement: 1 } },
  }),
  prisma.coQuanDonVi.update({
    where: { id: newUnit },
    data: { so_luong: { increment: 1 } },
  }),
]);
```

Tuyệt đối tránh: đọc rồi tính rồi ghi:

```
// XẤU — race condition
const unit = await prisma.coQuanDonVi.findUnique({ where: { id: oldUnit } });
await prisma.coQuanDonVi.update({ where: { id: oldUnit }, data: { so_luong: unit.so_luong + 1 } });
```

Cách project thực sự đếm — single primary unit: Quân nhân chỉ thuộc **một** đơn vị “primary” (DVTT nếu có, ngược lại CQDV). Đếm so_luong chỉ chạm vào unit primary cũ và unit primary mới, **không** đếm chéo cả CQDV + DVTT. Code thật tại services/personnel/update.ts:

```
// DVTT takes priority over CQDV when determining effective unit
const oldPrimaryUnitId = oldDonViTrucThuocId || oldCoQuanDonViId;
const oldIsCqdv = !oldDonViTrucThuocId && !!oldCoQuanDonViId;
if (oldPrimaryUnitId) {
  await adjustUnitCount(prismaTx, oldPrimaryUnitId, oldIsCqdv, 'decrement');
}

const newPrimaryUnitId = newDonViTrucThuocId || newCoQuanDonViId;
const newIsCqdv = !newDonViTrucThuocId && !!newCoQuanDonViId;
if (newPrimaryUnitId) {
  await adjustUnitCount(prismaTx, newPrimaryUnitId, newIsCqdv, 'increment');
}
```

adjustUnitCount dispatch sang coQuanDonViRepository.{increment,decrement}SoLuong hoặc donViTrucThuocRepository.{increment,decrement}SoLuong tùy isCqdv. Bản thân repository dùng data: { so_luong: { increment/decrement: 1 } } — atomic SQL UPDATE ... SET so_luong = so_luong ± 1.

Rule trong CLAUDE.md: “Khi thay đổi đơn vị quân nhân, dùng if/else (chỉ increment/decrement 1 đơn vị), không dùng 2 if riêng biệt — tránh đếm dư”. Lý do: nếu cả 2 nhánh if cùng chạy (vd: cả oldUnit và newUnit đều có giá trị), không thận trọng có thể đếm dư khi chuyển nội bộ.

D.5 — Refresh token rotation race

Tình huống: Client gọi /api/auth/refresh 2 lần liên tiếp do retry → 2 refresh token mới được sinh, token nào mới hơn thắng.

Cơ chế trong project: - accountRepository.update(id, { refreshToken: newToken }) là atomic UPDATE. - Nếu request 2 đến sau request 1: refreshToken ghi đè → request 1 còn dùng được token mới? Không, vì client chỉ giữ token cuối cùng nhận. - Server không reject token cũ cho đến khi expire (7 ngày) — đây là rủi ro.

Cải tiến đã ghi vào hướng phát triển: sliding window rotation với grace period 30s + blacklist.

D.6 — Backup chạy trùng

Cơ chế: Cron `backup.service.ts` dùng singleton flag trong process. Nếu cron tích hoạt khi job trước đang chạy → skip.

```
let isRunning = false;
async function runBackup() {
  if (isRunning) return;
  isRunning = true;
  try {
    await buildBackupSql(); // tự build INSERT (raw SQL text)
  } finally {
    isRunning = false;
  }
}
```

Hạn chế: flag chỉ trong 1 process. PM2 cluster mode sẽ có nhiều process → cần Redis lock. Hiện em chạy `pm2 ... -i 1` (single instance).

D.7 — Recalc trùng lặp gây sai dữ liệu

Cơ chế: `recalculateAnnualProfile(personnelId)` là **idempotent** — đọc dữ liệu nguồn → tính → upsert hồ sơ suy diễn `HoSoHangNam`. Chạy nhiều lần liên tiếp cho ra cùng kết quả.

```
async function recalculateAnnualProfile(personnelId) {
  const danhHieus = await prisma.danhHieuHangNam.findMany({ where: { quan_nhan_id: personnelId } });
  const context = computeChainContext(danhHieus);
  const flags = computeEligibilityFlags(context);
  await prisma.hoSoHangNam.upsert({
    where: { quan_nhan_id: personnelId },
    create: { quan_nhan_id: personnelId, ...flags },
    update: flags,
  });
}
```

upsert giải quyết race “create-or-update”.

D.8 — Approve và xoá đề xuất cùng lúc

Tình huống: ADMIN A bấm Phê duyệt; ADMIN B bấm Xoá cùng lúc.

Cơ chế: - Approve mở transaction → fetch → check status. Nếu lúc đó B đã DELETE: `findUniqueOrThrow` ném `P2025 (Record not found)` → throw `NotFoundError`. - Delete cũng dùng `deleteMany({ where: { id, status: 'PENDING' } })` — `count=0` nghĩa là proposal đã được duyệt → trả lỗi.

D.9 — Hai đề xuất khác nhau dùng cùng `so_quyet_dinh` đồng bộ vào `FileQuyếtĐịnh` (check-then-create race)

Tình huống: Trong quy trình phê duyệt, sau khi ghi danh hiệu/khen thưởng, hệ thống đồng bộ một bản ghi vào bảng `FileQuyếtĐịnh` để tra cứu tệp PDF sau này (`services/proposal/approve/decisionMappings.ts`). Khoá duy nhất là `so_quyet_dinh`. Khi Admin A duyệt đề xuất 1 và Admin B duyệt đề xuất 2 mà hai đề xuất gắn cùng số quyết định (ví dụ: gộp nhiều quân nhân vào cùng 1 quyết định) — hai transaction cùng chạy đoạn:

```
// ❌ XẤU — check-then-create race
const existing = await tx.fileQuyếtĐịnh.findUnique({ where: { so_quyet_dinh } });
if (!existing) {
  await tx.fileQuyếtĐịnh.create({ data: { so_quyet_dinh, ... } }); // ← P2002 ở tx thứ 2
} else if (!existing.file_path && newFilePath) {
  await tx.fileQuyếtĐịnh.update({ where: { so_quyet_dinh }, data: { file_path: newFilePath } });
}
```

Cả hai transaction đều thấy `existing = null` (READ COMMITTED không thấy row chưa commit của tx kia), cả hai cùng `create` → tx commit sau gặp P2002 (Unique constraint). Catch block log lỗi nhưng nuốt → đồng bộ nửa chừng, lịch sử kiểm toán không khớp.

Cơ chế chống — atomic upsert + conditional backfill:

```
// □ TỐT — upsert là một câu UPSERT atomic ở DB
await tx.fileQuyetDinh.upsert({
  where: { so_quyet_dinh },
  create: {
    so_quyet_dinh,
    nam: proposal.nam,
    ngay_ky,
    nguoi_ky,
    file_path,
    loai_khen_thuong,
    ghi_chu: `Tự động đồng bộ từ đề xuất ${proposalId}`,
  },
  update: {}, // no-op: KHÔNG ghi đề ngay_ky / nguoi_ky của tx đầu tiên
});

// Backfill file_path chỉ khi row cũ còn null và tx hiện tại có path mới.
// updateMany với where lồng điều kiện là atomic: chỉ update đúng các row khớp.
if (file_path) {
  await tx.fileQuyetDinh.updateMany({
    where: { so_quyet_dinh, file_path: null },
    data: { file_path },
  });
}
```

Tại sao split thành 2 câu thay vì 1 upsert có update đầy đủ? - `upsert.update` chạy luôn khi row đã tồn tại, không có điều kiện. Nếu để `update: { file_path }` thì tx sau sẽ ghi đề `file_path` của tx trước — không an toàn nếu tx trước cũng đã set. - Tách `updateMany` với `where: { file_path: null }` đảm bảo backfill chỉ xảy ra một lần, và chỉ khi cần. - `update: {}` (no-op) khiến upsert trở thành “ensure-row-exists” thuần túy — đây là pattern an toàn cho ON CONFLICT DO NOTHING.

Phản biện 1: “Sao không dùng `INSERT ... ON CONFLICT DO NOTHING` raw SQL?” → “Prisma upsert với `update: {}` chính là tương đương ON CONFLICT DO UPDATE SET <empty>. Hành vi giống nhau, không cần escape \$queryRaw.”

Phản biện 2: “Sao không SELECT FOR UPDATE để serialize?” → “Sẽ chặn các tx khác đọc cùng row trong khi tx hiện tại chạy. Với upsert atomic ở DB-level, hai tx đan xen vẫn cho kết quả đúng mà không phải lock — hiệu năng tốt hơn.”

Phản biện 3: “Sao đặt `update: {}` mà không là `update: { updatedAt: new Date() }`?” → “Việc bump timestamp khi không có thay đổi thật làm audit log hiểu nhầm ‘có ai sửa’. Empty update là chính xác nhất với ngữ nghĩa ‘chỉ đảm bảo row tồn tại’.”

Ghi chú phát hiện: Lỗi cũ (`findUnique` → `create`) chỉ xảy ra khi 2 đề xuất cùng `so_quyet_dinh` được duyệt cùng phút — trong domain Phòng Chính trị Học viện chỉ 1-2 Admin, xác suất thấp. Nhưng vì có audit log nuốt lỗi nên rất khó phát hiện khi đã xảy ra → fix chủ động bằng upsert.

Câu hỏi follow-up có thể bị dồn:

“Catch block dưới try giờ catch lỗi gì? Có còn nuốt lỗi không?”

Catch vẫn còn, vẫn ghi vào `SystemLog` với `action: 'ERROR'`. Nhưng giờ chỉ còn bắt lỗi **hiếm**: network với DB, schema mismatch, transient timeout. Không còn bắt P2002 vì upsert đã xử lý. Đồng bộ `FileQuyetDinh` là tác vụ **best-effort phụ** sau khi transaction chính đã commit dữ liệu khen thưởng — nếu fail, có thể chạy lại từ proposal đã duyệt mà không ảnh hưởng tính nhất quán.

“Hai tx có `file_path` khác nhau thì tx nào thắng?”

Tx commit trước thắng — `file_path` của tx đầu được giữ. Tx sau gặp `update: {}` (no-op) nên không ghi đè. Sau đó nhánh `if (filePath)` của tx sau chạy `updateMany where: { file_path: null }` — không match (vì `file_path` đã có), nên cũng không ghi gì. Hành vi: **first writer wins** cho cả `file_path`, `ngay_ky`, `nguoi_ky`. Đúng nghĩa — một số quyết định chỉ có một bộ metadata “gốc”.

“Sao không deadlock khi 2 upsert cùng row?”

Postgres implement upsert bằng `INSERT ... ON CONFLICT DO UPDATE`. Khác `SELECT FOR UPDATE` ở chỗ: không lấy lock trước rồi mới check, mà lấy row-level lock khi `INSERT`, nếu trùng key thì release insert lock và lấy update lock trên row đã tồn tại. Hai tx đan xen sẽ serialize tự động ở DB-level mà không cần lock thủ công.

“Có test concurrent specifically không?”

Hiện chỉ có integration test cho happy path (`tests/approve/decisionMappings.test.ts`). Test concurrent thực sự cần spawn 2 worker, khó setup trong jest. Hướng phát triển: viết test dùng `Promise.all([approve(p1), approve(p2)])` với cùng `so_quyet_dinh` và assert cả hai succeed + chỉ 1 row trong `FileQuyếtDinh`.

“Sao không dùng `SERIALIZABLE` isolation level?”

`SERIALIZABLE` gây retry hàng loạt khi conflict, throughput giảm rõ rệt. Upsert ở `READ COMMITTED` đã đủ vì atomic ở mức row constraint. Project chỉ cần `SERIALIZABLE` cho các luồng có pattern “đọc-rời-quyết-định-rời-ghi” mà không có cách diễn đạt qua atomic operation.

D.10 — Transaction timeout cho đợt phê duyệt lớn

Tình huống: Cuối năm, Admin phê duyệt một đợt 300+ quân nhân trong 1 đề xuất. Transaction Prisma có timeout mặc định 5 giây; project set `PROPOSAL_APPROVE_TX_TIMEOUT_MS = 180000` (3 phút) tại `services/proposal/approve/import.ts`.

Bên trong transaction: 1. Per-personnel writes — N lần `tx.danhHieuHangNam.upsert` (mỗi quân nhân). 2. Decision file sync — như D.9 ở trên. 3. Profile recalc — gọi `computeEligibilityFlags` cho từng quân nhân bị thay đổi. 4. Audit log — `tx.systemLog.create` ghi payload chi tiết.

Nếu chậm hơn timeout → Prisma rollback toàn bộ, user nhận lỗi `Transaction not found` không thân thiện.

Cơ chế chống:

```
// services/proposal/approve/import.ts
// Approve transaction covers per-personnel writes + profile recalc + audit + decision sync
// 60s was too tight for end-of-year batches (~300+ personnel). Bumped to 180s; if a sin
// approve ever needs more, split the proposal rather than raising further.
const PROPOSAL_APPROVE_TX_TIMEOUT_MS = 180000;

await prisma.$transaction(
  async tx => {
    // ... per-personnel writes, decision sync, recalc, audit
  },
  { timeout: PROPOSAL_APPROVE_TX_TIMEOUT_MS }
);
```

Phản biện: “Tại sao không bỏ timeout hẳn?” → “Đề timeout vô hạn rất nguy hiểm — nếu một query bị deadlock hoặc treo, transaction sẽ giữ lock vô hạn, chặn các transaction khác. 180s là cân bằng giữa ‘đủ cho đợt lớn nhất từng gặp (~300 quân nhân, đo thực tế ~2 phút)’ và ‘fail-fast khi có sự cố’.”

Hướng dài hạn: Nếu một đợt vượt 180s, kiến trúc đúng là **chia nhỏ proposal** (mỗi đề xuất tối đa 200 quân nhân) thay vì tăng timeout. Giới hạn được enforce ở tầng validation khi tạo đề xuất.

Câu hỏi follow-up có thể bị đôn:

“Sao 180s mà không 60s như cũ, đo cụ thể bao nhiêu?”

Số 60s cũ là default tại thời điểm viết, không qua benchmark. 180s là chọn dựa trên ước lượng: ~300 quân nhân × ~400ms cho mỗi vòng lặp (upsert + recalc + audit) ≈ 120s, cộng buffer 50% cho file sync + Excel attachments. Em chưa có benchmark

định lượng formal — nếu hội đồng hỏi “có dữ liệu đo không?” em sẽ trả lời thật: “Em chưa benchmark có hệ thống, 180s là estimate dựa trên test thủ công trên dataset mẫu ~150 quân nhân (đo ~50s), nhân hệ số an toàn $\times 3$ ”.

“Sao không đặt env var để cấu hình theo môi trường?”

Có thể, nhưng timeout transaction là quyết định kiến trúc, không phải tuning runtime. Nếu để env var, dev có thể nâng lên 600s tránh được vấn đề “đợt quá to” thay vì giải quyết gốc (chia nhỏ proposal). Hardcode + comment là cố ý — “nếu cần tăng hãy đọc comment và split proposal”.

“Tx đang chạy 180s có chặn các tx khác không?”

Không full block — Postgres MVCC cho phép đọc thoải mái. Chỉ chặn WRITE vào cùng row đang lock (vd: đang upsert quân nhân X thì tx khác sửa quân nhân X sẽ wait). Trong domain này, đợt phê duyệt thường chỉ Admin Phòng Chính trị làm, không ai khác cùng sửa quân nhân đang được duyệt → tác động thực tế thấp.

“User thấy lỗi gì nếu timeout?”

Prisma ném P2028 Transaction not found hoặc Transaction API timeout. Backend bắt ở error handler global, trả 500 Có lỗi xảy ra khi phê duyệt, vui lòng thử lại. Hiện chưa có message tiếng Việt riêng cho case timeout vs lỗi khác — đây là cải tiến nhỏ có thể thêm. Tất cả thao tác trong tx đã rollback nên DB ở trạng thái sạch, user retry an toàn.

“Có monitor cảnh báo khi tx gần timeout không?”

Hiện chưa. Có thể thêm bằng cách wrap `prisma.$transaction` với `Date.now()` đầu/cuối, log warning nếu duration > 80% timeout. Hướng phát triển trong báo cáo §6.2.

E. Logic chuỗi danh hiệu

E.1 — Giải thích rule chuỗi BKBQP / CSTĐTQ / BKTTCP cá nhân

Danh hiệu	Số năm chu kỳ	Cờ tiền điều kiện	Yêu cầu khác	Lifetime
BKBQP	2 năm CSTĐCS liên tục	—	NCKH mỗi năm trong chuỗi	Lặp lại
CSTĐTQ	3 năm CSTĐCS liên tục	≥ 1 BKBQP trong cửa sổ trượt 3 năm cuối	NCKH mỗi năm	Lặp lại
BKTTCP	7 năm CSTĐCS liên tục	đúng 3 BKBQP và đúng 2 CSTĐTQ trong 7 năm cuối	NCKH mỗi năm	Một lần duy nhất

E.2 — Cycle nghĩa là gì? Có gì khác lifetime?

- **Cycle (isLifetime: false):** danh hiệu lặp lại mỗi `cycleYears`. `Eligibility = streak >= cycleYears && streak % cycleYears === 0`. Vd: BKBQP nhận 2024 → đủ điều kiện đề nghị lần kế tiếp khi đạt CSTĐCS năm 2025 + 2026.
- **Lifetime (isLifetime: true):** chỉ nhận 1 lần. Sau khi nhận, hệ thống block với message “Đã có ... chưa hỗ trợ cao hơn ...”. Áp dụng cho BKTTCP cá nhân.

E.3 — “Lỡ đợt” nghĩa là gì? Hệ thống xử lý ra sao?

Định nghĩa: Đến mốc đề nghị mà không có đề xuất → cycle tiếp tục đếm. Sau `cycleYears` năm sẽ lại đến mốc, vẫn được xét.

Ví dụ BKBQP cycle 2 năm: - 2023: CSTĐCS. 2024: CSTĐCS. → Đáng lẽ đề nghị BKBQP cuối 2024 nhưng “lỡ”. - 2025: CSTĐCS. 2026: CSTĐCS. → `Streak = 4`, chia hết 2 → eligible cuối 2026.

Không cần “đứt chuỗi CSTĐCS” giữa chừng.

E.4 — Cửa sổ trượt 3 năm / 7 năm là gì?

Cửa sổ trượt 3 năm cho CSTĐTQ: đếm BKBQP trong 3 năm gần nhất tính từ `year - 1`. - Ví dụ xét CSTĐTQ năm 2026: cửa sổ từ 2023 đến 2025. - BKBQP nhận năm 2022 → ngoài cửa sổ → không đếm.

Cửa sổ 7 năm cho BKTTCP: đếm BKBQP và CSTĐTQ trong 7 năm gần nhất từ `year - 1`. - Personal BKTTCP yêu cầu **đúng 3 BKBQP và đúng 2 CSTĐTQ** vì lifetime. - Unit BKTTCP yêu cầu ≥ 3 BKBQP vì non-lifetime.

Vì sao tính từ `year - 1` chứ không phải `year`? Khi xét eligibility cho **năm hiện tại**, dữ liệu năm hiện tại có thể chưa đầy đủ (chưa có quyết định danh hiệu năm hiện tại). Cửa sổ tính từ `year - 1` nhìn về quá khứ — đảm bảo dữ liệu đã chốt.

Ví dụ cụ thể: Tháng 6/2026, xét CSTĐTQ cho năm 2026 → cửa sổ 3 năm là 2023, 2024, 2025. Nếu tính từ `year` thì cửa sổ là 2024-2026, mà 2026 chưa có dữ liệu danh hiệu hằng năm (thường ban hành cuối năm).

E.5 — ChainContext là gì?

Là object derive từ `DanhHieuHangNam` rows, không lưu DB. Chứa: - `chainStartYear`: năm bắt đầu chuỗi CSTĐCS hiện tại. - `lastBkbqpYear`, `lastCstdtqYear`, `lastBkttcpYear`: năm gần nhất nhận từng cờ. - `streakSinceLastBkbqp`, `streakSinceLastCstdtq`, `streakSinceLastBkttcp`: số năm CSTĐCS liên tục kể từ lần nhận gần nhất (0 nếu chưa từng nhận). - `missedBkbqp`, `missedCstdtq`: số chu kỳ đã lỡ.

Hàm `computeChainContext(danhHieus, currentYear)` đọc tất cả `DanhHieuHangNam` của 1 quân nhân và tính context. Phức tạp $\sim O(n)$ với n = số năm có dữ liệu.

E.6 — Tại sao tính context realtime mà không lưu DB?

Lý do: - Source of truth là `DanhHieuHangNam` — context chỉ là derive. - Nếu lưu DB → mỗi khi sửa danh hiệu phải invalidate cache, dễ lệch nhau. - Tính nhanh (~ 1 ms / quân nhân) → trade-off chấp nhận được.

Lưu vào `HoSoHangNam`: chỉ lưu **flags** đã quyết định (`du_dieu_kien_bkbqp/cstdtq/bkttcp`) cộng gọi_y text — phục vụ filter danh sách. Khi sửa nguồn → gọi `recalculateAnnualProfile(id)` để cập nhật.

E.7 — Tại sao có `computeEligibilityFlags (recalc)` và `checkAwardEligibility (API)` — chỉ 1 không được sao?

Lý do tách: - `computeEligibilityFlags` chạy cho recalc → ghi flag vào DB cho **toàn bộ tier** (BKBQP, CSTĐTQ, BKTTCP). - `checkAwardEligibility` chạy cho 1 endpoint khi user định gửi 1 đề xuất cụ thể → chỉ check 1 tier + trả message lỗi cụ thể.

Đảm bảo nhất quán: Cả hai gọi chung `chainEligibility.checkChainEligibility` cho rule core. Khác nhau ở phần “lifetime block” mà chỉ recalc cần xử lý (vì recalc set gọi_y cho UI).

E.8 — Test case cho rule chuỗi

Tổng cộng 937 ca kiểm thử, trong đó các test suite riêng cho rule chuỗi danh hiệu gồm: - `eligibility-bkbqp-personal.test.ts`: vừa đủ chu kỳ, lỡ chu kỳ, lặp chu kỳ, NCKH thiếu, NCKH có nhưng CSTĐCS dứt. - `eligibility-cstdtq-personal.test.ts`: cửa sổ trượt 3 năm có/không có BKBQP, BKBQP rơi khỏi cửa sổ. - `eligibility-bkttcp-personal.test.ts`: lifetime block, đếm === 3 BKBQP và === 2 CSTĐTQ strict. - `eligibility-bkbqp-unit.test.ts`: ĐVQT 2 năm. - `eligibility-bkttcp-unit.test.ts`: cửa sổ trượt 7 năm, non-lifetime. - `chainContext.test.ts`: derive context. - `chainCycleScenarios.test.ts`: scenarios tổng hợp.

F. Cú pháp Prisma đối chiếu SQL

F.0 — Setup Prisma client

```
// models/index.ts
import { PrismaClient } from '../generated/prisma';
export const prisma = new PrismaClient({
```

```
log: process.env.NODE_ENV === 'development' ? ['query', 'error'] : ['error'],
});
```

Bật log: ['query'] để xem SQL Prisma sinh ra trong console khi debug.

F.1 — Tìm 30 log gần nhất theo loại action 'A' trong 30 ngày qua

Prisma:

```
const days30Ago = new Date(Date.now() - 30 * 24 * 60 * 60 * 1000);

const logs = await prisma.systemLog.findMany({
  where: {
    action: 'CREATE',
    createdAt: { gte: days30Ago },
  },
  orderBy: { createdAt: 'desc' },
  take: 30,
  select: {
    id: true,
    actor_role: true,
    description: true,
    createdAt: true,
    NgnoiThucHien: { select: { username: true } },
  },
});
```

SQL tương đương:

```
SELECT
  l.id,
  l.actor_role,
  l.description,
  l."createdAt",
  t.username AS nguoi_thuc_hien
FROM "SystemLog" l
LEFT JOIN "TaiKhoan" t ON t.id = l.nguoi_thuc_hien_id
WHERE l.action = 'CREATE'
AND l."createdAt" >= NOW() - INTERVAL '30 days'
ORDER BY l."createdAt" DESC
LIMIT 30;
```

Index hỗ trợ: @@index([action, createdAt]) đã có trong schema → query dùng index B-tree, ~O(log n).

F.2 — Tìm 30 đề xuất loại A hoặc B mới nhất

Prisma:

```
const proposals = await prisma.bangDeXuat.findMany({
  where: {
    loai_de_xuat: { in: ['CA_NHAN_HANG_NAM', 'NIEN_HAN'] },
  },
  orderBy: { createdAt: 'desc' },
  take: 30,
  include: {
    NgnoiDeXuat: { select: { id: true, username: true } },
    CoQuanDonVi: { select: { ten_don_vi: true } },
  },
});
```

SQL:

```
SELECT bdx.*, t.username, cqdv.ten_don_vi
FROM "BangDeXuat" bdx
LEFT JOIN "TaiKhoan" t ON t.id = bdx.nguoi_de_xuat_id
LEFT JOIN "CoQuanDonVi" cqdv ON cqdv.id = bdx.co_quan_don_vi_id
WHERE bdx.loai_de_xuat IN ('CA_NHAN_HANG_NAM', 'NIEN_HAN')
ORDER BY bdx."createdAt" DESC
LIMIT 30;
```

F.3 — Tìm log theo nhiều bộ lọc kết hợp (date range + action + role + resource + keyword)

Prisma:

```
const where: Prisma.SystemLogWhereInput = {
  AND: [
    { createdAt: { gte: from, lte: to } },
    actions.length > 0 ? { action: { in: actions } } : {},
    role ? { actor_role: role } : {},
    resource ? { resource } : {},
    keyword ? { description: { contains: keyword, mode: 'insensitive' } } : {},
  ],
};

const [data, total] = await Promise.all([
  prisma.systemLog.findMany({
    where,
    orderBy: { createdAt: 'desc' },
    skip: (page - 1) * limit,
    take: limit,
  }),
  prisma.systemLog.count({ where }),
]);
```

SQL:

```
-- Đếm tổng
SELECT COUNT(*) FROM "SystemLog"
WHERE "createdAt" BETWEEN $1 AND $2
  AND action = ANY($3)
  AND actor_role = $4
  AND resource = $5
  AND description ILIKE '%' || $6 || '%';

-- Lấy data
SELECT * FROM "SystemLog"
WHERE "createdAt" BETWEEN $1 AND $2
  AND action = ANY($3)
  AND actor_role = $4
  AND resource = $5
  AND description ILIKE '%' || $6 || '%'
ORDER BY "createdAt" DESC
LIMIT $7 OFFSET $8;
```

Lưu ý: Em dùng Promise.all chạy song song count + data → giảm latency. Nếu chạy tuần tự sẽ tốn gấp đôi thời gian.

F.4 — Pagination với cursor (cho bảng > 100k rows)

Prisma offset (đơn giản, chậm khi page sâu):

```
const data = await prisma.quanNhan.findMany({
  skip: (page - 1) * limit,
  take: limit,
  orderBy: { id: 'asc' },
});
```

Prisma cursor (nhanh hơn ở page sâu):

```
const data = await prisma.quanNhan.findMany({
  cursor: lastId ? { id: lastId } : undefined,
  skip: lastId ? 1 : 0,
  take: limit,
  orderBy: { id: 'asc' },
});
const nextCursor = data[data.length - 1]?.id;
```

SQL offset (giáo trình):

```
SELECT * FROM "QuanNhan" ORDER BY id LIMIT 50 OFFSET 1000; -- chậm khi offset lớn
```

SQL cursor (production):

```
SELECT * FROM "QuanNhan" WHERE id > 'last_id' ORDER BY id LIMIT 50;
```

Ngữ cảnh project: Hiện em dùng offset vì dataset nhỏ (< 5k quân nhân). Khi vượt 100k records sẽ chuyển cursor.

F.5 — Đếm số quân nhân theo đơn vị (group by)

Prisma:

```
const counts = await prisma.quanNhan.groupBy({
  by: ['co_quan_don_vi_id'],
  _count: { id: true },
  orderBy: { _count: { id: 'desc' } },
});
```

SQL:

```
SELECT co_quan_don_vi_id, COUNT(id) AS _count_id
FROM "QuanNhan"
GROUP BY co_quan_don_vi_id
ORDER BY _count_id DESC;
```

F.6 — Thống kê khen thưởng theo năm và loại danh hiệu

Prisma raw query (vì cần CASE WHEN):

```
const stats = await prisma.$queryRaw<Array<{ nam: number; danh_hieu: string; so_luong: b
  SELECT nam, danh_hieu, COUNT(*) AS so_luong
  FROM "DanhHieuHangNam"
  WHERE nam BETWEEN ${fromYear} AND ${toYear}
  GROUP BY nam, danh_hieu
  ORDER BY nam DESC, danh_hieu;
`;
```

Convert bigint → number:

```
const result = stats.map(s => ({ ...s, so_luong: Number(s.so_luong) }));
```

F.7 — Find quân nhân thuộc cây đơn vị MANAGER quản lý

Prisma:

```
const personnelInUnit = await prisma.quanNhan.findMany({
  where: {
    OR: [
      { co_quan_don_vi_id: managerCqdvId },
      { don_vi_truc_thuoc_id: { in: dvttIdsUnderCqdv } },
    ],
  },
  include: { ChucVu: true, CoQuanDonVi: true, DonViTrucThuoc: true },
});
```

SQL:

```
SELECT qn.*, cv.ten_chuc_vu, cqdv.ten_don_vi AS cqdv_ten, dvtt.ten_don_vi AS dvtt_ten
FROM "QuanNhan" qn
LEFT JOIN "ChucVu" cv ON cv.id = qn.chuc_vu_id
LEFT JOIN "CoQuanDonVi" cqdv ON cqdv.id = qn.co_quan_don_vi_id
LEFT JOIN "DonViTrucThuoc" dvtt ON dvtt.id = qn.don_vi_truc_thuoc_id
WHERE qn.co_quan_don_vi_id = $1
      OR qn.don_vi_truc_thuoc_id = ANY($2);
```

F.8 — Eager loading nested 3 cấp

Prisma:

```
const quanNhan = await prisma.quanNhan.findUnique({
  where: { id },
  include: {
    DonViTrucThuoc: {
      include: {
        CoQuanDonVi: true,
      },
    },
    DanhHieuHangNam: {
      orderBy: { nam: 'desc' },
      take: 10,
    },
    LichSuChucVu: {
      include: { ChucVu: true },
      orderBy: { ngay_bat_dau: 'desc' },
    },
  },
});
```

SQL: Prisma sinh nhiều query JOIN. Có thể debug bằng log: `['query']` trong client config.

F.9 — Insert batch nhiều bản ghi

Prisma createMany (nhanh nhất):

```
await prisma.danhHieuHangNam.createMany({
  data: rows.map(r => ({ quan_nhan_id: r.id, nam: r.nam, danh_hieu: r.code })),
  skipDuplicates: true, // bỏ qua row vi phạm @unique
});
```

SQL:

```
INSERT INTO "DanhHieuHangNam" (id, quan_nhan_id, nam, danh_hieu)
VALUES (gen_random_uuid(), $1, $2, $3),
       (gen_random_uuid(), $4, $5, $6),
       ...
ON CONFLICT (quan_nhan_id, nam) DO NOTHING;
```

Hạn chế **createMany**: không trả về danh sách đã tạo (Postgres). Em phải query lại nếu cần ID.

F.10 — Update có điều kiện (atomic)

Prisma **updateMany**:

```
const result = await prisma.bangDeXuat.updateMany({
  where: { id, status: 'PENDING' },
  data: { status: 'APPROVED', nguoi_duyet_id, ngay_duyet: new Date() },
});
if (result.count === 0) throw new ValidationError('Đã được người khác xử lý');
```

SQL:

```
UPDATE "BangDeXuat"
SET status = 'APPROVED', nguoi_duyet_id = $1, ngay_duyet = NOW()
WHERE id = $2 AND status = 'PENDING'
RETURNING id;
-- Nếu không có row trả về → đã bị update bởi transaction khác
```

F.11 — Upsert (insert or update)

Prisma:

```
await prisma.hoSoHangNam.upsert({
  where: { quan_nhan_id: personnelId },
  create: { quan_nhan_id: personnelId, ...flags },
  update: flags,
});
```

SQL:

```
INSERT INTO "HoSoHangNam" (quan_nhan_id, du_dieu_kien_bkbqp, ...)
VALUES ($1, $2, ...)
ON CONFLICT (quan_nhan_id) DO UPDATE
SET du_dieu_kien_bkbqp = EXCLUDED.du_dieu_kien_bkbqp;
```

F.12 — Transaction nhiều thao tác

Prisma sequential transaction (array):

```
await prisma.$transaction([
  prisma.bangDeXuat.update({ where: { id }, data: { status: 'APPROVED' } }),
  prisma.danhHieuHangNam.createMany({ data: rows }),
  prisma.systemLog.create({ data: logEntry }),
]);
```

Prisma interactive transaction (linh hoạt hơn):

```
await prisma.$transaction(async tx => {
  const proposal = await tx.bangDeXuat.findUnique({ where: { id } });
  if (proposal.status !== 'PENDING') throw new ValidationError();
  await tx.bangDeXuat.update({ where: { id }, data: { status: 'APPROVED' } });
  await tx.danhHieuHangNam.createMany({ data: rows });
  await tx.systemLog.create({ data: logEntry });
});
```

SQL:

```
BEGIN;
UPDATE "BangDeXuat" SET status='APPROVED' WHERE id=$1;
INSERT INTO "DanhHieuHangNam" (...) VALUES (...);
INSERT INTO "SystemLog" (...) VALUES (...);
```

```
COMMIT;  
-- Nếu một câu lỗi: ROLLBACK;
```

F.13 — Find với JSON field (PostgreSQL JSONB)

Prisma — query JSONB path:

```
const logs = await prisma.systemLog.findMany({  
  where: {  
    payload: {  
      path: ['before', 'status'],  
      equals: 'PENDING',  
    },  
  },  
});
```

SQL:

```
SELECT * FROM "SystemLog"  
WHERE payload #>> '{before,status}' = 'PENDING';
```

F.14 — Đếm theo điều kiện (counted aggregation)

Prisma:

```
const count = await prisma.danhHieuHangNam.count({  
  where: { quan_nhan_id, danh_hieu: 'CSTDACS', nam: { gte: 2020, lte: 2024 } },  
});
```

SQL:

```
SELECT COUNT(*) FROM "DanhHieuHangNam"  
WHERE quan_nhan_id = $1 AND danh_hieu = 'CSTDACS' AND nam BETWEEN 2020 AND 2024;
```

F.15 — Aggregate (sum, avg, min, max)

Prisma:

```
const stat = await prisma.lichSuChucVu.aggregate({  
  where: { quan_nhan_id },  
  _sum: { so_thang: true },  
  _avg: { he_so_chuc_vu: true },  
  _min: { ngay_bat_dau: true },  
  _max: { ngay_bat_dau: true },  
});
```

SQL:

```
SELECT  
  SUM(so_thang) AS sum_so_thang,  
  AVG(he_so_chuc_vu) AS avg_he_so,  
  MIN(ngay_bat_dau) AS min_start,  
  MAX(ngay_bat_dau) AS max_start  
FROM "LichSuChucVu"  
WHERE quan_nhan_id = $1;
```

F.16 — Distinct values

Prisma:

```
const years = await prisma.danhHieuHangNam.findMany({  
  distinct: ['nam'],
```

```

    select: { nam: true },
    orderBy: { nam: 'desc' },
  });

```

SQL:

```
SELECT DISTINCT nam FROM "DanhHieuHangNam" ORDER BY nam DESC;
```

F.17 — Find với LIKE (search keyword)

Prisma:

```

const result = await prisma.quanNhan.findMany({
  where: {
    OR: [
      { ho_ten: { contains: keyword, mode: 'insensitive' } },
      { cccd: { contains: keyword } },
      { so_dien_thoai: { contains: keyword } },
    ],
  },
});

```

SQL:

```

SELECT * FROM "QuanNhan"
WHERE ho_ten ILIKE '%' || $1 || '%'
      OR cccd LIKE '%' || $1 || '%'
      OR so_dien_thoai LIKE '%' || $1 || '%';

```

F.18 — Soft delete pattern (giả sử)

Schema chưa dùng nhưng nếu cần:

```

// Filter chỉ lấy không bị xóa
const active = await prisma.quanNhan.findMany({
  where: { deletedAt: null },
});

// "Xóa" = set deletedAt
await prisma.quanNhan.update({
  where: { id },
  data: { deletedAt: new Date() },
});

```

F.19 — Cascade delete

Schema:

```

model QuanNhan {
  id String @id
  TaiKhoan TaiKhoan?
  DanhHieuHangNam DanhHieuHangNam[]
}
model TaiKhoan {
  quan_nhan_id String? @unique
  QuanNhan QuanNhan? @relation(fields: [quan_nhan_id], references: [id], onDelete: Cascade)
}

```

Khi xóa QuanNhan: Postgres tự xóa TaiKhoan, DanhHieuHangNam liên quan.

Tránh accidentally cascade: Dùng onDelete: Restrict cho FK quan trọng (vd: FileQuyetDinh không cho xóa khi còn bản ghi tham chiếu).

F.20 — Transaction với isolation level

Prisma (mặc định READ COMMITTED):

```
await prisma.$transaction(  
  async tx => { ... },  
  { isolationLevel: Prisma.TransactionIsolationLevel.Serializable }  
);
```

Khi nào cần Serializable? Khi rule yêu cầu “không có 2 transaction đồng thời ghi đè nhau” — vd: kiểm phong tỏa số quyết định trùng. Trade-off: chậm hơn 2-3 lần do Postgres phải retry.

F.21 — Window function (rank, lag)

Prisma không hỗ trợ window function tự nhiên → dùng \$queryRaw:

```
const ranked = await prisma.$queryRaw<Array<{ id: string; rank: bigint }>>`  
  SELECT id, ho_ten,  
         RANK() OVER (PARTITION BY co_quan_don_vi_id ORDER BY (  
           SELECT COUNT(*) FROM "DanhHieuHangNam" dh WHERE dh.quan_nhan_id = qn.id AND dh.danh_hieu = 'CSTD' )  
         ) DESC) AS rank  
  FROM "QuanNhan" qn;  
`;
```

SQL: giống raw query trên, parameterize.

F.22 — CTE (Common Table Expression)

```
await prisma.$queryRaw`  
  WITH personnel_with_streak AS (  
    SELECT quan_nhan_id, COUNT(*) AS streak  
    FROM "DanhHieuHangNam"  
    WHERE danh_hieu = 'CSTD' AND nam >= ${currentYear - 2}  
    GROUP BY quan_nhan_id  
    HAVING COUNT(*) >= 2  
  )  
  SELECT qn.*, p.streak FROM "QuanNhan" qn  
  JOIN personnel_with_streak p ON p.quan_nhan_id = qn.id;  
`;
```

F.23 — Filter theo relation field

Prisma — quân nhân có ít nhất 1 BKBQP từ năm 2020:

```
const result = await prisma.quanNhan.findMany({  
  where: {  
    DanhHieuHangNam: {  
      some: { nhan_bkbqp: true, nam: { gte: 2020 } },  
    },  
  },  
});
```

SQL:

```
SELECT DISTINCT qn.* FROM "QuanNhan" qn  
JOIN "DanhHieuHangNam" dh ON dh.quan_nhan_id = qn.id  
WHERE dh.nhan_bkbqp = TRUE AND dh.nam >= 2020;
```

F.24 — Filter ngược (none)

Prisma — quân nhân chưa từng nhận BKBQP:

```
const result = await prisma.quanNhan.findMany({
  where: {
    DanhHieuHangNam: { none: { nhan_bkbqp: true } },
  },
});
```

SQL:

```
SELECT qn.* FROM "QuanNhan" qn
WHERE NOT EXISTS (
  SELECT 1 FROM "DanhHieuHangNam" dh
  WHERE dh.quan_nhan_id = qn.id AND dh.nhan_bkbqp = TRUE
);
```

F.25 — Filter với every (all relations match)

Prisma — quân nhân mọi năm đều có CSTĐCS:

```
const result = await prisma.quanNhan.findMany({
  where: {
    DanhHieuHangNam: { every: { danh_hieu: 'CSTD' } },
  },
});
```

F.26 — Update many với increment

Prisma:

```
await prisma.coQuanDonVi.update({
  where: { id },
  data: { so_luong: { increment: 1 } },
});
```

SQL:

```
UPDATE "CoQuanDonVi" SET so_luong = so_luong + 1 WHERE id = $1;
```

F.27 — Connect / disconnect relation

Prisma — gắn quân nhân vào tài khoản:

```
await prisma.taiKhoan.update({
  where: { id: accountId },
  data: { QuanNhan: { connect: { id: quanNhanId } } },
});
```

SQL:

```
UPDATE "TaiKhoan" SET quan_nhan_id = $1 WHERE id = $2;
```

F.28 — Disconnect

Prisma:

```
await prisma.taiKhoan.update({
  where: { id },
  data: { QuanNhan: { disconnect: true } },
});
```

SQL:

```
UPDATE "TaiKhoan" SET quan_nhan_id = NULL WHERE id = $1;
```

F.29 — Nested write (tạo cha + con cùng lúc)

Prisma:

```
await prisma.quanNhan.create({
  data: {
    ho_ten: 'Nguyễn Văn A',
    chuc_vu_id: 'cv001',
    DanhHieuHangNam: {
      create: [
        { nam: 2023, danh_hieu: 'CSTDACS' },
        { nam: 2024, danh_hieu: 'CSTDACS' },
      ],
    },
  },
});
```

SQL: Prisma sinh INSERT QuanNhan ... RETURNING id rồi INSERT DanhHieuHangNam (... , quan_nhan_id) VALUES Nằm trong cùng transaction.

F.30 — Find unique theo composite key

Prisma:

```
const dh = await prisma.danhHieuHangNam.findUnique({
  where: { quan_nhan_id_nam: { quan_nhan_id, nam } }, // tên key sinh từ @unique([quan_
});
```

SQL:

```
SELECT * FROM "DanhHieuHangNam" WHERE quan_nhan_id = $1 AND nam = $2;
```

F.31 — Truy vấn với raw + Prisma typed

```
const result = await prisma.$queryRaw<Array<{ id: string; total: bigint }>>`
  SELECT quan_nhan_id AS id, COUNT(*) AS total
  FROM "DanhHieuHangNam"
  WHERE danh_hieu = 'CSTDACS' AND nam >= ${fromYear}
  GROUP BY quan_nhan_id
  HAVING COUNT(*) >= ${minCount};
`;
```

<> là explicit type cho TypeScript autocomplete.

F.32 — Find với date range hỗn hợp

Prisma:

```
const logs = await prisma.systemLog.findMany({
  where: {
    createdAt: {
      gte: startOfDay(from),
      lte: endOfDay(to),
    },
  },
});
```

SQL:

```
SELECT * FROM "SystemLog" WHERE "createdAt" >= $1 AND "createdAt" <= $2;
```

F.33 — Truy vấn join nhiều bảng cộng aggregation

```
// "Top 10 đơn vị có nhiều đề xuất chờ duyệt nhất"
const top = await prisma.$queryRaw<Array<{ ten_don_vi: string; pending: bigint }>>`
  SELECT cqdv.ten_don_vi, COUNT(bdx.id) AS pending
  FROM "CoQuanDonVi" cqdv
  LEFT JOIN "BangDeXuat" bdx ON bdx.co_quan_don_vi_id = cqdv.id AND bdx.status = 'PENDING'
  GROUP BY cqdv.id, cqdv.ten_don_vi
  HAVING COUNT(bdx.id) > 0
  ORDER BY pending DESC
  LIMIT 10;
`;
```

F.34 — N+1 query — anti-pattern

XẤU:

```
const quanNhans = await prisma.quanNhan.findMany();
for (const qn of quanNhans) {
  qn.danhHieus = await prisma.danhHieuHangNam.findMany({ where: { quan_nhan_id: qn.id } })
}
// → 1 + N query
```

TỐT (eager loading):

```
const quanNhans = await prisma.quanNhan.findMany({
  include: { DanhHieuHangNam: true },
});
// → 2 query, JOIN trong query 2
```

TỐT HƠN (batch with Map):

```
const quanNhans = await prisma.quanNhan.findMany();
const ids = quanNhans.map(q => q.id);
const allDanhHieus = await prisma.danhHieuHangNam.findMany({
  where: { quan_nhan_id: { in: ids } },
});
const danhHieuMap = new Map<string, DanhHieuHangNam[]>();
for (const dh of allDanhHieus) {
  const arr = danhHieuMap.get(dh.quan_nhan_id) || [];
  arr.push(dh);
  danhHieuMap.set(dh.quan_nhan_id, arr);
}
quanNhans.forEach(q => (q.danhHieus = danhHieuMap.get(q.id) || []));
// → 2 query, không có JOIN nặng
```

F.35 — Promise.all cho query song song

```
const [quanNhanCount, donViCount, deXuatPending] = await Promise.all([
  prisma.quanNhan.count(),
  prisma.coQuanDonVi.count(),
  prisma.bangDeXuat.count({ where: { status: 'PENDING' } }),
]);
```

3 query chạy song song → tổng thời gian $\approx \max(t_1, t_2, t_3)$ thay vì $t_1+t_2+t_3$.

G. Hiệu năng và mở rộng

G.1 — Hệ thống chịu được bao nhiêu user concurrent?

Trả lời: Em chưa load test cụ thể. Ước tính theo benchmark Express + Prisma + Postgres trên server 4 GB RAM: - ~500 request/giây cho endpoint đọc đơn giản (find by id). - ~100 request/giây cho endpoint phức tạp (recalc). - Số user concurrent ước tính 200-300.

Cách scale: PM2 cluster mode (pm2 ... -i max) → 1 process / CPU core. 4 cores = 4 process. Cần Redis cho session store nếu cluster.

Hiện tại: chạy single instance vì user thực tế ~50 người LAN nội bộ → không cần cluster.

G.2 — Endpoint nào chậm nhất? Tối ưu ra sao?

Hiện tại chậm nhất: POST /api/profiles/recalculate-all — chạy recalculateAnnualProfile cho 1247 quân nhân tốn 18 giây.

Đã tối ưu: - Dùng Promise.all chạy song song theo batch 50 quân nhân/lần. - Mỗi recalculateAnnualProfile chỉ ~14 ms. - Có thể cache kết quả nếu dataset chưa đổi.

Có thể cải thiện thêm: - Worker thread riêng (worker_threads Node.js). - Hàng đợi (BullMQ + Redis) cho job dài.

G.3 — Index database

Index có sẵn theo schema: - @@unique([quan_nhan_id, nam]) cho DanhHieuHangNam → composite unique index. - @@index([co_quan_don_vi_id]) cho QuanNhan. - @@index([action, createdAt]), @@index([resource, createdAt]), @@index([actor_role, createdAt]) cho SystemLog → tối ưu cho 3 dimension lọc phổ biến. - @@index([so_quyet_dinh]) cho mọi bảng output → tối ưu lookup theo số quyết định.

Cần thêm khi scale: index trên BangDeXuat.createdAt + loai_de_xuat cho filter list.

G.4 — Database connection pooling

Prisma mặc định pool size = num_physical_cpus * 2 + 1. Với server 4 cores → pool = 9.

Có thể chỉnh trong DATABASE_URL:

```
postgresql://user:pass@localhost:5432/db?connection_limit=20&pool_timeout=10
```

G.5 — Bundle size frontend

Next.js tự code-split theo route. Bundle initial ~ 250 KB gzipped.

Tối ưu đã làm: - Lazy import Ant Design component khi cần (import { Table } from 'antd' thay vì import * as antd). - Tailwind purge content './src/**/*.{js,ts,jsx,tsx}'. - Image optimization của Next.js (next/image).

G.6 — Caching

Chưa có cache layer riêng. Hiện tại mọi query trực tiếp DB.

Hướng phát triển: Redis cache cho: - Permission lookup (~5 ms saving / request). - Dashboard statistics (refresh mỗi 5 phút). - Đơn vị tree (ít thay đổi, có thể cache 1 giờ).

H. Kiểm thử

H.1 — Tỷ lệ unit test bao nhiêu là đủ?

Trả lời: Mục tiêu của em là cover 100 % rule logic phức tạp (chuỗi danh hiệu, eligibility), > 80 % service layer, ≥ 70 % overall.

Hiện tại: - 937 test cases / 80 file pass 100 %. - Coverage $\geq 85\%$ cho services/profile, services/eligibility, services/proposal. - Một số helper pure function 100 %. - Controller layer thấp hơn (~60 %) — em ưu tiên test logic hơn integration.

H.2 — Mock Prisma thế nào?

```
// tests/setup.ts
import { mockDeep, mockReset } from 'jest-mock-extended';
import { PrismaClient } from '../src/generated/prisma';

export const prismaMock = mockDeep<PrismaClient>();
jest.mock('../src/models', () => ({ prisma: prismaMock }));

beforeEach(() => mockReset(prismaMock));

// Trong test
prismaMock.quanNhan.findMany.mockResolvedValue([{ id: '1', ho_ten: 'A' }] as any);
```

H.3 — Khác giữa unit test và integration test

Loại	Phạm vi	Tốc độ	Mock	Đại diện
Unit	1 hàm pure	Rất nhanh (ms)	Không cần	chainEligibility.test.ts
Service unit	1 service + mock Prisma	Nhanh	Có	proposalService.test.ts
Integration	Nhiều layer + DB thật	Chậm (giây)	Không	tests/scenarios/...

Project em chủ yếu unit + service unit, có ~10 integration test trong tests/scenarios/ chạy trên DB test riêng.

H.4 — TDD không? Code-first hay test-first?

Trả lời: Em không TDD strict. Quy trình thường là: 1. Viết code logic (~1 giờ). 2. Viết test cases ngay sau đó (~30 phút) — chủ yếu cho rule chuỗi. 3. Bug fix → bắt buộc viết test reproduce trước, sau đó fix.

Trường hợp TDD: rule eligibility chuỗi (197 ca test) — em viết test case dạng table-driven trước, sau đó implement.

I. Triển khai và vận hành

I.1 — Quy trình deploy production

1. Pull code mới về server.
2. cd BE-QLKT && npm ci && npm run build && npx prisma migrate deploy.
3. cd FE-QLKT && npm ci && npm run build.
4. pm2 reload ecosystem.config.js — zero-downtime reload.
5. Kiểm tra health check: curl http://localhost:4000/health.

I.2 — Rollback nếu deploy lỗi

1. Backup DB ngay trước migrate (cron tự chạy 2:00 sáng + manual trigger qua DevZone).
2. pm2 reload --revert để quay lại version cũ.
3. Nếu migrate đã chạy → restore DB từ backup (tự build INSERT, raw SQL text) (~14 giây cho dataset hiện tại).

I.3 — Monitor production

Hiện tại: - PM2 monitor (pm2 logs, pm2 monit). - PM2 bắt stdout/stderr (không dùng winston — winston không phải dependency). - System log trong DB cho audit.

Hướng phát triển: Sentry cho error tracking, Grafana + Prometheus cho metrics.

I.4 — Backup chiến lược

- **Cron tự động:** 02:00 sáng hằng ngày. Lưu vào `backups/<timestamp>.sql` (tự build INSERT, raw SQL text).
- **Retention:** 15 ngày (mặc định). File cũ tự xóa qua `cleanup cron`.
- **Manual trigger:** SUPER_ADMIN qua DevZone.
- **Test restore:** thử khôi phục mỗi tháng 1 lần, mất ~14 giây cho dataset hiện tại.

I.5 — Khi server crash

1. PM2 auto-restart 3 lần liên tục (config `max_restarts: 3, restart_delay: 5000`).
2. Nếu vẫn crash: log Slack/Telegram (chưa setup).
3. Health check Nginx report 502 → tạm hiển thị trang “Hệ thống đang bảo trì”.

J. Câu hỏi khai và edge case

J.1 — Hệ thống chống trùng dữ liệu ở những đâu?

Các điểm check trùng trong project (đã verify trong code):

Dữ liệu	Cột unique	File chặn ở tầng service	Message
CCCD quân nhân	QuanNhan.cccd @unique	personnel.service.ts:417	“CCCD đã tồn tại trong hệ thống”
Username tài khoản	TaiKhoan.username @unique	account.service.ts:232 personnel.service.ts:232	“Tên đăng nhập đã tồn tại” / “Username (CCCD) đã tồn tại trong hệ thống tài khoản”
Số quyết định	FileQuyếtDinh.so_quyet_dinh @unique	decision.service.ts:337	“Số quyết định đã tồn tại” (409)
Mã đơn vị	CoQuanDonVi.ma_don_vit @unique, DonViTrucThuoc.ma_don_vit @unique	unit.service.ts:89, 130, 154	“Mã đơn vị đã tồn tại” (409)
Tên chức vụ trong cùng đơn vị	@@unique([co_quan_don_vit_position, ten_chuc_vu])	position.service.ts:77	“Tên chức vụ đã tồn tại trong đơn vị này” (409)
Thành tích NCKH	composite unique	nckhStrategy.ts:140, proposal/approve/validation.ts:189	“Thành tích ‘...’ năm X đã tồn tại”
Danh hiệu hằng năm	@@unique([quan_nhan_db_level, nam])	DB-level + import preview	bắt qua P2002

Lưu ý quan trọng — Username = CCCD: Khi tạo tài khoản cho quân nhân, username chính là CCCD (`personnel.service.ts:232`) ghi rõ “Username (CCCD)”. Vì vậy check trùng username thực chất là check CCCD ở tầng tài khoản — đảm bảo 1 CCCD chỉ có tối đa 1 tài khoản đăng nhập.

Cơ chế chống trùng 2 lớp (defense in depth): 1. **Service layer:** Trước khi `prisma.create`, em `findFirst` check trùng → trả message thân thiện tiếng Việt. 2. **DB layer:** Cột `@unique` chặn cuối cùng. Nếu race condition lọt qua check service, INSERT trùng → Postgres trả P2002 → `middleware.errorHandler.ts:71` trả “Dữ liệu đã tồn tại (trùng lặp)”.

Trường hợp NULL: Cột `cccd` nullable. Hai quân nhân cùng `cccd = NULL` không bị conflict (Postgres coi `NULL ≠ NULL` trong unique constraint). Em chấp nhận điều này — quân nhân chưa có CCCD vẫn tạo được hồ sơ.

KHÔNG có check trùng họ_tên + ngày_sinh: Việt Nam có nhiều người trùng tên + ngày sinh (vd: “Nguyễn Văn An” 01/01/2000) → khó phân biệt thật là CCCD. Em chỉ unique theo CCCD, cho phép trùng tên + ngày sinh.

J.2 — Quân nhân chuyển đơn vị giữa năm có ảnh hưởng eligibility không?

Trả lời: Không. Eligibility chuỗi danh hiệu tính theo cờ trong `DanhHieuHangNam(nhan_bkbqp/cstdtq/bkttcp + năm)` — không phụ thuộc đơn vị tại thời điểm xét.

Ảnh hưởng cách hiển thị: Sau khi chuyển đơn vị, `DanhHieuHangNam` của năm cũ có thể có `cap_bac/chuc_vu` lưu snapshot. Nhưng chuỗi vẫn tính đúng.

J.3 — Năm sau hệ thống đổi rule eligibility (vd: BKBQP cycle 3 năm thay vì 2)?

Cơ chế: Đổi `cycleYears` trong `PERSONAL_CHAIN_AWARDS[BKBQP]` của `chainAwards.constants.ts`. Toàn bộ hệ thống tự áp rule mới.

Cần làm thêm: 1. Chạy `recalculateAnnualProfile` cho tất cả quân nhân để cập nhật flag `du_dieu_kien_*`. 2. Update test cases cho rule mới. 3. Migration data nếu cần (vd: vô hiệu hoá đề xuất pending bị mất hiệu lực).

J.4 — Backup file bị xoá nhầm — lấy lại được không?

Cơ chế bảo vệ: - Backup lưu trên cùng server BE → nếu disk hỏng thì mất. - **Hướng phát triển:** rsync sang NAS nội bộ Học viện, encrypted.

Hiện tại: chấp nhận rủi ro do hạ tầng LAN cô lập, không có cloud.

J.5 — Nếu Prisma migration fail giữa chừng

Cơ chế: Mỗi migration là 1 transaction Postgres. Lỗi giữa chừng → tự động rollback. Schema không bị nửa cập nhật.

Vấn đề: Migration đã commit một phần (vd: ALTER TABLE thành công, INSERT data fail) → cần manual rollback bằng SQL ngược lại.

Best practice em làm: Mỗi migration chỉ làm 1 việc. Tách migration phức tạp thành nhiều migration nhỏ.

J.6 — Database mất kết nối giữa transaction

Cơ chế: Postgres tự rollback transaction khi connection drop. Prisma client throw `PrismaClientKnownRequestError`. Service catch và trả 500.

Cải tiến: Retry với exponential backoff cho transaction quan trọng (chưa làm).

J.7 — Concurrent upload Excel: 2 user upload cùng tệp

Cơ chế: Multer lưu file với tên unique (`<timestamp>-<originalName>`). Hai upload không ghi đè nhau. Sau khi import xong, file tạm có thể xoá.

J.8 — File PDF quyết định upload xong nhưng DB update fail

Cơ chế: Em đã chú ý — flow là (1) upload file vào tmp, (2) mở DB transaction, (3) ghi DB, (4) move file vào thư mục chính, (5) commit DB. Nếu DB fail → file tmp bị orphan → cron cleanup mỗi giờ.

Hạn chế: Còn rủi ro orphan trong khoảng giữa step 4 và 5. Cải tiến: dùng outbox pattern.

J.9 — Validation BE bypass nếu attacker gọi trực tiếp API?

Trả lời: Zod validate ở middleware, chạy TRƯỚC controller. Bypass FE không bypass được BE.

```
router.post('/', verifyToken, validate(schema), controller.create);  
//                                     ^ chạy trước controller.create
```

Nếu schema validate fail → response 400 ngay, controller không được gọi.

J.10 — User nhập SQL injection vào search box?

Trả lời: Prisma parameterize `contains: input` → input bị escape, không thực thi như SQL. Vd: ' OR 1=1 -- chỉ là chuỗi tìm kiếm, không thoát query.

J.11 — Tiếng Việt có dấu trong URL?

Trả lời: URL được encode bằng `encodeURIComponent` ở FE (vd: `/personnel?keyword=Nguy%E1%BB%85n`). Express tự decode `req.query.keyword` về 'Nguyễn'. Prisma xử lý Unicode đúng.

J.12 — Time zone — server và client khác nhau

Hiện tại: PostgreSQL lưu `Timestamp(0)` không kèm timezone. Server giả định Asia/Ho_Chi_Minh (UTC+7).

Vấn đề: Nếu server đặt ở múi khác → sai. Em set `TZ=Asia/Ho_Chi_Minh` trong `ecosystem.config.js` để đảm bảo nhất quán.

Cải tiến tương lai: Đổi sang `Timestamptz` (with time zone) để rõ ràng.

J.13 — Truy vấn 1000 record về client cùng lúc

Cơ chế chống: - `MAX_LIMIT = 100` trong `helpers/paginationHelper.ts` → service tự cap limit xuống 100. - Zod schema validate limit: `z.coerce.number().max(100)`.

J.14 — User download file không có quyền

Cơ chế: Endpoint `GET /api/decisions/download/:soQuyetDinh` đi qua `verifyToken`. USER không có ownership → service trả 403.

```
async downloadDecision(soQuyetDinh, user) {
  const decision = await fileQuyetDinhRepository.findBySoQuyetDinh(soQuyetDinh);
  if (!decision) throw new NotFoundError();
  if (user.role === 'USER' && !await isMyOwnDecision(user.quan_nhan_id, decision)) {
    throw new ForbiddenError('Bạn không có quyền tải file này');
  }
  // serve file
}
```

J.15 — User edit URL ?personnelId= để xem đơn vị khác

Cơ chế: Manager chỉ thấy quân nhân trong cây đơn vị mình. Service luôn lọc:

```
const where = { id: req.params.id };
if (req.user.role === 'MANAGER') {
  where.id = { in: req.unitFilter.personnelIds }; // intersect với cây đơn vị
}
const personnel = await prisma.quanNhan.findFirst({ where });
if (!personnel) throw new NotFoundError();
```

J.16 — Race khi tạo so_quyet_dinh trùng

Cơ chế: Cột `FileQuyetDinh.so_quyet_dinh` có `@unique`. INSERT trùng → P2002 → catch và trả 409 + gợi ý số tiếp theo.

K. Câu hỏi nghiệp vụ quân đội

K.1 — Tại sao 7 loại khen thưởng mà không gộp?

Trả lời: Mỗi loại có: - Quy tắc xét khác nhau (chuỗi vs thời gian phục vụ vs chức vụ vs NCKH). - Mẫu quyết định khác nhau. - Cấp duyệt khác nhau (đơn vị, học viện, BQP). - Yêu cầu input khác (vd: KNC chỉ có 1 lần, HCCSVV có 3 hạng).

Gộp 7 loại thành 1 abstract sẽ tạo ra rule phức tạp hơn và mất tính linh hoạt khi 1 loại đổi rule.

K.2 — Tại sao BKTTCPC cá nhân lifetime mà đơn vị thì không?

Trả lời: Đơn vị có thể nhận BKTTCPC nhiều lần qua các chu kỳ 7 năm vì đơn vị tồn tại lâu, có thể “tái xuất sắc”. Cá nhân quân nhân — theo luật hiện hành — chỉ nhận BKTTCPC một lần trong sự nghiệp.

Nếu thay đổi rule: Đổi `isLifetime: false` trong `PERSONAL_CHAIN_AWARDS[BKTTCPC]` là xong. Logic core chấp nhận cả hai mode.

K.3 — Hệ thống có hỗ trợ Anh hùng LLVT, Anh hùng Lao động không?

Trả lời: Chưa. Đây là tier cao hơn BKTTCPC, em đã đề xuất ở §6.2 hướng (i). Mở rộng để dùng `ChainAwardConfig`.

Code mock cho buổi bảo vệ:

```
const ANH_HUNG_LLVT: ChainAwardConfig = {
  code: 'ANH_HUNG_LLVT',
  cycleYears: 0, // không cycle
  isLifetime: true,
  prerequisites: [{ code: 'BKTTCPC', count: 1 }],
  requiresNCKH: true,
  // ... thêm tiêu chí phức tạp khác
};
```

K.4 — Cấp Bộ Quốc phòng vs cấp Học viện duyệt — phân biệt thế nào?

Trả lời: - Trong project, mọi đề xuất đều do Phòng Chính trị Học viện duyệt thông qua vai trò ADMIN. - Việc gửi lên cấp BQP/Tổng cục là quy trình ngoài hệ thống (gửi văn bản giấy). - Em chưa có module tương tác với BQP — đề xuất ở Chương 6 hướng (vi) tích hợp SSO + dashboard cấp Bộ.

K.5 — Bảo mật khi LAN bị xâm nhập

Trả lời: Project chạy LAN nội bộ Học viện, không expose Internet. Nếu LAN bị xâm nhập: - Attacker vẫn cần valid credentials để đăng nhập. - Audit log ghi mọi thao tác → forensic được. - Backup chu kỳ → restore được.

Hạn chế: Nếu attacker có access DB trực tiếp (qua psql), họ thấy được password hash bcrypt → có thể brute (chậm vì cost 10).

K.6 — Có hỗ trợ kiểm tra trùng tên đơn vị không?

Trả lời: Cột `ma_don_vi@unique` chống trùng theo mã. Tên đơn vị không unique để cho phép 2 đơn vị khác nhau có tên giống (vd: hai “Tiểu đoàn 1”).

L. Khi không biết câu trả lời

L.1 — Khi bị hỏi về rule pháp luật cụ thể em không biết

Trả lời mẫu: “Cảm ơn thầy/cô. Phần rule pháp luật cụ thể em chưa nắm sâu — em đã căn cứ vào đề tài cấp Học viện do Thượng tá ThS. Đặng Quốc Hưng và Trung tá ThS. Bùi Đình Thế chủ trì [23] và Luật Thi đua Khen thưởng số 06/2022/QH15 [1]. Em sẽ ghi nhận để bổ sung.”

L.2 — Khi bị hỏi công nghệ em chưa làm

Trả lời mẫu: “Em chưa thực hành phần đó trong đồ án. Theo em hiểu thì [trả lời lý thuyết]. Em sẽ thử trong dự án thực tế.”

L.3 — Khi bị bắt sai logic / bug

Trả lời mẫu: “Vâng, đúng là chỗ đó em xử lý chưa hết. Em sẽ ghi vào danh sách hạn chế và phát triển thêm. Cảm ơn thầy/cô đã chỉ ra.”

Tuyệt đối tránh: cãi tay đôi, đổ lỗi cho thư viện hoặc người khác.

L.4 — Khi bị hỏi “tại sao không dùng X mới hơn?”

Trả lời mẫu: “Thời điểm em chọn công nghệ (đầu 2025), X chưa đủ chín hoặc cộng đồng còn nhỏ. Em ưu tiên ổn định và hỗ trợ lâu dài. Trong tương lai, khi X stable hơn, em sẽ cân nhắc migrate.”

L.5 — Khi bị hỏi về số liệu cụ thể em không nhớ

Trả lời mẫu: “Em chưa nhớ chính xác con số đó. Theo em ước tính khoảng [X]. Nếu cần em có thể chạy lại benchmark trên máy demo.”

L.6 — Khi câu hỏi quá rộng / quá khó

Trả lời mẫu: “Câu hỏi này khá rộng. Em xin trả lời trong phạm vi project — [trả lời phần em biết]. Phần [phần khác] vượt ngoài phạm vi đồ án, em sẽ tìm hiểu thêm.”

L.7 — Khi bị hỏi “em có dùng AI (ChatGPT, Claude, Copilot...) để làm đồ án không?”

Nguyên tắc cốt lõi:

1. **Trung thực** — đừng phủ nhận. Hội đồng có thể test bằng cách yêu cầu giải thích chi tiết bất kỳ file/function nào. Nói dối, bị bắt thóp = mất credibility nặng hơn nhiều.
2. **Định vị AI là công cụ, không phải tác giả** — như IDE, autocomplete, hoặc senior code reviewer.
3. **Khẳng định ownership** — mọi quyết định kiến trúc + business logic + trade-off đều của em.
4. **Demonstrate hiểu code** — sẵn sàng giải thích bất kỳ dòng nào.

Trả lời mẫu (kịch bản chuẩn):

“Vâng, em có dùng AI như công cụ hỗ trợ — chủ yếu cho 3 việc: (1) generate boilerplate code lặp lại (vd: CRUD controller skeleton, Zod schema, Excel column config), (2) review pattern và gợi ý refactor khi em thấy code dài hoặc lặp, (3) viết test fixtures và unit test cases edge case mà em chưa nghĩ ra. Em coi AI giống IDE thông minh hơn — giúp em viết nhanh hơn nhưng không quyết định thay em.

Tất cả quyết định kiến trúc (chọn Next.js + Express + Prisma, layered + repository pattern, strategy pattern cho 7 loại đề xuất), business logic (rule chuỗi danh hiệu BKBQP/CSTĐTQ/BKTTCP, cửa sổ trượt 3/7 năm, lifetime block cho BKTTCP cá nhân), và trade-off thiết kế (vd: FK string vs id, cascade rename scope) đều em hiểu sâu — em đã viết spec và prompt cụ thể cho AI dựa trên ngữ cảnh nghiệp vụ quân đội. Em sẵn sàng giải thích bất kỳ dòng code nào trong repo, dù do em viết tay hay AI hỗ trợ generate.”

Follow-up trả lời sẵn:

Hội đồng có thể hỏi	Trả lời
“AI viết bao nhiêu phần trăm?”	“Em không đo cụ thể được vì em sửa nhiều sau khi AI generate. Nhưng có thể nói: AI hỗ trợ ~30-40% boilerplate (form CRUD lặp, test fixture, Excel config), còn 60-70% còn lại — gồm toàn bộ business logic (eligibility, chain awards), schema design, và security middleware — em viết hoặc heavily edit. Quan trọng hơn: em hiểu hết.”
“Vậy có gì là của riêng em?”	“Domain knowledge và kiến trúc tổng thể. AI không biết về Luật Thi đua Khen thưởng 06/2022/QH15, không biết về rule chuỗi danh hiệu trong quân đội, không biết về thứ tự duyệt 4 cấp (USER → MANAGER → ADMIN → SUPER_ADMIN), không biết về workflow Phòng Chính trị Học viện. Em phải research luật + interview hướng dẫn + viết spec rồi mới prompt được AI để generate đúng.”

Hội đồng có thể hỏi	Trả lời
“Nếu AI bảo dùng cách A, em có biết khi nào A sai không?”	“Có. Em đã gặp tình huống AI suggest dùng pattern không phù hợp — ví dụ AI hay dùng default export cho React component nhưng em rule là named export (đã ghi trong CLAUDE.md của project). Em luôn <code>typecheck + jest + browser test</code> trước khi commit. Em đã reject nhiều code AI generate vì không đúng convention.”
“Em có biết tự code không, hay phải dựa AI?”	“Em biết tự code. AI giúp em làm nhanh hơn, nhưng nếu mất AI thì em vẫn build được — chỉ chậm hơn. Em đã viết tay nhiều phần khi cần precision (vd: cascade rename, transaction logic, audit log middleware). Em coi AI như Stack Overflow + IntelliSense thông minh hơn — không thay thế việc học và hiểu.”
“Theo em AI có làm giảm giá trị đồ án không?”	“Em nghĩ không — vì giá trị đồ án nằm ở: (1) hiểu nghiệp vụ + research luật, (2) thiết kế kiến trúc + chọn trade-off, (3) đảm bảo correctness + security + maintainability, (4) viết test cover được edge case, (5) deploy + vận hành thực tế. AI không làm được những phần này — nó chỉ tăng tốc gõ code. Industry hiện tại (GitHub Copilot, Cursor) cũng vậy: AI là công cụ, dev senior vẫn cần để guide nó.”

Tuyệt đối tránh:

- ❑ “Em không dùng AI” — risk fact-check lớn, mất credibility hoàn toàn nếu bị bắt thóp.
- ❑ “AI làm hết, em chỉ ghép” — mất ownership, hội đồng có thể fail.
- ❑ Lảng tránh / đánh trống lảng — giống như đang giấu.
- ❑ Phòng thủ quá đà (“nhưng em hiểu hết...”) trước khi được hỏi follow-up — có vẻ guilty.

Tone khuyên dùng: Bình thân, tự tin, coi như chuyện đương nhiên (vì đúng là chuyện đương nhiên ở industry hiện tại). Đừng xin lỗi vì dùng AI — không có gì sai để xin lỗi.

Chốt câu hỏi (nếu hội đồng vẫn truy):

“Em xin được khẳng định: dù em có dùng AI hỗ trợ hay không, em đã đầu tư đủ thời gian để hiểu mọi quyết định kỹ thuật trong project này. Em sẵn sàng được hội đồng test bằng cách hỏi chi tiết bất kỳ file, function, hay design decision nào — em sẽ giải thích được lý do và defend được trade-off.”

M. Khả năng bảo trì và mở rộng kiến trúc

M.1 — Tại sao kiến trúc của em “dễ bảo trì”? Đo bằng tiêu chí gì?

Ngắn: Em đánh giá maintainability theo 5 chỉ số định lượng: (1) tách lớp rõ qua quy ước thư mục, (2) phụ thuộc đơn hướng giữa các lớp, (3) file ≤ 500 LOC, (4) test coverage $\geq 85\%$ cho service trọng yếu, (5) tài liệu CLAUDE.md cho mọi quy ước. Cả 5 đều có số đo cụ thể trong project.

Chi tiết:

Tiêu chí	Đo trong project	Cách verify
Tách lớp rõ	6 lớp Route → Middleware → Controller → Service → Repository → Prisma; mỗi lớp 1 thư mục riêng	<code>ls BE-QLKT/src/</code> thấy 6 folder tương ứng
Phụ thuộc đơn hướng	Controller chỉ import service, service chỉ import repository, repository chỉ import prisma	<code>grep "from '../models'"</code> trong controllers/ ra 0 kết quả (anti-pattern AP-1)

Tiêu chí	Đo trong project	Cách verify
File $\leq$ 500 LOC	95 % file đạt; file vượt được tách (vd: proposal/approve.ts từ 2001 LOC $\rightarrow$ 480 LOC + 4 sub-file)	<code>find src/services -name "*.ts" -exec wc -l {} \;</code>
Test coverage $\geq$ 85 %	services/profile, services/eligibility, services/proposal đạt; controller $\sim$ 60 %	<code>npm run test:coverage</code>
Tài liệu quy ước	Root CLAUDE.md + BE-QLKT/CLAUDE.md + FE-QLKT/CLAUDE.md ghi rõ naming convention, anti-pattern, pattern bắt buộc	Mở 3 file đó

Phản biện: “5 chỉ số đó có chuẩn công nghiệp nào không?” $\rightarrow$ “Em tham khảo từ Clean Code (Robert Martin) và bộ chỉ số Sonar (cyclomatic complexity, code duplication, nesting depth) — đã ghi vào BE-QLKT/CLAUDE.md thành 9 anti-pattern bắt buộc tránh.”

M.2 — Thêm 1 loại khen thưởng mới (ví dụ “Anh hùng LLVT”) cần sửa bao nhiêu file?

Trả lời ngắn: 5–7 file. Quy trình rõ ràng.

Quy trình từng bước:

- constants/danhHieu.constants.ts
 $\rightarrow$ thêm enum 'ANH_HUNG_LLVT' vào DANH_HIEU_CA_NHAN
- constants/chainAwards.constants.ts
 $\rightarrow$ thêm vào PERSONAL_CHAIN_AWARDS:


```
{
  code: 'ANH_HUNG_LLVT',
  cycleYears: 0, // không cycle
  isLifetime: true, // 1 lần duy nhất
  prerequisites: [{ code: 'BKTTCP', count: 1 }],
  requiresNCKH: true,
}
```
- prisma/schema.prisma
 $\rightarrow$ thêm cột nhan_anh_hung_llvt + so_quyet_dinh_anh_hung_llvt vào DanhHieuHangNam
 $\rightarrow$ `npx prisma migrate dev --name add_anh_hung_llvt`
- services/proposal/strategies/anhHungLlvtStrategy.ts (file mới)
 $\rightarrow$ implement ProposalStrategy interface (4 method)
- services/proposal/strategies/index.ts
 $\rightarrow$ thêm 1 dòng vào REGISTRY
- validations/proposal.validation.ts
 $\rightarrow$ thêm Zod schema cho loại mới
- tests/services/eligibility-anh-hung-llvt-personal.test.ts (file mới)
 $\rightarrow$ 30–50 ca kiểm thử

Không cần sửa: route file (proposal route generic), controller (dispatch qua REGISTRY), audit log (helper generic), notification (helper generic), FE form (UI generate từ schema).

Đây là minh chứng mạnh nhất cho extensibility — em sẵn sàng demo live nếu hội đồng yêu cầu.

M.3 — Thêm 1 vai trò mới (ví dụ “ANALYST” — chỉ đọc dashboard)

Quy trình:

1. constants/roles.constants.ts
→ thêm `ROLES.ANALYST = 'ANALYST'`
2. middlewares/auth.ts
→ thêm `const requireAnalyst = checkRole([ROLES.SUPER_ADMIN, ROLES.ANALYST])`
3. routes/dashboard.route.ts
→ đổi `requireAdmin` thành `checkRole([ROLES.ADMIN, ROLES.ANALYST])`
4. seed script hoặc account create endpoint
→ cho phép tạo tài khoản ANALYST

Không cần sửa logic eligibility, proposal, personnel.

M.4 — Thêm 1 endpoint mới (vd: “Lấy thống kê theo quý”)

Quy trình theo BE-QLKT/CLAUDE.md mục “Adding a New Feature”:

1. validations/dashboard.validation.ts
→ schema query (year + quarter)
2. services/dashboard.service.ts
→ method `getQuarterlyStats(year, quarter)`
3. controllers/dashboard.controller.ts
→ `catchAsync` wrap, gọi service, trả `ResponseHelper.success`
4. routes/dashboard.route.ts
→ `router.get('/quarterly', verifyToken, requireAdmin, validate(...), auditLog(...), controller)`

Total ~ 4 file, 30 phút công việc nếu logic không phức tạp.

M.5 — Service A có 1500 dòng — em xử lý sao để dễ bảo trì?

Quy tắc trong project (BE-QLKT/CLAUDE.md):

LOC	Hành động
< 500	Để nguyên
500–800	Cân nhắc tách
800–1000	Bắt buộc tách logic phức tạp ra helper
> 1000	Bắt buộc tách theo pattern: <code><feature>.ts</code> orchestration môđ + <code><feature>/types.ts</code> + <code><feature>/<concern>.ts</code> cho mỗi concern

Ví dụ thực tế — `services/proposal/approve.ts` từng có 2001 LOC:

TRƯỚC:

`services/proposal/approve.ts` (2001 LOC) — chứa hết validation, mapping, import dispatch

SAU:

`services/proposal/approve.ts` (480 LOC) — orchestration: `parse` → `validate` → `map` → `import` → `log`

`services/proposal/approve/types.ts` (50 LOC) — shared interface

`services/proposal/approve/validation.ts` (380 LOC) — pre-flight check

`services/proposal/approve/decisionMappings.ts` (240 LOC) — decision metadata + PDF persist

`services/proposal/approve/import.ts` (320 LOC) — transaction import dispatch

Mỗi file giờ đảm nhận 1 concern, dễ test riêng, dễ tìm khi debug.

M.6 — Strategy pattern khi nào nên dùng? Khi nào overengineering?

Dùng khi: - ≥ 4 nhánh `if/else` dispatch theo enum/type. - Mỗi nhánh có ≥ 2 method tương tự cấu trúc. - Có khả năng thêm nhánh mới trong tương lai.

Đừng dùng khi: - 2-3 nhánh đơn giản $\rightarrow$ `if/else` rõ ràng hơn. - Mỗi nhánh chỉ 1 dòng $\rightarrow$ switch statement gọn hơn. - Không có khả năng mở rộng $\rightarrow$ tạo interface chỉ để có 1 implementation là overengineering.

Trong project em dùng Strategy ở 2 chỗ: 1. **services/proposal/strategies/** — 7 loại đề xuất, mỗi loại có 4 method. Đáng dùng. 2. **(KHÔNG dùng)** cho `notification/` — chỉ 3 loại notification, dispatch đơn giản, để `if/else`.

M.7 — Đặt tên biến/file/function — quy ước thế nào?

Bảng quy ước (trong CLAUDE.md):

Loại	Pattern	Ví dụ
React component	PascalCase.tsx	LoginForm.tsx
Hook	camelCase.ts	useFetch.ts
BE service	camelCase.service.ts	proposal.service.ts
BE controller	camelCase.controller.ts	account.controller.ts
BE route	camelCase.route.ts	auth.route.ts
Constants file	camelCase.constants.ts	roles.constants.ts
Constant value	UPPER_SNAKE_CASE + as const	ROLES.SUPER_ADMIN
Function/var	camelCase	computeChainContext
Prisma model	PascalCase Vietnamese + @@map("snake_case")	model QuanNhan { ... @@map("QuanNhan") }
DB field	snake_case Vietnamese	ho_ten, ngay_sinh, quan_nhan_id
Type/Interface	PascalCase	ApiResponse<T>, ProposalStrategy

Lý do: Convention nhất quán $\rightarrow$ grep nhanh, IDE autocomplete chính xác, code review giảm tranh cãi.

M.8 — Khi nào extract helper, khi nào để inline?

Extract khi: - Logic lặp lại 2+ lần (DRY). - Logic phức tạp ≥ 10 dòng có thể test riêng. - Pure function (không side effect).

Để inline khi: - Dùng 1 lần và chỉ 3-5 dòng. - Phụ thuộc vào nhiều biến local context.

Quy tắc helper (AP-3): Helper trong `helpers/` **không được** import `prisma`, `apiClient`, hay `service`. Chỉ pure function. Exception: `auditLog/` và `notification/` vốn coupling với DB.

M.9 — Database schema thay đổi — workflow đảm bảo không lệch giữa môi trường?

Workflow chuẩn:

1. Sửa `prisma/schema.prisma` trên máy dev
2. `npx prisma migrate dev --name <tên_migration>`
 $\rightarrow$ Sinh file migration SQL trong `prisma/migrations/<timestamp>_<tên>/`
3. Test trên dev xong $\rightarrow$ commit cả `schema.prisma` + thư mục migration
4. Trên staging/production: `npx prisma migrate deploy` (chỉ apply, không sinh migration mới)
5. Verify: `npx prisma migrate status`

Schema diff luôn versioned trong git $\rightarrow$ bất kỳ ai checkout code cũng có cùng schema.

Quy tắc nguy hiểm (AP-8): Cột có data $\rightarrow$ KHÔNG dùng `prisma db push` để rename. Phải viết script raw SQL `ALTER TABLE ... RENAME COLUMN ...` trong `src/scripts/`, chạy script trước, rồi mới `db push` để sync.

M.10 — Mở rộng sang microservice trong tương lai — kiến trúc hiện tại có cản trở không?

Trả lời: Layered architecture hiện tại đã sẵn sàng tách microservice vì các lớp coupling lỏng:

Module	Tách thành microservice riêng	Mức độ khó
services/auth.service.ts + routes/auth.route.ts	Auth Service	Dễ — đã decoupled
services/eligibility/ + services/profile/	Eligibility Engine (gRPC)	Trung bình — phụ thuộc DB schema chuỗi
services/notification/ + utils/socketService.ts	Notification Service	Dễ — message queue (Redis/Kafka)
services/backup.service.ts services/proposal/	Backup Worker Đề ở core	Dễ — cron riêng Phức tạp do liên kết nhiều bảng

Hạn chế hiện tại: Tất cả service share 1 database PostgreSQL → tách microservice phải tách DB (database-per-service), kèm theo distributed transaction (saga pattern). Đây là công việc lớn — em đã ghi vào hướng phát triển.

M.11 — Scale ngang (horizontal scaling) thế nào?

Cấu hình hiện tại: 1 server LAN, ~50 user concurrent.

Khi cần scale: 1. **PM2 cluster mode:** `pm2 start ecosystem.config.js -i max` → 1 process / CPU core. 4 cores = 4 instance trên cùng máy. 2. **Phân tách stateful state:** - refreshToken đang lưu DB → vẫn share giữa instance. - Socket.IO room — cần Redis adapter (@socket.io/redis-adapter) khi nhiều process. 3. **Load balancer:** Nginx upstream round-robin giữa các process. 4. **Database read replica:** Tách query đọc sang replica, ghi vào primary. Prisma support qua replicaUrls. 5. **Migration sang Kubernetes:** Khi vượt 1 server vật lý.

Bottleneck dự kiến đầu tiên: Database write throughput. Khi đó dùng partitioning theo năm cho DanhHieuHangNam (table lớn nhất khi tích lũy nhiều năm).

M.12 — Module split theo ngữ cảnh — nguyên tắc?

Nguyên tắc 1 helper file = 1 responsibility: - helpers/excelImportHelper.ts chỉ import (đọc workbook, validate dòng). - helpers/excelTemplateHelper.ts chỉ template (sinh workbook mẫu). - KHÔNG có file helpers/excelHelper.ts chứa cả hai.

Quy tắc service: - 1 service = 1 entity (vd: personnel.service.ts, proposal.service.ts). - Nếu 3+ service có logic giống nhau → extract vào services/<feature>/shared.ts hoặc helpers/<feature>/.

Quy tắc route: - 1 route file = 1 prefix (vd: auth.route.ts cho /api/auth). - Tránh route file > 200 LOC bằng cách tách subresource ra file riêng (vd: personnelNested.route.ts cho /api/personnel/:id/...).

M.13 — TypeScript strict mode

Hiện tại: strict: false, strictNullChecks: false trong tsconfig.json — relaxed mode.

Lý do chọn relaxed: - Cho phép linh hoạt khi xử lý kết quả Prisma (nhiều null trong nested include). - Giảm boilerplate ?? null không cần thiết.

Bù lại: - Zod validate input ở boundary → đảm bảo type runtime. - Test cases cover các trường hợp null/undefined.

Hạn chế thừa nhận: strict: true sẽ bắt nhiều bug hơn ở compile time. Nếu rebuild project sẽ bật strict ngay từ đầu.

M.14 — Dependency injection — em có dùng không?

Trả lời: Không dùng container DI (như NestJS/InversifyJS). Em dùng **manual wiring** qua singleton:

```
// services/proposal.service.ts
class ProposalService {
  // không inject — dùng repository singleton trực tiếp
```



```
}
export const proposalService = new ProposalService();
```

Lý do: - Project quy mô vừa (~50 service) — manual wiring vẫn quản lý được. - Container DI thêm phụ thuộc + decorator + reflect-metadata → không cần thiết. - Để mock test, em mock module qua `jest.mock('../repositories/...')`.

Khi nào nên đổi: Nếu project lên 200+ service hoặc cần DI scope (request scope, transient) thì chuyển NestJS.

M.15 — Versioning API — em xử lý sao khi cần breaking change?

Hiện tại: Tất cả endpoint dưới `/api/...` không có version. Đây là acceptable cho project nội bộ 1 client.

Khi cần versioning: - Strategy 1: URL prefix `/api/v1/...`, `/api/v2/...` - Strategy 2: Header `X-API-Version: 2` - Strategy 3: Field trong response: thêm `_version: 2` cho client kiểm.

Best practice: Giữ v1 song song với v2 ít nhất 6 tháng → client có thời gian migrate.

M.16 — Code review process

Hiện tại: Solo developer (1 mình em).

Cơ chế tự review: - Trước khi commit: `npm run typecheck && npm test && npm run lint`. - File `PROJECT_REVIEW.md` ghi các issue self-found theo CRITICAL/MEDIUM/LOW. - File `CLAUDE.md` định nghĩa anti-pattern bắt buộc tránh khi viết code mới.

Khi mở team: - Bắt buộc PR review qua GitHub. - CI pass mới merge. - Rule pre-commit hook chạy lint + format.

N. Tổng hợp chống tấn công và đánh giá an toàn

N.1 — Hệ thống của em có an toàn không? Đánh giá thẳng thắn.

Trả lời ngắn — KHÔNG né:

“Hệ thống của em an toàn ở mức **đủ cho môi trường LAN nội bộ Học viện**, nhưng **chưa đủ** để expose ra Internet công cộng. Em chia mức độ an toàn theo 3 cấp:

- **Đã chống tốt (đạt OWASP Top 10):** SQLi, XSS, CSRF, IDOR, BOLA, Mass Assignment, Brute force, file upload độc, Privilege escalation, Path traversal, missing auth.
- **Đã có nhưng cần cải thiện:** Rate limit (chỉ theo IP, chưa theo account), JWT rotation (chưa có grace period), audit log (chưa ghi failed login chi tiết).
- **Chưa có (thừa nhận):** 2FA, Web Application Firewall (WAF), Penetration test chính thức, Security headers nâng cao (CSP nonce), Anti-CSRF token (em né bằng JWT header), DDoS protection layer 4, Encryption at rest cho DB.”

N.2 — Bảng tổng hợp 10 kiểu tấn công × cơ chế phòng

#	Kiểu tấn công (OWASP/CWE)	Cơ chế chống trong project	Tập xử lý chính	Còn rủi ro?
1	SQL Injection (CWE-89)	Prisma parameterize tự động; không dùng <code>\$queryRawUnsafe</code> cho input từ user	<code>models/index.ts</code> + mọi service	Không
2	XSS (CWE-79)	React escape mọi <code>{value}</code> ; không có <code>dangerouslySetInnerHTML</code> cho user input; helmet headers	FE components + <code>index.ts</code> helmet	Thấp

#	Kiểu tấn công (OWASP/CWE)	Cơ chế chống trong project	Tập xử lý chính	Còn rủi ro?
3	CSRF (CWE-352)	JWT trong header Authorization, không cookie session → browser không tự gửi cross-origin	middlewares/auth	Không
4	IDOR / BOLA (CWE-639)	3 lớp: verifyToken + requireRole + ownership check trong service; unitFilter lọc theo cây đơn vị cho MANAGER	auth.ts, unitFilter.ts, services	Thấp
5	Brute force password (CWE-307)	authLimiter 30 lần đăng nhập thất bại / 5 phút / IP (chỉ đếm lần thất bại); bcrypt cost 10 (~100 ms/lần thử)	configs/rateLimit	Tung bình (chưa account lockout)
6	Mass Assignment (CWE-915)	Zod z.object() mặc định strip field ngoài schema ở mọi endpoint; service không truyền req.body thẳng vào prisma.create	middlewares/validation + validations/	Không
7	File upload độc (CWE-434)	Multer whitelist extension + MIME + size 10 MB; lưu ngoài web root; check magic byte cho PDF	configs/multer.ts	Thấp (chưa scan virus)
8	Privilege escalation (CWE-269)	Role trong JWT chữ ký HMAC, không sửa được client-side; Zod schema không cho update field role qua self-update	JWT + Zod	Không
9	Path traversal (CWE-22)	File path từ DB chứ không từ user; path.basename strip mọi ../	decision.service download	Không
10	Missing auth (CWE-306)	verifyToken middleware bắt buộc trước mọi route nghịệp vụ; không có endpoint nghịệp vụ public	Mọi routes/*.ts	Không
11	Sensitive Data Exposure (CWE-200)	Prisma select whitelist field; không trả password_hash, refreshToken; CCCD ẩn cho USER	services + helpers	Thấp

#	Kiểu tấn công (OWASP/CWE)	Cơ chế chống trong project	Tệp xử lý chính	Còn rủi ro?
12	Insecure Deserialization (CWE-502)	Không có deserialize từ user input (không dùng <code>JSON.parse</code> lên payload nhạy cảm); JWT verify chữ ký trước khi đọc	<code>auth.ts</code>	Không
13	DoS / Resource exhaustion (CWE-400)	Rate limit, body limit 10 MB, MAX_LIMIT 100 records, file size 10 MB	<code>rateLimiter</code> , <code>paginationHelper</code>	Trung bình (DDoS layer 4 không chống)
14	Clickjacking (CWE-1021)	helmet X-Frame-Options: DENY	<code>index.ts</code>	Không
15	Insufficient logging (CWE-778)	Audit log mọi mutate, ghi <code>actor_role</code> , <code>payload</code> , <code>ip_address</code> , <code>user_agent</code>	<code>middlewares/audit.ts</code>	Thấp (chưa log failed auth)

N.3 — Tự tay tấn công hệ thống — em đã thử những gì?

Em đã tự pentest cơ bản:

Kịch bản tấn công	Công cụ	Kết quả
SQLi vào search box “ho_ten=’ OR 1=1 –”	Postman manual	Bị Prisma escape, trả về 0 record (tìm tên = chuỗi đó)
XSS payload <code><script>alert(1)</script></code> vào ghi chú	UI nhập tay	Render thành text, không execute
Đăng nhập sai 31 lần trong 5 phút	Postman lặp	Lần 31 nhận 429 Too Many Requests (chỉ đếm lần thất bại)
USER call GET <code>/api/personnel/<other_id></code>	Postman với JWT của USER	403 Forbidden
MANAGER call GET <code>/api/personnel?co_quan_don_vi_id=<other_unit></code>	Postman	unitFilter lọc, trả 0 record
Tự sửa JWT đổi role thành ADMIN	jwt.io tool, decode + re-sign sai key	<code>jwt.verify</code> fail → 401
Upload file .exe đổi tên .pdf	Postman	MIME check fail → 400
Upload file 50 MB	Postman	Multer limit 10 MB → 413 Payload Too Large
Tạo 1000 đề xuất / 1 phút từ MANAGER	Script	<code>writeLimiter</code> 30 req / 15 phút → 429

Chưa thử: automated scan với Burp Suite, OWASP ZAP, sqlmap. Em đề xuất ở Chương 6 hướng phát triển.

N.4 — Threat model: ai có thể tấn công, mức độ thiệt hại?

Bảng STRIDE:

Threat	Tác nhân	Mức độ	Đã chống	Còn lại
Spoofing (giả mạo identity)	User cùng LAN biết username người khác	Cao	bcrypt + JWT signature	Brute force chậm (cost 10)

Threat	Tác nhân	Mức độ	Đã chống	Còn lại
Tampering (sửa data)	Attacker có access DB	Cao	Audit log mọi mutate	Không chống được DBA
Repudiation (chối bỏ)	User chối thao tác đã làm	Trung bình	Audit log có IP + user agent	Cần legal disclaimer
Information Disclosure (rò rỉ)	User truy vấn nhằm endpoint	Trung bình	Role + unit filter	Internal LAN — không firewall ngoài
Denial of Service	User phá hệ thống bằng request	Thấp (LAN)	Rate limit, body limit	DDoS layer 4 (cần WAF)
Elevation of Privilege	USER tự nâng thành ADMIN	Cao	JWT signed + role không updateable qua self-update	Không (đã chống tốt)

N.5 — JWT secret bị lộ — quy trình rotation?

Hiện tại: JWT_SECRET và JWT_REFRESH_SECRET lưu trong file .env, chmod 600, owner root.

Nếu lộ: 1. **Khẩn cấp:** đổi JWT_SECRET mới trong .env, restart server (pm2 reload). 2. Mọi access token cũ trở thành invalid → user phải đăng nhập lại. 3. **Refresh token:** đổi JWT_REFRESH_SECRET riêng → buộc đăng nhập lại với username/password. 4. Audit log review để xem có thao tác bất thường trong khoảng lộ.

Cải tiến: Lưu secret trong vault (HashiCorp Vault, AWS Secrets Manager) thay vì .env. Hiện chưa có vì LAN cô lập.

N.6 — Database admin (DBA) có lạm quyền — chống thế nào?

Trả lời thẳng thắn:

“Em chưa có cơ chế chống DBA insider threat hoàn toàn. Người có access PostgreSQL trực tiếp (qua psql hoặc pgAdmin) có thể: - Xem tất cả data (kể cả password hash bcrypt — phải brute mới ra plaintext). - Sửa data bypass audit log (vì audit log ghi qua middleware app, không qua DB trigger). - Xóa cả audit log.

Cách giảm thiểu: - Phân tách tài khoản DB: 1 tài khoản app dùng (chỉ INSERT/UPDATE/DELETE/SELECT các bảng cụ thể), 1 tài khoản DBA tách biệt. - Database trigger ghi log song song (em chưa làm — đề xuất ở hướng phát triển). - Encrypt at rest dùng pgcrypto cho cột nhạy cảm (em chưa làm). - Audit ở tầng OS: auditd Linux ghi access vào file DB.”

N.7 — Backup file bị copy ra ngoài — rủi ro?

Trả lời: - Backup tự build INSERT, lưu file .sql text. - Chứa **tất cả data** + password hash bcrypt. - Attacker copy được → có thể brute password offline.

Bảo vệ: - Thư mục backups/ chmod 700, owner postgres. - Khi sao lưu sang storage ngoài (NAS), phải mã hoá bằng GPG hoặc AES-256. - Hiện chưa làm — đề xuất ở hướng phát triển.

N.8 — Audit log bị xóa bởi SUPER_ADMIN — có chống không?

Trả lời thẳng: - Có endpoint DELETE /api/system-logs/... chỉ SUPER_ADMIN gọi được. - SUPER_ADMIN xóa log → mất bằng chứng. - Đây là rủi ro inherent của insider threat.

Cách giảm thiểu: - Log mọi thao tác xóa log (meta-log) cũng vào bảng SystemLog → SUPER_ADMIN xóa cả meta-log thì mất luôn. - Forward log realtime sang server log riêng (Splunk, ELK) — write-only từ phía app, SUPER_ADMIN của app không có quyền sửa server log. - Hiện em chưa làm — đề xuất hướng phát triển.

N.9 — DDoS layer 4 — chống được không?

Trả lời: - Hiện không. Express + Node.js không chống được SYN flood, UDP flood ở tầng mạng. - Cần WAF (Cloudflare, AWS Shield) hoặc reverse proxy có rate-limit IP-level (Nginx + limit_req_zone).

Trong context project: - Chạy LAN nội bộ — DDoS phải từ trong LAN → có nghĩa là người trong tổ chức. - Nếu cần thiết: kích hoạt Nginx limit_req_zone \$binary_remote_addr zone=one:10m rate=10r/s chặn ở edge.

N.10 — Encryption at rest — DB có mã hoá không?

Hiện tại: Không. PostgreSQL data trên disk dạng plain.

Có thể mã hoá: 1. **Disk-level:** LUKS encryption cho partition chứa `/var/lib/postgresql`. 2. **Column-level:** dùng `pgcrypto`, `vdpassword_hash = pgp_sym_encrypt(value, key)`. 3. **TLS giữa app và DB:** PostgreSQL hỗ trợ TLS, hiện chưa bật.

Trong project: chưa cần vì LAN cô lập, server đặt trong phòng khoá. Đề xuất khi expose ra Internet.

N.11 — Dependency vulnerabilities — em quản lý sao?

Cơ chế: - `npm audit` chạy thủ công mỗi tuần. - `npm outdated` để xem package cũ. - Khi có CVE: update minor/patch version qua `npm update`. Major version cần test cẩn thận.

Hiện tại: `npm audit` báo 0 high/critical (chỉ vài low).

Đề xuất: Dependabot hoặc Renovate bot tự tạo PR khi có CVE.

N.12 — Security headers — có set đầy đủ không?

Hiện tại qua helmet():

```
import helmet from 'helmet';
app.use(helmet({ crossOriginResourcePolicy: { policy: 'cross-origin' } }));
```

Helmet mặc định set: - Strict-Transport-Security (HSTS) - X-Content-Type-Options: nosniff - X-Frame-Options: DENY - X-XSS-Protection: 0 (disabled vì gây bug ở browser cũ) - Referrer-Policy: no-referrer

`helmet()` bật mặc định nhưng CSP chưa được cấu hình riêng.

Có thể cải thiện: - CSP nonce-based thay vì 'self' cho strict hơn. - Permissions-Policy chặn camera/microphone API. - HSTS preload list.

N.13 — Logging chuẩn forensic — đủ chưa?

Audit log hiện ghi: - `nguoai_thuc_hien_id`, `actor_role` - `action`, `resource`, `tai_nguyen_id` - `description` tiếng Việt - `payload` JSON (before/after) - `ip_address`, `user_agent` - `createdAt` precision second

Đủ cho: - Truy ai làm gì lúc nào. - Reconstruct dữ liệu trước khi sửa. - Identify session anomaly (vd: 1 user login từ 2 IP khác nhau trong 1 phút).

Chưa đủ cho: - Failed login attempt (chưa log fail). - Tracking session hijack (chưa correlate user agent + IP fingerprint). - Long-term retention (chưa có archive policy → audit log lớn vô hạn).

N.14 — Penetration test chính thức — có không?

Trả lời thẳng: “Em chưa thuê pentest chính thức (do chi phí và quy trình). Em chỉ tự pentest cơ bản như mô tả ở N.3. Trước khi triển khai thật cho Học viện, sẽ đề xuất nhờ Phòng Bảo mật của Tổng cục II hoặc đơn vị chuyên môn quân đội thực hiện pentest.”

N.15 — Compliance / GDPR / Luật bảo vệ dữ liệu cá nhân Việt Nam

Trả lời: - Project xử lý dữ liệu cá nhân (CCCD, ngày sinh, quê quán, lịch sử công tác) — thuộc **dữ liệu cá nhân nhạy cảm** theo Nghị định 13/2023/NĐ-CP. - Đã có: - Audit log truy cập. - Phân quyền hạn chế chia sẻ. - Backup an toàn. - **Chưa có:** - Quyền xoá data theo yêu cầu cá nhân (right to erasure). - Export data theo định dạng chuẩn (right to portability). - Privacy policy hiển thị cho user. - Vì là hệ thống nội bộ quân sự, áp dụng quy chế quân đội khác với GDPR — cần tuân theo Nghị quyết của Quân uỷ Trung ương và quy chế của Học viện về bảo vệ thông tin.

N.16 — Tóm tắt đánh giá an toàn cuối cùng

Tổng kết 1 đoạn:

“Hệ thống của em đạt mức an toàn **xếp loại Khá** cho ứng dụng web nội bộ. **Đã chống được** 12/15 mục OWASP Top 10 ở mức tốt. **Còn 3 hạn chế** cần cải thiện trước khi mở rộng quy mô: (1) chưa có 2FA cho tài khoản đặc quyền cao, (2) chưa có cơ chế chống DBA insider threat, (3) chưa có pentest chính thức. Trong phạm vi project nội bộ LAN Học viện chạy ~50 user, mức độ an toàn này **đủ dùng** và **không có lỗ hổng cấp critical** mà em phát hiện được. Em đã ghi 3 hạn chế trên vào hướng phát triển ở Chương 6.”

N.17 — Câu hỏi 2FA — tại sao chưa làm?

Trả lời: - 2FA cho LAN nội bộ ít cần thiết hơn cho Internet — vì đã có lớp bảo vệ vật lý (vào phòng máy phải qua kiểm tra). - Triển khai 2FA cần infrastructure: SMS gateway (quân đội có riêng), TOTP app (Google Authenticator), hoặc hardware token (YubiKey). - Em đề xuất 2FA **chỉ cho SUPER_ADMIN và ADMIN** ở phiên bản kế tiếp.

Code stub:

```
// Tương lai: thêm cột totp_secret vào TaiKhoan
// Login flow: username + password → check totp_code (otplib) → cấp JWT
```

N.18 — Khi attacker đã vào được system rồi — incident response

Quy trình em đề xuất (chưa formalize):

1. **Detection:** Audit log alert khi có pattern bất thường (vd: 1 ADMIN xóa > 100 record / phút).
2. **Containment:**
 - Vô hiệu tài khoản nghi ngờ qua endpoint admin.
 - Đổi mọi JWT_SECRET → buộc tất cả user đăng nhập lại.
3. **Eradication:** Restore DB từ backup gần nhất trước thời điểm bị xâm nhập.
4. **Recovery:** Verify integrity → khởi động lại dịch vụ.
5. **Post-mortem:** Phân tích audit log để hiểu vector xâm nhập, fix lỗ hổng.

Hiện tại chưa có: - Automated alerting (Slack/email khi pattern bất thường). - Runbook chi tiết. - Drill incident response định kỳ.

N.19 — Câu chốt khi bị truy vấn nặng về bảo mật

Trả lời mẫu cho hội đồng:

“Em hiểu rằng bảo mật là quá trình liên tục, không có hệ thống nào tuyệt đối an toàn. Trong phạm vi đồ án sinh viên với 6 tháng thực hiện, em đã ưu tiên (1) chống các lỗ hổng OWASP Top 10 phổ biến nhất, (2) áp dụng nguyên tắc đặc quyền tối thiểu qua phân quyền 4 cấp, (3) ghi audit log đầy đủ để truy hồi khi sự cố. Những phần còn thiếu như 2FA, encryption at rest, pentest chính thức — em đã ghi vào hướng phát triển và sẽ triển khai khi có thêm nhân lực hỗ trợ. Em xin tiếp thu mọi góp ý của thầy/cô để cải thiện thêm.”

O. Truy vấn nâng cao và tối ưu cơ sở dữ liệu

Phần này tập trung vào những truy vấn thực tế phức tạp mà hội đồng giới DB có thể yêu cầu viết tại chỗ. Mỗi câu có (1) mô tả nghiệp vụ, (2) Prisma cách làm, (3) SQL tương đương, (4) giải thích kỹ thuật và (5) cách tối ưu.

O.1 — Cây đơn vị nhiều cấp: tìm tất cả đơn vị con cháu (recursive)

Nghệ vụ: Một CQDV có nhiều DVTT con; cần lấy danh sách tất cả quân nhân thuộc cây con của 1 CQDV bất kỳ. Schema hiện tại 2 cấp (CQDV → DVTT). Nếu mở rộng nhiều cấp sẽ cần recursive CTE.

SQL recursive CTE:

```
WITH RECURSIVE don_vi_tree AS (
  SELECT id, parent_id, ten_don_vi, 0 AS depth
```

```

FROM "DonVi" WHERE id = $1

UNION ALL

SELECT dv.id, dv.parent_id, dv.ten_don_vi, t.depth + 1
FROM "DonVi" dv
JOIN don_vi_tree t ON dv.parent_id = t.id
WHERE t.depth < 10 -- chống infinite loop
)
SELECT qn.* FROM "QuanNhan" qn
WHERE qn.don_vi_id IN (SELECT id FROM don_vi_tree);

```

Prisma không hỗ trợ recursive CTE native → fallback \$queryRaw.

Trong project hiện tại: vì cây 2 cấp em dùng cách đơn giản ở unitFilter.ts:

```

const dvttIds = await donViTrucThuocRepository.findIdsByCoQuanDonViId(cqdvId);
const personnel = await prisma.quanNhan.findMany({
  where: { OR: [{ co_quan_don_vi_id: cqdvId }, { don_vi_truc_thuoc_id: { in: dvttIds } } ] }
});

```

Phản biện: “Sao không dùng nested set / materialized path?” → “Cây 2 cấp chưa cần. Khi mở > 3 cấp em chuyển materialized path (path: '/cqdv1/dvtt2/td3') để query 1 phát.”

O.2 — Top N per group: 3 danh hiệu mới nhất của mỗi quân nhân

SQL với ROW_NUMBER:

```

WITH ranked AS (
  SELECT *,
    ROW_NUMBER() OVER (PARTITION BY quan_nhan_id ORDER BY nam DESC) AS rn
  FROM "DanhHieuHangNam"
)
SELECT * FROM ranked WHERE rn <= 3;

```

SQL alternative LATERAL JOIN:

```

SELECT qn.id, dh.*
FROM "QuanNhan" qn
LEFT JOIN LATERAL (
  SELECT * FROM "DanhHieuHangNam" dh
  WHERE dh.quan_nhan_id = qn.id
  ORDER BY nam DESC
  LIMIT 3
) dh ON TRUE;

```

Prisma: Không hỗ trợ window/lateral → \$queryRaw. Hoặc workaround query toàn bộ rồi filter trong code (chỉ chấp nhận khi dataset nhỏ).

O.3 — Tìm chuỗi CSTĐCS dài nhất liên tục — gaps and islands

SQL:

```

WITH numbered AS (
  SELECT quan_nhan_id, nam,
    nam - ROW_NUMBER() OVER (PARTITION BY quan_nhan_id ORDER BY nam) AS grp
  FROM "DanhHieuHangNam"
  WHERE danh_hieu = 'CSTĐCS'
),
streaks AS (
  SELECT quan_nhan_id, grp, COUNT(*) AS streak_length,

```

```

    MIN(nam) AS streak_start, MAX(nam) AS streak_end
  FROM numbered
  GROUP BY quan_nhan_id, grp
)
SELECT quan_nhan_id, MAX(streak_length) AS longest_streak
FROM streaks
GROUP BY quan_nhan_id;

```

Giải thích “gaps and islands”: Nếu các năm liên tục thì hiệu giữa nam và ROW_NUMBER không đổi. Khi có gap, hiệu thay đổi → group theo hiệu.

Trong project: Em không tính qua SQL trực tiếp mà dùng JavaScript trong lastFlagYearInChain của services/profile/annual.ts — đơn giản, dễ test, đủ nhanh cho dataset 1 quân nhân.

O.4 — Tìm gaps: năm nào quân nhân X bị đứt CSTDCS

SQL với generate_series:

```

WITH year_range AS (SELECT generate_series(2015, 2024) AS nam),
qn_dh AS (
  SELECT nam FROM "DanhHieuHangNam"
  WHERE quan_nhan_id = $1 AND danh_hieu = 'CSTDSC'
)
SELECT yr.nam AS missing_year
FROM year_range yr
LEFT JOIN qn_dh ON yr.nam = qn_dh.nam
WHERE qn_dh.nam IS NULL
ORDER BY yr.nam;

```

O.5 — Pagination tối ưu: tổng count + data trong 1 query

SQL với COUNT(*) OVER():

```

SELECT *, COUNT(*) OVER() AS total_count
FROM "QuanNhan"
WHERE co_quan_don_vi_id = $1
ORDER BY ho_ten
LIMIT 50 OFFSET 100;

```

Mỗi row trả về kèm total_count (giống nhau ở mọi row). Tiết kiệm 1 round-trip DB so với Promise.all([count, findMany]).

Trade-off: Phức tạp hơn, mất type-safe Prisma → em vẫn giữ 2 query với Promise.all cho code rõ ràng.

O.6 — Anti-join: quân nhân chưa có đề xuất nào trong 3 năm

SQL với NOT EXISTS:

```

SELECT qn.* FROM "QuanNhan" qn
WHERE NOT EXISTS (
  SELECT 1 FROM "BangDeXuat" bdx
  WHERE bdx.nguoi_de_xuat_id IN (
    SELECT id FROM "TaiKhoan" WHERE quan_nhan_id = qn.id
  )
  AND bdx.nam >= EXTRACT(YEAR FROM CURRENT_DATE) - 3
);

```

Prisma none:

```

const forgotten = await prisma.quanNhan.findMany({
  where: {
    TaiKhoan: {

```



```

        DeXuatDaGui: { none: { nam: { gte: new Date().getFullYear() - 3 } } },
    },
});

```

O.7 — Conditional aggregation với FILTER

Nhiệm vụ: Dashboard — đếm pending/approved/rejected theo từng loại trong 1 query.

SQL với FILTER:

```

SELECT
    loai_de_xuat,
    COUNT(*) FILTER (WHERE status = 'PENDING') AS pending,
    COUNT(*) FILTER (WHERE status = 'APPROVED') AS approved,
    COUNT(*) FILTER (WHERE status = 'REJECTED') AS rejected,
    COUNT(*) AS total
FROM "BangDeXuat"
WHERE nam = $1
GROUP BY loai_de_xuat;

```

FILTER là syntax SQL chuẩn, gọn hơn SUM (CASE WHEN ... THEN 1 ELSE 0 END).

O.8 — Pivot: chuyển hàng thành cột (số CSTDCS theo đơn vị × năm)

SQL với crosstab (cần extension tablefunc):

```

CREATE EXTENSION IF NOT EXISTS tablefunc;

SELECT * FROM crosstab(
    $$
        SELECT cqdv.ten_don_vi, dh.nam, COUNT(*)
        FROM "DanhHieuHangNam" dh
        JOIN "QuanNhan" qn ON qn.id = dh.quan_nhan_id
        JOIN "CoQuanDonVi" cqdv ON cqdv.id = qn.co_quan_don_vi_id
        WHERE dh.danh_hieu = 'CSTDCS' AND dh.nam BETWEEN 2020 AND 2024
        GROUP BY cqdv.ten_don_vi, dh.nam
        ORDER BY 1, 2
    $$,
    $$ SELECT generate_series(2020, 2024) $$
) AS ct(ten_don_vi TEXT, "2020" BIGINT, "2021" BIGINT, "2022" BIGINT, "2023" BIGINT, "2024" BIGINT);

```

Cách đơn giản hơn: truy vấn long-format rồi pivot trong code TypeScript.

O.9 — DISTINCT ON: lấy bản ghi mới nhất của mỗi nhóm

SQL DISTINCT ON (PostgreSQL-specific):

```

SELECT DISTINCT ON (nguai_thuc_hien_id) *
FROM "SystemLog"
WHERE action = 'LOGIN'
ORDER BY nguoi_thuc_hien_id, "createdAt" DESC;

```

SQL chuẩn portable:

```

SELECT * FROM "SystemLog" l1
WHERE action = 'LOGIN'
    AND "createdAt" = (
        SELECT MAX("createdAt") FROM "SystemLog" l2
        WHERE l2.nguai_thuc_hien_id = l1.nguai_thuc_hien_id AND l2.action = 'LOGIN'
    );

```

DISTINCT ON nhanh hơn ~2-3 lần nếu có index (ngoi_thuc_hien_id, createdAt DESC).

O.10 — Cumulative sum: tổng đề xuất tích lũy theo tháng

SQL với SUM() OVER (ORDER BY):

```
SELECT
    DATE_TRUNC('month', "createdAt") AS thang,
    COUNT(*) AS so_de_xuat_thang,
    SUM(COUNT(*)) OVER (ORDER BY DATE_TRUNC('month', "createdAt")) AS tinh_luy
FROM "BangDeXuat"
WHERE EXTRACT(YEAR FROM "createdAt") = 2025
GROUP BY DATE_TRUNC('month', "createdAt")
ORDER BY thang;
```

Kết hợp aggregation + window function — SUM(COUNT(*)) chạy sau GROUP BY.

O.11 — Median với PERCENTILE_CONT

SQL:

```
SELECT
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY EXTRACT(YEAR FROM AGE(NOW(), ngay_nhap_ngu))
    AS so_nam_phuc_vu_trung_vi
FROM "QuanNhan"
WHERE ngay_nhap_ngu IS NOT NULL;
```

PERCENTILE_CONT(0.5) = median; (0.25) = quartile 1.

O.12 — Tìm bản ghi trùng lặp (duplicate detection)

SQL:

```
SELECT ho_ten, ngay_sinh, COUNT(*) AS so_lan_trung, ARRAY_AGG(id) AS cac_id
FROM "QuanNhan"
WHERE ho_ten IS NOT NULL AND ngay_sinh IS NOT NULL
GROUP BY ho_ten, ngay_sinh
HAVING COUNT(*) > 1;
```

ARRAY_AGG(id) gom các id trùng vào 1 mảng → admin reconcile.

O.13 — Quân nhân sắp đến mốc niên hạn 10/15/20 năm trong 6 tháng

SQL:

```
SELECT qn.id, qn.ho_ten, qn.ngay_nhap_ngu,
    EXTRACT(YEAR FROM AGE(NOW(), ngay_nhap_ngu)) AS so_nam_phuc_vu,
    CASE
        WHEN EXTRACT(YEAR FROM AGE(NOW() + INTERVAL '6 months', ngay_nhap_ngu)) = 10 THEN 'H10'
        WHEN EXTRACT(YEAR FROM AGE(NOW() + INTERVAL '6 months', ngay_nhap_ngu)) = 15 THEN 'H15'
        WHEN EXTRACT(YEAR FROM AGE(NOW() + INTERVAL '6 months', ngay_nhap_ngu)) = 20 THEN 'H20'
    END AS sap_dat_moc
FROM "QuanNhan" qn
WHERE EXTRACT(YEAR FROM AGE(NOW() + INTERVAL '6 months', ngay_nhap_ngu)) IN (10, 15, 20)
    AND NOT EXISTS (
        SELECT 1 FROM "KhenThuongHCCSVV" kt
        WHERE kt.quan_nhan_id = qn.id
    );
```

O.14 — Đề xuất pending lâu nhất, cảnh báo SLA > 7 ngày

Prisma:

```
const stale = await prisma.bangDeXuat.findMany({
  where: {
    status: 'PENDING',
    createdAt: { lt: new Date(Date.now() - 7 * 24 * 60 * 60 * 1000) },
  },
  orderBy: { createdAt: 'asc' },
});
```

SQL kèm số ngày chờ:

```
SELECT bdx.*,
  EXTRACT(EPOCH FROM (NOW() - bdx."createdAt")) / 86400 AS so_ngay_cho
FROM "BangDeXuat" bdx
WHERE bdx.status = 'PENDING' AND bdx."createdAt" < NOW() - INTERVAL '7 days'
ORDER BY bdx."createdAt" ASC;
```

O.15 — KPI người duyệt: tỷ lệ approve, thời gian duyệt trung bình

SQL:

```
SELECT
  t.username AS admin,
  COUNT(*) AS tong_duyet,
  COUNT(*) FILTER (WHERE bdx.status = 'APPROVED') AS approved,
  COUNT(*) FILTER (WHERE bdx.status = 'REJECTED') AS rejected,
  ROUND(100.0 * COUNT(*) FILTER (WHERE bdx.status = 'APPROVED') / NULLIF(COUNT(*), 0), 2)
  AS ty_le_approve_pct,
  AVG(EXTRACT(EPOCH FROM (bdx.ngay_duyet - bdx."createdAt"))) / 3600 AS gio_duyet_trung_binh
FROM "BangDeXuat" bdx
JOIN "TaiKhoan" t ON t.id = bdx.nguoi_duyet_id
WHERE bdx.ngay_duyet IS NOT NULL
  AND bdx.ngay_duyet >= NOW() - INTERVAL '6 months'
GROUP BY t.id, t.username
ORDER BY tong_duyet DESC;
```

NULLIF(COUNT(*), 0) chống chia cho 0.

O.16 — Self-join: cặp danh hiệu liên tiếp 2 năm

Nhiệm vụ: Tìm cặp (năm n, năm n+1) mà quân nhân đạt CSTĐCS cả 2 — chuẩn bị rule BKBQP.

SQL:

```
SELECT a.quan_nhan_id, a.nam AS nam_dau, b.nam AS nam_sau
FROM "DanhHieuHangNam" a
INNER JOIN "DanhHieuHangNam" b
  ON a.quan_nhan_id = b.quan_nhan_id
  AND b.nam = a.nam + 1
WHERE a.danh_hieu = 'CSTDSCS' AND b.danh_hieu = 'CSTDSCS';
```

O.17 — JSON aggregation: lịch sử thao tác dạng JSON array

SQL với jsonb_agg:

```
SELECT
  tai_nguyen_id,
  jsonb_agg(
    jsonb_build_object(
```

```

        'action', action, 'time', "createdAt",
        'by', actor_role, 'description', description
    )
    ORDER BY "createdAt" DESC
) AS history
FROM "SystemLog"
WHERE resource = 'personnel' AND tai_nguyen_id = $1
GROUP BY tai_nguyen_id;

```

Trả về 1 row, cột history là mảng JSON đã sort.

O.18 — UNION ALL: gộp đề xuất cá nhân + đơn vị

SQL:

```

SELECT id, 'CA_NHAN' AS doi_tuong, nam, status, "createdAt"
FROM "BangDeXuat"
WHERE loai_de_xuat IN ('CA_NHAN_HANG_NAM', 'NIEN_HAN', 'CONG_HIEN', 'NCKH', 'HC_QKQT', '
UNION ALL
SELECT id, 'DON_VI' AS doi_tuong, nam, status, "createdAt"
FROM "BangDeXuat"
WHERE loai_de_xuat = 'DON_VI_HANG_NAM'
ORDER BY "createdAt" DESC LIMIT 50;

```

UNION ALL không loại trùng → rõ hơn UNION.

O.19 — Full-text search với tsvector

Setup index:

```

ALTER TABLE "BangDeXuat" ADD COLUMN search_vector tsvector
GENERATED ALWAYS AS (to_tsvector('simple', coalesce(ghi_chu, ''))) STORED;
CREATE INDEX idx_bdx_search ON "BangDeXuat" USING GIN(search_vector);

```

Query:

```

SELECT * FROM "BangDeXuat"
WHERE search_vector @@ plainto_tsquery('simple', $1)
ORDER BY ts_rank(search_vector, plainto_tsquery('simple', $1)) DESC;

```

Hiện tại: em dùng ILIKE '%keyword%' đơn giản — không scale khi text dài. Đề xuất khi dataset > 100k bản ghi.

O.20 — Khoảng thời gian phức tạp: quý hiện tại, 12 tháng trượt

SQL — quý hiện tại:

```

SELECT * FROM "BangDeXuat"
WHERE "createdAt" >= DATE_TRUNC('quarter', CURRENT_DATE)
AND "createdAt" < DATE_TRUNC('quarter', CURRENT_DATE) + INTERVAL '3 months';

```

SQL — 12 tháng trượt:

```

SELECT
    TO_CHAR(DATE_TRUNC('month', "createdAt"), 'MM/YYYY') AS thang,
    COUNT(*)
FROM "BangDeXuat"
WHERE "createdAt" >= DATE_TRUNC('month', CURRENT_DATE - INTERVAL '11 months')
GROUP BY DATE_TRUNC('month', "createdAt")
ORDER BY 1;

```

O.21 — Skip locked: queue worker pattern

Nhiệm vụ: Background worker xử lý job recalc, không lock toàn bảng.

SQL FOR UPDATE SKIP LOCKED:

```
BEGIN;
SELECT * FROM "RecalcQueue"
WHERE status = 'PENDING'
ORDER BY created_at LIMIT 10
FOR UPDATE SKIP LOCKED;

UPDATE "RecalcQueue" SET status = 'DONE' WHERE id = ANY($1);
COMMIT;
```

SKIP LOCKED cho phép nhiều worker chạy song song không tranh chấp 10 row giống nhau.

Trong project: chưa có queue worker. Khi scale sẽ chuyển BullMQ + Redis hoặc Postgres queue + skip locked.

O.22 — Optimistic locking chống lost update

Vấn đề: Hai user mở form sửa cùng quân nhân, save gần nhau → user save sau đè user save trước.

Cơ chế: Thêm cột version: Int @default(0).

```
const personnel = await prisma.quanNhan.findUnique({ where: { id } });
// FE submit kèm personnel.version

const updated = await prisma.quanNhan.updateMany({
  where: { id, version: clientVersion },
  data: { ...newData, version: clientVersion + 1 },
});
if (updated.count === 0) {
  throw new ValidationError('Bản ghi vừa được người khác sửa, vui lòng tải lại.');
```

Trong project: chưa có cột version riêng. Em dựa vào updatedAt cho 1 vài endpoint quan trọng — đề xuất cải tiến.

O.23 — Pessimistic lock với SELECT FOR UPDATE

Khi cần: Ngăn 2 transaction đọc cùng row rồi update đè.

```
await prisma.$transaction(async tx => {
  await tx.$queryRaw`SELECT * FROM "TaiKhoan" WHERE id = ${id} FOR UPDATE`;
  await tx.taiKhoan.update({ where: { id }, data: { ... } });
});
```

Trade-off: Block transaction khác → dùng cẩn thận, transaction phải nhanh (< 100 ms).

O.24 — Advisory lock cho job singleton

SQL:

```
SELECT pg_try_advisory_lock(12345);
-- ... chạy job ...
SELECT pg_advisory_unlock(12345);
```

Trong project: Em dùng flag isRunning trong process — đủ cho single instance. Khi cluster phải chuyển advisory lock.

O.25 — EXPLAIN ANALYZE để debug query chậm

Cách dùng:

EXPLAIN (**ANALYZE**, BUFFERS, FORMAT JSON)

SELECT * **FROM** "DanhHieuHangNam" **WHERE** quan_nhan_id = 'cl...' **AND** nam **BETWEEN** 2020 **AND** 202

Đọc output: - Seq Scan → quét toàn bảng → cần index. - Index Scan → dùng index → tốt. - Bitmap Heap Scan → dùng index nhưng nhiều row → trung bình. - cost=... ước tính, actual time=... thực tế. - Estimate sai nhiều → chạy ANALYZE <table> cập nhật statistics.

Trong project: chưa có endpoint chậm critical. Bật log: ['query'] Prisma để xem SQL sinh ra, EXPLAIN ANALYZE thủ công nếu cần.

O.26 — Index types: B-tree, GIN, GiST, partial, covering

Loại	Khi dùng	Ví dụ
B-tree (mặc định)	Equality, range, ORDER BY	@@index([action, createdAt]) SystemLog
GIN	JSONB, array, full-text	(đề xuất) cho payload JSONB
GiST	Range, geometry	(chưa dùng)
Partial	Index chỉ cho subset row	WHERE status = 'PENDING'
Covering (INCLUDE)	Đọc data từ index, không heap fetch	INCLUDE (ho_ten)

Partial index:

```
CREATE INDEX idx_pending_proposals ON "BangDeXuat" ("createdAt")  
WHERE status = 'PENDING';
```

Index nhỏ hơn, query “lấy đề xuất pending mới” cực nhanh.

O.27 — Khi nào nên thêm index? Đánh đổi gì?

Thêm khi: - Query thường xuyên (>100 lần/ngày). - Cột dùng trong WHERE / JOIN / ORDER BY. - Selectivity cao (vd: status 4 giá trị → index không hiệu quả; CCCD unique → rất hiệu quả).

Đánh đổi: - Index tốn disk (~10-30 % size table). - Mỗi INSERT/UPDATE/DELETE phải update index → ghi chậm. - Quá nhiều → planner chọn nhầm.

Quy tắc: Bắt đầu 0 index, thêm theo pg_stat_user_indexes cho thấy index không dùng → drop; query slow → thêm.

O.28 — VACUUM, ANALYZE, REINDEX — khi nào?

- **ANALYZE:** Cập nhật statistics planner. Sau bulk INSERT/UPDATE > 10 % size.
- **VACUUM:** Giải phóng space của row đã DELETE/UPDATE. Autovacuum chạy tự động.
- **VACUUM FULL:** Compact table — block toàn table, dùng khi disk phình.
- **REINDEX:** Rebuild index khi bloat.

Trong project: dựa vào autovacuum mặc định, không can thiệp.

O.29 — Connection pooling tuning

Mặc định Prisma: connection_limit = num_cpu * 2 + 1.

Thay đổi qua URL:

```
postgresql://user:pass@host:5432/db?connection_limit=20&pool_timeout=10
```

Khi pool exhaust: Request mới phải đợi → log error “Timed out fetching a new connection”. Cải thiện: 1. Tăng connection_limit (không quá max_connections Postgres). 2. Dùng PgBouncer transaction pooler. 3. Giảm transaction time.

O.30 — Transaction isolation levels

Level	Dirty read	Non-repeatable read	Phantom read	Mặc định Postgres
READ UNCOMMITTED	Có	Có	Có	Không hỗ trợ
READ COMMITTED	Không	Có	Có	Có (mặc định)
REPEATABLE READ	Không	Không	Postgres: không	Có
SERIALIZABLE	Không	Không	Không	Có (chậm nhất)

Trong project: READ COMMITTED đủ cho > 95 % case. Chỉ dùng SERIALIZABLE khi rule cần nhất quán cao (vd: gán số quyết định không trùng):

```
await prisma.$transaction(
  async tx => { ... },
  { isolationLevel: Prisma.TransactionIsolationLevel.Serializable }
);
```

O.31 — Tại sao CUID thay vì UUID hay BIGSERIAL?

- **BIGSERIAL:** dễ đoán → enumerate attack. Defense in depth → không nên expose ID liên tục.
- **UUID v4:** 128-bit random, an toàn. Nhược: không sortable theo thời gian.
- **CUID:** 25 ký tự, sortable theo thời gian (prefix timestamp), URL-safe, collision-resistant. Phù hợp web app.

Schema hiện tại:

```
id String @id @default(cuid()) @db.VarChar(30)
```

CUID2 (mới hơn) còn an toàn hơn: thêm entropy, không leak server identity. Có thể migrate sau.

O.32 — FK award tables trở tới FileQuyếtĐịnh.so_quyet_dinh (string) thay vì id (cuid) — đánh đổi gì?

Thiết kế hiện tại: 8 bảng award (ThanhTichKhoaHoc, DanhHieuHangNam, KhenThuongHCBVTQ, HuanChuongQuanKyQuangThang, KyNiemChuongVSNXDQDNDVN, KhenThuongHCCSVV, KhenThuongDotXuat, DanhHieuDonViHangNam) — tổng 13 FK relations — đều trở tới FileQuyếtĐịnh.so_quyet_dinh (business identifier), không phải FileQuyếtĐịnh.id (surrogate cuid).

```
model ThanhTichKhoaHoc {
  so_quyet_dinh String? @db.VarChar(100)
  FileQuyếtĐịnh FileQuyếtĐịnh? @relation(
    fields: [so_quyet_dinh],
    references: [so_quyet_dinh], // → string, không phải id
    onUpdate: Cascade,
    onDelete: Restrict
  )
}
```

Pros (lý do giữ thiết kế):

Điểm	Diễn giải
Số QĐ là natural business identifier	Mỗi QĐ chỉ có 1 số duy nhất theo luật ban hành — có thuộc tính “stable enough”
Hiển thị không cần JOIN	record.so_quyet_dinh đã đủ cho UI; không cần include: { FileQuyếtĐịnh }
JSON proposal payload đồng bộ với relational	data_danh_hieu: [{ so_quyet_dinh: 'QD-123' }] cùng kiểu với cột relational — không cần lookup id khi build payload

Điểm	Diễn giải
ON UPDATE CASCADE xử lý rename tự động	Postgres tự update tất cả award rows khi rename — code không phải biết
Debug DB dễ	SELECT * FROM ThanhTichKhoaHoc đọc thấy “QD-123/2026” thay vì “cmphxxx”

Cons (nhược điểm thật sự):

Vấn đề	Hệ quả
Anti-pattern phổ biến (FK to mutable business key) Rename cost O(refs)	Trái standard practice — FK nên trở surrogate immutable DB cascade quét 8 bảng + 10 FK constraints; app phải chạy <code>cascadeRename.ts</code> rewrite JSON tất cả proposal (PENDING + APPROVED + REJECTED) để UI nhất quán
Index lớn hơn	@db.VarChar(100) vs @db.VarChar(30) cuid — index disk + memory footprint cao hơn ~3x
FK validation chậm hơn marginal	String comparison vs cuid comparison
Khó multi-tenant trong tương lai	Nếu cần đồng QĐ trùng số ở 2 tenant khác nhau → unique constraint global breaks
Khó versioning QĐ	Nếu cần lịch sử các phiên bản QĐ → cần id stable làm version chain

Alternative design (surrogate id FK):

```
model ThanhTichKhoaHoc {
  file_quyet_dinh_id String? @db.VarChar(30)
  FileQuyếtDinh FileQuyếtDinh? @relation(
    fields: [file_quyet_dinh_id],
    references: [id], // → cuid immutable
    onDelete: Restrict
  )
}
```

- Read: include: { FileQuyếtDinh: { select: { so_quyet_dinh, file_path } } } → record.FileQuyếtDinh.so_quyet_dinh
- Rename: 1 UPDATE row trong FileQuyếtDinh, JOIN reads tự thấy số mới — không cascade
- Code: bỏ được `cascadeRename.ts` cho award tables (vẫn cần cho proposal JSON nếu JSON giữ string)

Vì sao không refactor về surrogate id?

1. **Scale hiện tại:** ~hàng trăm QĐ, vài nghìn award rows → cost cascade rename = miligiây, không phải hot path
2. **Tần suất rename thấp:** chỉ khi fix typo lúc tạo mới — QĐ đã ban hành không bao giờ đổi số
3. **Refactor scope quá rộng:** 59 file BE + 52 file FE + 48 file test + 1 migration + docs → 160+ file → risk:reward không cân với benefit “cleaner code”
4. **Đã có test coverage:** `tests/approve/decisionMappings.test.ts`, `tests/scenarios/*` đã cover cascade rename + concurrent approve — refactor sẽ phá hết, viết lại tốn công
5. **Code gốc behavior đúng:** đáp ứng đầy đủ rename số QĐ, thay file_path, restrict delete khi còn ref — không có bug user-facing

Phạm vi cascade JSON proposal — tại sao quét cả APPROVED/REJECTED, không chỉ PENDING?

`cascadeRename.ts` rewrite JSON trên **mọi proposal status**, không filter theo PENDING. Lý do:

1. **UI consistency là ưu tiên:** `ProposalDetailModal` đọc số QĐ từ `proposal.data_danh_hieu` (JSON snapshot); `personnel/[id]/annual-rewards` đọc từ `DanhHieuHangNam.so_quyet_dinh` (FK đã cascade). Nếu chỉ rewrite PENDING, user xem 1 proposal APPROVED sẽ thấy số QĐ cũ trong khi award list hiển thị số mới — confused.

2. **Audit không bị mất:** lịch sử rename đã được ghi đầy đủ trong `system_logs` (resource = decisions, action = UPDATE, có old/new value qua audit middleware). JSON snapshot không phải single source of truth cho audit.
3. **Domain quản đội ưu tiên consistency:** user (Admin Phòng Chính trị) không phân biệt “snapshot lúc submit” vs “state hiện tại”; họ chỉ thấy 2 view cùng entity → expect cùng giá trị.
4. **Cost vẫn nhỏ:** scale ĐATN có ~vài trăm proposal max, scan + rewrite mất < 200ms tổng.

```
// services/decision/cascadeRename.ts
const proposals = await tx.bangDeXuat.findMany({
  // không có where filter — quét mọi status
  select: { id, data_danh_hieu, data_thanh_tich, data_nien_han, data_cong_hien },
});
// rewrite từng row nếu chứa oldSqd
```

Khi nào nên refactor (tiêu chí cụ thể):

- Award rows > 10k → cascade rename lock contention thấy được
- Cần multi-tenant (khác đơn vị có cùng số QĐ)
- Cần file version chain (QĐ này từng dùng file A, sau replace bằng file B — muốn keep history)
- Audit yêu cầu immutable trail cho FK references

Đáp khi hội đồng hỏi “tại sao không dùng id làm FK?”:

“Đây là trade-off em đã cân nhắc. Em chọn string FK vì 3 lý do: (1) số quyết định là natural business identifier — mỗi QĐ chỉ có 1 số duy nhất theo luật, (2) JSON payload proposal cũng lưu string nên đồng bộ, (3) cost cascade rename ở scale ĐATN không đáng kể. Em thừa nhận theo standard practice nên FK trở surrogate id immutable — em đã đánh giá refactor sẽ đụng hơn 160 file. Lợi ích chính là rename O(1) thay vì O(refs), nhưng ở scale này không justify được effort. Em ghi nhận trong hướng phát triển: nếu hệ thống mở rộng quá ngưỡng (>10k award rows, multi-tenant, file versioning) thì sẽ migrate.”

Phòng câu phản biện: “Nếu một ngày luật đổi format số QĐ — tất cả award phải update theo, có scalable không?”

“ON UPDATE CASCADE tự xử lý ở DB level, không cần code app. Với mỗi QĐ có ~vài award rows, rename 1 QĐ = vài UPDATE — mili giây. Nếu cần batch rename hàng loạt theo format mới, em sẽ viết script SQL chạy ngoài giờ cao điểm, tốn vài giây. Bottleneck thực sự là `cascadeRename.ts` cho JSON proposal — phần này quét toàn bộ `BangDeXuat` (mọi status, để UI nhất quán giữa proposal detail và award list); ở scale này còn rất nhanh nhưng nếu cần optimize có thể thêm `WHERE data_::text LIKE '%' || oldSqd || '%'` filter trước.”

O.33 — Prisma sinh SQL kém hiệu quả — fix thế nào?

Triệu chứng: Query Prisma chậm, EXPLAIN cho thấy SQL Prisma sinh ra dùng nhiều JOIN không cần.

Cách debug: Bật log: `['query']` → copy SQL → EXPLAIN ANALYZE trong psql.

Workaround: - Đơn giản hoá include. - Tách thành 2 query với `Promise.all` thay vì 1 mega-query. - Fallback `$queryRaw` cho query phức tạp.

```
// Chậm — 1 query 4 cấp
const data = await prisma.quanNhan.findMany({
  include: {
    DanhHieuHangNam: { include: { FileQuyDetDinh: true } },
    LichSuChucVu: { include: { ChucVu: true } },
  },
});

// Nhanh — 3 query song song + ghép trong code
const [quanNhans, danhHieus, lichSus] = await Promise.all([
  prisma.quanNhan.findMany(),
  prisma.danhHieuHangNam.findMany({ include: { FileQuyDetDinh: true } }),
  prisma.lichSuChucVu.findMany({ include: { ChucVu: true } }),
]);
```

O.34 — Migration với data transformation phức tạp (zero downtime)

Tình huống: Đổi `cap_bac` String thành `cap_bac_id` String FK referencing bảng mới CapBac.

Quy trình: 1. **Migration 1:** Tạo bảng CapBac + cột `cap_bac_id` mới (nullable). Schema cũ giữ nguyên. 2. **Backfill:** Script SQL INSERT INTO CapBac các giá trị unique từ `QuanNhan.cap_bac`. Update `cap_bac_id` based trên text. 3. **Đổi code:** App ghi cả 2 cột (dual-write). Đọc cột mới, fallback cột cũ. 4. **Migration 2:** Drop cột cũ. Đổi code đọc/ghi chỉ cột mới.

Trong project: Em có scenario tương tự khi rename `so_quyet_dinh` thành hard FK với `FileQuyếtDinh` (commit 29f741f) — dùng raw SQL ALTER TABLE ... RENAME COLUMN để giữ data, sau đó db push.

O.35 — Backup chiến lược chuyên sâu

Cấp	Công cụ	Đặc điểm
Logical	pg_dump	SQL text, dễ restore từng phần, chậm cho DB lớn. (Project hiện không dùng pg_dump; tự sinh INSERT.)
Physical	pg_basebackup	Snapshot file system, nhanh, restore toàn bộ.
WAL archiving	archive_command	Continuous backup → point-in-time recovery (PITR).

Strategy đề xuất khi scale: - pg_basebackup hằng tuần. - WAL archive liên tục. - pg_dump hằng ngày (logical, dễ migrate). - Test restore mỗi tháng.

O.36 — Prisma version 6 breaking change — em xử lý sao?

- Prisma đánh dấu rõ breaking change trong CHANGELOG.
- Em khai báo version trong package.json ("@prisma/client": "^6.17.1", "prisma": "^6.17.1" — Prisma 6, dùng caret ^).
- Khi upgrade: đọc migration guide, chạy regression test (937 ca), sửa breaking nếu có.
- Có thể giữ version cũ vài năm nếu Prisma vẫn hỗ trợ.

Phụ chú: 20 câu hỏi tú thường gặp khi bảo vệ web

1. Demo nhanh chức năng quan trọng nhất.
2. Vẽ tay sơ đồ kiến trúc trên giấy.
3. Giải thích flow đăng nhập từ click submit đến cookie set.
4. Tại sao chia thành nhiều bảng thay vì 1 bảng to?
5. Foreign key onDelete Cascade vs Restrict — khi nào dùng cái nào?
6. Tại sao dùng cuid thay vì uuid hoặc auto-increment?
7. Migration đã chạy 50 lần — làm sao quản lý version?
8. Test coverage 85 % nghĩa là gì? 15 % còn lại là gì?
9. Nếu bỏ Prisma, cần thay đổi bao nhiêu file?
10. Prisma có hỗ trợ MySQL không? Migrate cần làm gì?
11. JWT secret leak — quy trình rotation thế nào?
12. Chứng thực hai yếu tố (2FA) — em có nghĩ đến không?
13. Vi phạm GDPR / Luật bảo vệ dữ liệu cá nhân — có quan tâm không?
14. Logging có bao gồm password không? (Trả lời: KHÔNG)
15. Nếu admin xóa nhầm 100 quân nhân — recovery thế nào?
16. Có rate-limit cho download file không?
17. CORS đặt * không? (Trả lời: KHÔNG, whitelist nhiều origin qua ALLOWED_ORIGINS)
18. Đo performance bằng tool gì? (Có thể dùng autocannon, wrk cho load test)
19. CI/CD có không? (Hiện chưa, đề xuất GitHub Actions tương lai)

20. Nếu được làm lại, em sẽ thay đổi gì? (Câu hỏi đánh giá tự phản tỉnh — có sẵn câu trả lời)

Câu trả lời cho câu 20 — “Em sẽ làm khác gì?”

“Có ba điều em sẽ làm khác. **Một**, em sẽ áp dụng Repository pattern ngay từ đầu thay vì tách sau khi service đã lớn — sẽ tiết kiệm 1 tuần refactor. **Hai**, em sẽ viết test cho Controller layer ngay từ đầu, không chỉ Service — coverage Controller hiện chỉ 60 %. **Ba**, em sẽ tách schema Zod ra một workspace package dùng chung giữa BE và FE ngay từ đầu (thay vì khai báo song song) để loại bỏ phần duplicate hiện tại. Tuy nhiên những điểm này không phải critical, em vẫn hài lòng với kiến trúc tổng thể đã chọn.”

P. Phạm vi đề tài — đã làm và hướng phát triển

P.1 — Đề tài đã làm được những gì?

Ngắn (đọc thuộc — dùng mở đầu phần trình bày):

“Hệ thống quản lý khen thưởng cán bộ, chiến sĩ Quân đội gồm 5 nhóm chức năng chính: quản lý quân nhân — đơn vị — tài khoản, 7 loại khen thưởng theo quy chế, quy trình đề xuất và phê duyệt có kiểm tra điều kiện tự động, phân quyền 4 cấp vai trò, thống kê và sao lưu dữ liệu. Phần mềm được kiểm thử với 937 ca kiểm thử.”

Chi tiết theo nhóm (dùng khi bị hỏi sâu):

Nhóm	Nội dung đã hoàn thành
Quân nhân & tổ chức	CRUD quân nhân với import Excel hàng loạt; quản lý đơn vị 2 cấp (CQĐV → ĐVTT); quản lý chức vụ và lịch sử chức vụ; tính tự động niên hạn theo ngày nhập ngũ
7 loại khen thưởng	Cá nhân hằng năm (5 danh hiệu: CSTDCS, CSTT, BKBQP, CSTDTQ, BKTTCP), Đơn vị hằng năm (ĐVQT/ĐVTT/BKBQP/BKTTCP), HCCSVV (3 hạng), HCQKQT, KNC VSNXD QĐNDVN, HCBVTQ (3 hạng), NCKH. Ngoài ra có khen thưởng đột xuất do ADMIN tạo trực tiếp (không qua pipeline đề xuất 3 bước)
Quy trình đề xuất	3 bước: soạn đề xuất → ADMIN phê duyệt + gán số quyết định + upload PDF → tự động import vào hồ sơ. Có thể chỉnh sửa trước phê duyệt
Điều kiện tự động	Kiểm tra chuỗi danh hiệu hằng năm (BKBQP mỗi 2 năm, CSTDTQ mỗi 3 năm, BKTTCP mỗi 7 năm); cửa sổ trượt; phát hiện lỗi đột; hiển thị gợi ý danh hiệu tiếp theo
Phân quyền RBAC	4 vai trò: SUPER_ADMIN → ADMIN → MANAGER → USER; lọc dữ liệu theo đơn vị tự động
Thông báo real-time	Socket.IO: ADMIN nhận thông báo khi có đề xuất mới; USER nhận khi đề xuất được duyệt/từ chối
Dashboard & xuất dữ liệu	Thống kê tổng quan; xuất Excel danh hiệu hằng năm; tải PDF quyết định đã lưu
Vận hành nội bộ	Sao lưu tự động theo lịch (SQL dump); audit log toàn bộ thao tác; JWT access/refresh + force-logout phiên cũ
Kiểm thử	937 ca kiểm thử Jest, 80 test file, coverage > 85 % cho services/profile, services/eligibility, services/proposal

P.2 — Những gì chưa làm được và lý do?

Ngắn: “Có 5 điểm còn hạn chế, chủ yếu do phạm vi đề tài tập trung vào nghiệp vụ cốt lõi của một đơn vị.”

Chi tiết:

Hạn chế	Lý do	Ảnh hưởng thực tế
Danh hiệu cao hơn BKTTC (Anh hùng LLVT, Anh hùng Lao động)	Quy trình xét duyệt cấp Nhà nước, nằm ngoài thẩm quyền cấp đơn vị	Hiện hệ thống thông báo “chưa hỗ trợ, sẽ phát triển trong thời gian tới” khi quân nhân đã có BKTTC
Cấu hình thời hạn đề xuất theo loại khen thưởng	DevZone có toggle <code>allow_annual</code> , <code>allow_hccsvv</code> , ... nhưng các toggle này chỉ bật/tắt tính năng import Excel hàng loạt — không kiểm soát việc tạo đề xuất. Chưa có cơ chế <code>SUPER_ADMIN</code> định nghĩa “loại X chỉ được đề xuất trong khoảng tháng Y–Z”	MANAGER tự nhập năm/tháng đề xuất tự do — hệ thống không ngăn đề xuất “quá sớm” hay “quá muộn” so với chu kỳ xét khen của đơn vị
Phân cấp đơn vị nhiều tầng	Hiện chỉ hỗ trợ 2 cấp (CQĐV → ĐVTT); nếu áp dụng cho Trung đoàn → Tiểu đoàn → Đại đội cần thêm cấp trong schema	MANAGER chỉ thấy dữ liệu đơn vị mình; cấp trên chưa có view tổng hợp nhiều đơn vị con
Xuất báo cáo tổng hợp dạng PDF/Word	Hiện chỉ upload PDF quyết định từ ngoài vào; hệ thống chưa tự tạo được văn bản đề xuất theo mẫu	Cán bộ vẫn phải soạn văn bản tay, chi dùng phần mềm để lưu trữ và tra cứu
Nhập hàng loạt lịch sử chức vụ	Lịch sử chức vụ hiện phải nhập từng dòng thủ công; chưa có template Excel import	Tốn thời gian khi khởi tạo dữ liệu ban đầu cho đơn vị nhiều quân nhân

P.3 — Hướng phát triển trong thời gian tới?

Ngắn (câu mở đầu): “Em đề xuất 5 hướng phát triển tiếp theo, đều bám sát thực tế vận hành nội bộ.”

- Cấu hình thời hạn đề xuất theo loại khen thưởng** — thêm setting vào DevZone cho phép `SUPER_ADMIN` định nghĩa “loại X được mở đề xuất từ tháng Y đến tháng Z mỗi năm” (vd: danh hiệu hằng năm chỉ được đề xuất từ tháng 11–12). BE thêm validation trong `submit.ts` đọc setting này trước khi cho phép tạo đề xuất. Kiến trúc `settingsHelper` + DevZone sẵn sàng để mở rộng.
- Hỗ trợ danh hiệu Anh hùng LLVT / Anh hùng Lao động** — nghiên cứu thêm quy trình xét duyệt cấp Quân khu/Bộ Quốc phòng, thêm loại đề xuất mới vào strategy registry mà không ảnh hưởng luồng hiện tại. Kiến trúc strategy pattern sẵn sàng cho extension này.
- Mở rộng phân cấp đơn vị** — nếu triển khai rộng hơn (Tiểu đoàn, Đại đội), cần cho phép CQĐV có nhiều cấp con. Schema hiện tại có thể mở rộng bằng cách thêm `parent_id` để quy vào bảng đơn vị và cập nhật `unitFilter` middleware.
- Xuất văn bản đề xuất theo mẫu** — dùng thư viện `docx` hoặc `pdfmake` để tự động tạo file Word/PDF theo mẫu biểu quân đội từ dữ liệu đề xuất đã lưu, giảm thao tác thủ công cho MANAGER.
- Import hàng loạt lịch sử chức vụ bằng Excel** — mở rộng luồng import hiện có (đã có helper `loadWorkbook`, `batchQueryPersonnel`), thêm template và route mới, không cần thay đổi kiến trúc.

P.4 — Câu hỏi hay gặp về phạm vi

“Phần mềm đã được dùng thực tế chưa?” → “Hiện đang ở giai đoạn thử nghiệm nội bộ, em đã demo với cán bộ phụ trách công tác khen thưởng và nhận phản hồi tích cực về luồng đề xuất và xét điều kiện tự động. Cần thêm giai đoạn nhập dữ liệu

thực tế và đào tạo người dùng trước khi bàn giao chính thức.”

“Nhiều người dùng cùng lúc thì hệ thống có chịu được không?” → “Phạm vi triển khai là mạng nội bộ một đơn vị, số người dùng đồng thời thực tế khoảng 10–30 người. Ở quy mô đó Express + PostgreSQL hoàn toàn đủ — em đã đo thử với autocannon, latency p99 dưới 200 ms với 50 concurrent requests. Bottleneck tiềm năng nhất là recalc eligibility hàng loạt khi có nhiều quân nhân; em đã xử lý bằng `Promise.all batch query` thay vì sequential, và recalc chỉ chạy sau khi phê duyệt đề xuất, không phải mỗi request xem hồ sơ.”

“Nếu mở rộng áp dụng cho nhiều đơn vị trong toàn quân thì code phải sửa gì?” → “Có hai hướng. Hướng một: deploy nhiều instance độc lập (mỗi đơn vị một server nội bộ riêng) — không cần sửa code, phù hợp với mô hình mạng nội bộ tách biệt từng đơn vị. Hướng hai: thêm `tenant_id` vào schema để một instance phục vụ nhiều đơn vị — phức tạp hơn, cần sửa toàn bộ query filter và middleware. Với đặc thù bảo mật quân đội, hướng một thực tế hơn.”

“CI/CD thì em có làm không?” → “Em có chạy test tự động (jest) và kiểm tra kiểu (tsc --noEmit) trước mỗi lần build — đây là phần quan trọng nhất của CI. Deploy thì chạy thủ công bằng PM2 trên máy chủ nội bộ vì không có CI server riêng trong phạm vi đề tài. Nếu đơn vị có server build nội bộ (Jenkins/Gitea CI), có thể tích hợp pipeline chạy test tự động trước khi kéo code mới lên máy chủ.”

“Test Controller layer thì sao — coverage 60 % có đủ không?” → “Controller trong project này rất mỏng — trung bình 8–12 dòng, chỉ parse request rồi gọi service. Logic nghiệp vụ nằm hoàn toàn ở Service và Eligibility layer — đây là nơi em tập trung test với coverage > 85 %. Coverage Controller thấp hơn chấp nhận được vì nguy cơ bug ở đó gần như không có — nếu service đúng thì controller đúng.”

“Nếu quy chế khen thưởng thay đổi — ví dụ rút ngắn số năm để được xét BKBQP — thì hệ thống phải sửa gì? Có nên làm giao diện để SUPER_ADMIN tự chỉnh không?”

→ “Đây là rủi ro bảo trì thực tế nhất của hệ thống. Em đã cố gắng tập trung tham số nghiệp vụ vào constants, nhưng mức độ thay đổi khác nhau rõ rệt theo từng loại.

Các loại chỉ cần sửa constant — ít rủi ro:

- *HCQKQT*: 1 dòng `HCQKQT_YEARS_REQUIRED = 25` trong `danhHieu.constants.ts`. Toàn bộ logic đọc từ đó, sửa xong chạy lại test là xong.
- *KNC VSNXD QĐNDVN*: 2 dòng — `KNC_YEARS_REQUIRED_NAM = 25` và `KNC_YEARS_REQUIRED_NU = 20`. Quy chế có phân biệt Nam/Nữ nên tách 2 constant.
- *Cá nhân hằng năm / Đơn vị hằng năm*: Chu kỳ (2/3/7 năm) và số cờ yêu cầu (3 BKBQP + 2 CSTDTQ cho BKTTC) đều trong `chainAwards.constants.ts`. Sửa `cycleYears` hoặc `requiredFlags` là xong logic. **Tuy nhiên rủi ro ở test**: 937 test case có nhiều fixture dùng số năm cụ thể (quân nhân có 2 năm CSTDCS → đủ BKBQP). Nếu đổi thành 3 năm, phải rà lại fixture và assertion trong các test suite `eligibility-bkbqp`, `eligibility-cstdtq`, `eligibility-bkttcp`.

Trường hợp khó — HCBVTQ khi đổi nhóm hệ số:

Điều kiện HCBVTQ dựa trên tổng số tháng công tác chia theo nhóm hệ số chức vụ. Hiện có 3 nhóm: 0.7 / 0.8 / 0.9–1.0. Nếu quy chế đổi — ví dụ tách nhóm 0.9–1.0 thành 2 nhóm riêng (0.9 và 1.0), hoặc thêm nhóm 0.6 — thì **không phải chỉ sửa constant**. Phải thay đổi toàn bộ 4 tầng:

1. **Constants** (`danhHieu.constants.ts`): thêm key mới vào `CONG_HIEN_HE_SO_GROUPS` và range tương ứng trong `CONG_HIEN_HE_SO_GROUP_RANGES`.
2. **Service logic** (`aggregatePositionMonthsByGroup`): hàm này phân loại từng lịch sử chức vụ vào nhóm dựa trên hệ số — phải cập nhật để nhận diện nhóm mới.
3. **Database schema**: mỗi nhóm có 1 cột JSON riêng trong bảng `KhenThuongHCBVTQ` (`thoi_gian_nhom_0_7`, `thoi_gian_nhom_0_8`, `thoi_gian_nhom_0_9_1_0`). Thêm nhóm → thêm cột → phải viết Prisma migration và ALTER TABLE. Hiện có khoảng 40 chỗ trong code tham chiếu tên cột nhóm này.
4. **Dữ liệu đã lưu**: các đề xuất HCBVTQ đã phê duyệt lưu thời gian theo nhóm cũ. Dữ liệu đó không tự recalculate — cần script chạy lại `aggregatePositionMonthsByGroup` từ lịch sử chức vụ cho từng quân nhân và cập nhật lại cột.

Có nên làm giao diện để SUPER_ADMIN tự chỉnh thông số không?

Không nên, ít nhất với giai đoạn hiện tại. Lý do: - Các tham số này không đơn thuần là con số — chúng ảnh hưởng dây chuyền đến eligibility logic, test suite, và dữ liệu đã lưu. Cho admin tự chỉnh mà không chạy lại test là không có safety net. - Quy chế

quân đội thay đổi rất hiếm (hàng năm hoặc ít hơn), ROI của giao diện cấu hình thấp so với chi phí làm đúng. - Trường hợp đổi nhóm hệ số HCBVTQ là thay đổi structural — không thể expose qua UI mà không có migration đi kèm.

Giải pháp thực tế hơn là: document rõ quy trình bảo trì — ‘khi quy chế thay đổi, developer sửa constants, chạy 937 test, nếu test fixture cần cập nhật thì cập nhật, rồi chạy script recalc nếu cần’. Chi phí thấp hơn nhiều so với làm giao diện cấu hình an toàn.”

Chúc bạn bảo vệ thành công. Hệ thống đã đầy đủ tính năng, có số đo định lượng rõ ràng, có audit log đầy đủ, có 937 test pass — đều là vũ khí mạnh khi hội đồng truy vấn. Khi đứng trước hội đồng, hít sâu, nói chậm, mắt nhìn vào người hỏi và đừng quên: **mọi thứ trong đồ án này em đã sống với 6 tháng — em là người hiểu nó nhất phòng.**

Q. Mô phỏng phản biện hội đồng (3 vai chuyên gia)

Phụ lục này giả lập câu hỏi từ 3 giáo sư phản biện theo 3 chuyên ngành: Phần mềm/Kiến trúc web, Cơ sở dữ liệu, An toàn thông tin. Mỗi câu kèm “bây” (lý do hội đồng hỏi) và mẫu trả lời “đỉnh” để học luồng. Không cần đọc thuộc lòng — đọc để biết hướng phòng thủ.

Q.A. Giáo sư Phần mềm — Kiến trúc

Q.A.1 — “Em chọn layered architecture 6 lớp, nhưng dự án chỉ có 1 sinh viên và 23 model — không phải overengineering sao? Tại sao không pure MVC?” **Bây:** hỏi để xem em có hiểu trade-off hay chỉ học vẹt pattern.

Trả lời: “Em đồng ý câu hỏi hợp lý. Em chọn 6 lớp vì 2 lý do cụ thể, không phải vì pattern đẹp.

Thứ nhất, **Repository tách khỏi Service** là quyết định em đưa ra **giữa kỳ**, không phải đầu kỳ. Ban đầu service gọi `prisma.danhHieuHangNam.findMany(...)` trực tiếp. Khi viết test cho rule chuỗi, em phát hiện mỗi test phải mock toàn bộ Prisma client — rất nặng. Tách Repository xong, test chỉ mock `danhHieuRepository.findManyByPersonnelId()` — interface nhỏ hơn nhiều, test viết nhanh hơn. Lợi ích đo được, không chỉ lý thuyết.

Thứ hai, **Middleware tách khỏi Controller** xuất phát từ nhu cầu đa dạng route: route public (login) khác route admin-only khác route manager với unit-filter. Nếu để chung trong controller, sẽ lặp `if (req.user.role !== 'ADMIN')` ... ở 94 route mutate. Middleware chain `verifyToken` → `requireRole` → `validate` → `auditLog` → controller giúp em chỉ viết controller cho phần business, các concern khác declarative.

Em **đã loại bỏ Clean Architecture đầy đủ** (Use Case Layer + Entity Layer + Adapter) vì với 23 model, Use Case Layer sẽ trùng lặp gần như hoàn toàn với Service — chỉ thêm boilerplate không có lợi ích. Đây là chỗ em nghĩ ranh giới overengineering nằm.”

Q.A.2 — “Strategy pattern cho 7 loại đề xuất — nếu chỉ thêm 1 loại mới mỗi 2 năm, có cần Strategy không? switch case 7 nhánh chẳng phải đơn giản hơn?” **Trả lời:** “Em có 2 lý do thực dụng để chọn Strategy:

- (1) **switch 7 nhánh xuất hiện ở 4 chỗ** — `submitProposal`, `approveProposal`, `importExcel`, `build-SuccessMessage`. Nếu để switch, mỗi lần thêm danh hiệu mới phải sửa 4 hàm. Lỡ quên 1 chỗ → bug silent. Với Strategy registry, thêm 1 file `nckhStrategy.ts` + 1 dòng vào REGISTRY, TypeScript ép em implement đủ 4 method interface.
- (2) **Logic riêng từng loại quá khác nhau** — HC_QKQT và KNC là ‘single-medal’ (lifetime), CA_NHAN_HANG_NAM là chuỗi 3 cấp với cycle, HCCSVV là theo mốc 10/15/20 năm. Switch lồng if/else cho 7 nhánh khác nhau sẽ đẻ ra 1 hàm 600+ dòng — không test được unit.

Em thừa nhận trade-off: nếu chỉ 2-3 loại, Strategy là over-design. Em chọn ngưỡng từ 4 loại trở lên + 3+ chỗ dispatch. Với $7 \text{ loại} \times 4 \text{ chỗ} = 28$ nhánh dispatch — Strategy ROI rõ ràng.”

Q.A.3 — “Tại sao em không viết unit test cho Frontend? 0 test FE là điểm trừ lớn.” **Bây:** thừa nhận hay đổ lỗi.

Trả lời: “Em thừa nhận đây là hạn chế của đồ án.

Nguyên nhân thật: thời gian. Trong 4 tháng làm đồ án 1 mình, em ưu tiên (1) BE service test cho rule chuỗi danh hiệu — đây là logic phức tạp nhất, sai là sai quyết định khen thưởng; (2) integration test cho 7 strategy đề xuất; (3) kiểm thử thủ công FE qua giao diện. Kết quả: BE có 80 test file / 937 case pass, FE chỉ test tay.

Hậu quả: mỗi lần em sửa code FE, phải click lại toàn bộ luồng — tốn thời gian và dễ miss regression. Nếu hệ thống vào production và team mở rộng, đây là nợ kỹ thuật phải trả trước.

Hướng phát triển em đã viết trong báo cáo: setup Jest + React Testing Library cho FE, mục tiêu coverage 70% cho component logic-heavy (form đề xuất nhiều bước, bảng review). Em chưa làm vì đây là 1-2 tuần effort, không đủ thời gian.”

Nếu hội đồng vẫn tiếp “tại sao biết hạn chế mà không sắp xếp thời gian”: “Em đánh giá đúng vs sai logic nghiệp vụ là priority cao hơn UI regression — sai logic là duyệt khen thưởng nhầm, sai UI là khó dùng nhưng dữ liệu vẫn đúng. Em chấp nhận trade-off này, biết rằng nếu có thêm 1 tuần, em sẽ dồn vào FE test.”

Q.B. Giáo sư Cơ sở dữ liệu

Q.B.1 — “23 model trong 1 schema, em có làm normalization đúng 3NF không? Hay có denormalize cố ý?” Trả lời: “Em có cả 2 — phần lớn 3NF, một số denormalize cố ý.

Tuân 3NF nghiêm: QuanNhan, CoQuanDonVi, DonViTrucThuoc, ChucVu, LichSuChucVu — chuẩn quan hệ, không có transitive dependency.

Denormalize cố ý có 3 chỗ:

- (1) **Bảng HoSoHangNam, HoSoNienHan, HoSoCongHien** — gọi là ‘derived tables’. Mỗi row tính từ dữ liệu nguồn (DanhHieuHangNam, LichSuChucVu, ThanhTichKhoaHoc). Vi phạm 3NF (có thể tính lại từ nguồn), nhưng em lưu vì 2 lý do: (a) tính tới 47ms/quân nhân — nhân 1.247 quân nhân là quá chậm cho mỗi request list; (b) FE cần lọc theo `du_dieu_kien_bkbqp = true` — query trên bảng suy diễn nhanh hơn rebuild context cho mỗi row.

Đánh đổi: phải đảm bảo HoSoHangNam luôn đồng bộ với nguồn. Em làm bằng cách: mọi mutate trên DanhHieuHangNam đều trigger `recalculateAnnualProfile(personnelId)` trong cùng transaction. Hàm này idempotent — chạy nhiều lần cho cùng kết quả.

- (2) **he_so_chuc_vu lưu snapshot trong LichSuChucVu** thay vì chỉ FK. Vì hệ số chức vụ có thể đổi theo thời gian (vd: ‘Trợ lý’ năm 2020 hệ số 0.8, năm 2025 đổi thành 0.9), nhưng đợt phục vụ năm 2020 vẫn phải tính theo 0.8. Lưu snapshot tránh phải maintain bảng LichSuHeSo riêng.
- (3) **BangDeXuat.data_danh_hieu là JSON** — vi phạm 1NF strict. Lý do: 7 loại đề xuất có lược đồ chi tiết rất khác nhau, tạo 7 bảng trung gian sẽ phá tính đồng nhất khi query ‘pending proposals’. JSON đủ vì validation đã làm ở tầng Zod + Strategy.”

Q.B.2 — “Schema có index không? Index nào em chủ động tạo, index nào để Prisma tự sinh?” Trả lời: “Index tự sinh từ Prisma: tất cả @id (UUID primary key), tất cả @unique (vd: cccd, username), và mọi FK đều có index ngầm.

Index em **chủ động** thêm dựa trên query pattern thật:

- `@@unique([quan_nhan_id, nam])` trên DanhHieuHangNam — vừa là constraint nghiệp vụ vừa là composite index cho query ‘danh hiệu của quân nhân X năm Y’.
- `@@index([resource, createdAt])` trên SystemLog — query nhật ký lọc theo resource + sort theo thời gian là pattern dashboard chính.
- `@@index([status, loai_de_xuat])` trên BangDeXuat — list ‘pending proposals’ theo loại là endpoint hot nhất của Admin.

Em **chưa làm**: chưa có Explain Analyze cụ thể trên slow query. Hiện dataset thử nghiệm ~1.200 quân nhân, query đều dưới 100ms nên chưa có nhu cầu. Khi production có 10k+ quân nhân, em sẽ chạy EXPLAIN ANALYZE trên endpoint chậm để xác định index thiếu.”

Câu hỏi vặn: “Sao không thêm sẵn index cho mọi cột thường where?”

“Index không miễn phí: mỗi insert/update phải cập nhật B-tree. Với bảng DanhHieuHangNam (insert hàng loạt khi import Excel), thêm index thừa sẽ chậm import. Em theo nguyên tắc ‘add index khi có evidence’, không speculative.”

Q.B.3 — “Em dùng transaction cho approve, nhưng nếu transaction chạy 180 giây thì tất cả connection khác bị chặn không? Có connection pool không?” Trả lời: “Câu hỏi tốt, em phân tách 2 vấn đề.

- (1) **Connection pool**: Prisma có pool mặc định. Số connection = $\max(\text{num_cpus} \times 2 + 1, 1)$ — máy chủ 2 core của em là 5 connection. Một transaction chiếm 1 connection trong 180s nghĩa là còn 4 cho user khác — vẫn dùng được hệ thống, không ‘sập’.
- (2) **Lock**: PostgreSQL dùng MVCC — đọc không bao giờ block. Transaction approve chỉ giữ row-level lock trên các row đang ghi (DanhHieuHangNam của ~300 quân nhân được duyệt, BangDeXuat đó, FileQuyetDinh đồng bộ). Các tx khác đọc bất kỳ row nào — không bị chặn. Chỉ tx khác muốn UPDATE/DELETE đúng các row đang lock mới phải wait.

Trong domain Phòng Chính trị: Admin đang duyệt một đợt, không có người khác cùng sửa quân nhân đó cùng lúc. Risk thực tế thấp.

Nếu hội đồng hỏi: ‘180s là tự chọn hay do?’ — xem trả lời tại D.10 trong tài liệu này.”

Q.B.4 — “Backup chạy INSERT INTO ... VALUES 21 bảng nối chuỗi — sao không dùng pg_dump?” Bấy: hỏi vì pg_dump là chuẩn industry.

Trả lời: “Em chọn raw INSERT script vì 3 lý do:

- (1) **Không phụ thuộc pg_dump binary** trên máy production. Service backend chạy bằng Node.js, gọi pg_dump qua child process là thêm dependency runtime — nếu PostgreSQL update version, pg_dump cũng phải update đồng bộ. Code tự sinh INSERT chỉ phụ thuộc Prisma Client em đã ship cùng app.
- (2) **Cho phép selective backup** — em chỉ backup 21 bảng nghiệp vụ, bỏ qua _prisma_migrations, SystemLog cũ hơn N ngày. pg_dump lấy toàn bộ schema; em phải post-process strip ra.
- (3) **Backup script đọc được cho người**. Lúc demo có thể mở .sql thấy data — pg_dump xuất binary dump (-Fc) thì không.

Em **thừa nhận hạn chế**: format INSERT chậm hơn COPY cho dataset lớn; restore phải dùng psql ngoài hệ thống — không có UI restore one-click. Nếu production scale lên, em sẽ chuyển sang pg_dump -Fc + script wrapper. Hiện tại với dataset 4.2MB/backup, raw INSERT đủ nhanh.

Quan trọng nhất — em escape ‘ thành ’ ’ đúng chuẩn SQL string. Đây là điểm em làm cẩn thận để chống SQL injection chính trong dữ liệu (vd: tên có dấu nháy ‘D’arcy).”

Q.C. Giáo sư An toàn thông tin

Q.C.1 — “Em lưu Access Token trong localStorage. XSS là rủi ro lớn — sao không dùng httpOnly cookie?” Bấy: trade-off quan trọng nhất của JWT-based auth.

Trả lời: “Em đã cân nhắc kỹ trade-off này — chọn localStorage cố ý.

httpOnly cookie chống XSS vì JS không đọc được cookie. **Nhưng** mở ra CSRF — request từ tab khác đến cùng origin tự động đính cookie. Để chống CSRF phải thêm CSRF token, double-submit, hoặc SameSite=Strict.

localStorage chống CSRF (JS phải chủ động đọc), nhưng XSS đọc được.

Em chọn localStorage + biện pháp **chống XSS chủ động**: 1. React mặc định escape mọi text node — không có dangerouslySetInnerHTML ở đâu trong code (em đã grep). 2. Helmet middleware (helmet()) bật mặc định) set các header bảo vệ — CSP chưa được cấu hình riêng (hướng phát triển). 3. Mọi input đi qua Zod validation — chặn payload độc hại từ form.

Lý do thật quan trọng nhất: hệ thống chạy **mạng nội bộ Học viện**, không expose ra Internet. Vector tấn công chính không phải hacker XSS qua iframe — mà là user cài extension độc hoặc máy có malware. Cả 2 case này httpOnly cookie cũng không cứu được (malware đọc memory được).

Nếu hệ thống **deploy public Internet**, em sẽ đổi sang httpOnly + CSRF token. Hiện tại với context quân sự mạng nội bộ, localStorage đơn giản hơn, không phải mang chi phí CSRF protection.”

Q.C.2 — “Em cho phép user upload PDF làm quyết định. Nếu user upload .exe đổi đuôi thành .pdf thì sao?” Trả lời: “Em có nhiều lớp chống:

- (1) **Multer config** ở `configs/multer.ts` — `fileFilter` check MIME type `application/pdf`. Trình duyệt gửi MIME từ extension nên có thể giả mạo. Đây chỉ là lớp đầu.
- (2) **Magic bytes check** ở backend — đọc 4 byte đầu file, PDF thật bắt đầu bằng `%PDF (0x25 0x50 0x44 0x46)`. Nếu không khớp → reject.
- (3) **Lưu vào folder riêng uploads/decisions/** không nằm trong static serve của Next.js. Khi user request file qua `/api/decisions/:id/download`, backend kiểm tra quyền + stream file. **Không bao giờ** serve file user upload qua URL trực tiếp — tránh trường hợp file `.html` được render bởi trình duyệt khi click link.
- (4) **Filename sanitization** — em strip path component (`. /`), giữ chỉ ký tự `[a-zA-Z0-9._-]`. Tên lưu DB là tên đã sanitize + suffix timestamp tránh trùng.

Em thừa nhận hạn chế: chưa scan virus với ClamAV. Với scope quân sự, đây là phải có nếu deploy thật. Em đã ghi vào hướng phát triển: tích hợp ClamAV trước khi accept file.”

Q.C.3 — “Audit log của em có ghi mọi thao tác. Admin có thể xóa audit log của chính mình không?” Trả lời: “Không. Em thiết kế cố ý ngăn việc này:

Route DELETE `/api/system-logs` chỉ áp middleware `requireSuperAdmin`. Admin (vai trò khác SuperAdmin) gọi sẽ bị chặn ngay ở middleware, trả 403.

Lý do tách 2 vai trò: Admin có toàn quyền nghiệp vụ (tạo/duyệt/xóa đề xuất, quản lý quân nhân), nhưng **không có quyền chạm vào nhật ký kiểm toán**. SuperAdmin chỉ quản trị hạ tầng (account, backup) — không tham gia luồng nghiệp vụ. Hai vai này tách bạch cố ý để không một người duy nhất vừa thao tác vừa che dấu vết.

Còn 1 đặc thù: `resource = 'backup'` chỉ SuperAdmin xem được. Admin query `system-logs` **không thấy** các bản ghi backup — filter ngay ở tầng service, không phải UI hide. Mục đích: ngăn Admin biết khi nào SuperAdmin chạy backup → khó canh thời điểm.

Phản biện em chuẩn bị: nếu SuperAdmin muốn xóa log của chính mình thì sao? — Đúng, đây là điểm yếu. Trong hệ thống enterprise thật, audit log phải write-only (append-only), thậm chí lưu sang hệ thống riêng (vd: SIEM). Hiện tại em chưa có, chấp nhận limit này vì scope đồ án và SuperAdmin là bộ phận kỹ thuật có quy chế riêng.”

Q.C.4 — “JWT của em ký bằng HS256 (symmetric). Sao không RS256 (asymmetric)?” Trả lời: “HS256 và RS256 đều secure khi setup đúng. Em chọn HS256 vì:

- (1) **Chỉ 1 service verify token** — backend Express của em. RS256 (private sign, public verify) có lợi khi có nhiều microservice verify nhưng không sign. Em chỉ có 1 monolith → HS256 đủ.
- (2) **Implementation đơn giản hơn** — 1 secret env var (`JWT_SECRET`) thay vì cặp keypair phải sinh + lưu trữ + rotate.
- (3) **Hiệu năng tốt hơn** — HMAC nhanh hơn RSA verify nhiều lần. Mỗi request có verify 1 token, hệ thống chục user concurrent thì khác biệt nhỏ, nhưng vẫn là điểm cộng.

Em làm cẩn thận: dùng 2 secret khác nhau cho Access Token và Refresh Token (`JWT_SECRET` vs `JWT_REFRESH_SECRET`). Nếu attacker lấy được Access Token JWT secret từ memory dump, không tự ký được Refresh Token để chiếm phiên dài hạn. Đây là defense-in-depth.

Khi nào em chuyển sang RS256: nếu hệ thống mở rộng thành multi-tenant cho Bộ Quốc phòng (hướng phát triển iii), khi đó các tenant verify token độc lập, RS256 public key phân phối an toàn hơn rotate symmetric secret.”

Q.D. Câu khoai — vòng 2

Q.D.1 — “Em nói code 96.000 dòng TypeScript. Em viết tay hết hay dùng AI?” Bấy: nói dối là tự sát; nói thật sai cách cũng bị trừ.

Trả lời thẳng: “Em dùng AI assistant trong quá trình code, cụ thể là Claude. Em coi đó là công cụ như IDE autocomplete nâng cao.

Cách em dùng: em thiết kế kiến trúc, viết schema, vạch rule nghiệp vụ chuỗi danh hiệu — phần cần hiểu domain quân sự AI không biết. AI hỗ trợ em ở phần boilerplate: viết Zod schema từ TypeScript interface, sinh test case từ describe block, refactor pattern lặp.

Em **luôn đọc và hiểu mọi dòng code AI sinh** trước khi commit. Khi AI suggest sai (vd: dùng `prisma.X` trực tiếp trong service mà em đã có repository), em sửa lại theo convention của project — em viết `CLAUDE.md` chính là để áp ràng buộc này.

Bằng chứng em hiểu code: hội đồng có thể yêu cầu em mở bất kỳ file nào, em giải thích từng đoạn được. Em đặc biệt hiểu sâu phần rule chuỗi danh hiệu (`checkChainEligibility`, `computeChainContext`) vì phần đó AI không hiểu domain — em phải viết spec rõ trước rồi mới guide AI implement.

Em không xem việc dùng AI là gian lận — giống như sinh viên trước đây dùng Stack Overflow, IntelliSense, ChatGPT để học. Quan trọng là sản phẩm có chạy đúng, em có hiểu, có maintain được. Cả 3 em đều OK.”

Q.D.2 — “PROJECT_REVIEW.md của em flag ‘logic eligibility trùng 2 chỗ — semantic divergence risk’. Sao không fix đi mà còn flag?” Bầy: hỏi để xem em có biết tại sao không fix.

Trả lời: “Đây là rủi ro em nhận diện ra cuối kỳ — `computeEligibilityFlags` (chạy khi recalc profile) và `checkAwardEligibility` (chạy khi validate approve) thực hiện cùng một rule core. Nếu logic 2 hàm drift, một phía cho phép, phía kia từ chối → bug khó debug.

Lý do em chưa refactor:

- (1) **Cả 2 hàm hiện tại đều gọi `chainEligibility.checkChainEligibility` chung** — đã extract phần core. Phần riêng chỉ là `computeEligibilityFlags` áp dụng lifetime block cho personal BKTTC (kiểm tra has-ReceivedBKTTC flag riêng). Đây là edge case nhỏ, không phải logic chính.
- (2) **Refactor sâu hơn cần extract `EligibilityRuleEngine` class** — risk cao vì rule chuỗi danh hiệu là phần test phức tạp nhất (15+ test file). Sửa wrong sẽ break 50+ test case. Em ưu tiên ổn định trước demo, gán flag để team sau (nếu có) refactor đúng cách.
- (3) **Test em viết phủ chéo 2 hàm** — `tests/services/eligibility-*.test.ts` test `chainEligibility.checkChainEligibility` (core), `tests/approve/*.test.ts` test luồng `checkAwardEligibility` (wrapper). Nếu drift xảy ra, ít nhất một bộ test sẽ fail.

Đây là quyết định em đưa ra có ý — chấp nhận technical debt được track, hơn là refactor vội làm break test.”

Q.D.3 — “Em demo trên dataset bao nhiêu? Đã test với dữ liệu thật chưa?” **Trả lời thẳng:** “Dataset demo của em là dữ liệu mô phỏng, không phải dữ liệu thật từ Học viện. Em không có quyền truy cập dữ liệu quân nhân thật vì lý do bảo mật.

Em xây dataset mô phỏng dựa trên: - Cấu trúc cây tổ chức của Học viện (10 CQDV, ~40 DVTT, dựa trên thông tin công khai) - Phân bố tuổi quân nhân theo nhóm hệ số 0.7/0.8/0.9-1.0 - ~1.247 quân nhân ảo với lịch sử khen thưởng giả định 5-25 năm - 50 hồ sơ chứa các kịch bản edge case của rule chuỗi (lỡ 1 chu kỳ, lỡ nhiều chu kỳ, đã có BKTTC, chuyển đơn vị giữa năm)

Em đã trao đổi với cán bộ phụ trách thi đua tại Học viện qua phòng vấn nghiệp vụ. Kết quả phỏng vấn vào báo cáo §2.1 — ước tính tỷ lệ bỏ sót 15-20%, thời gian xét 1 quân nhân 20-30 phút, là từ cán bộ nói trực tiếp.

Em không thể claim ‘đã test với dữ liệu thật’ — nếu hội đồng hỏi, em trả lời thật như trên. Hướng phát triển ngay: pilot deploy ở Phòng Chính trị với dữ liệu thật 50-100 quân nhân để validate edge case từ thực tế.”

Q.D.4 — “Backup .sql chứa toàn bộ dữ liệu nhạy cảm (CCCD, hồ sơ cán bộ). Em lưu ở backups/ — ai có thể đọc folder này?” **Trả lời:** “Backups lưu tại `BE-QLKT/backups/` thuộc filesystem của user chạy PM2 (thường là user `qlkt` deploy). Quyền filesystem **chỉ user đó read/write**, group/others không có quyền — em set `chmod 700` cho folder.

SuperAdmin **không truy cập file system trực tiếp** — phải qua DevZone UI (`/api/backups/:id/download`) có auth + role check. File path trong DB là internal, không expose ra ngoài API.

Em thừa nhận điểm yếu: backup .sql là plaintext. Nếu disk bị compromise (vd: SuperAdmin laptop bị mất, hacker SSH vào server), backup là ‘treasure’.

Cải tiến đã ghi vào hướng phát triển: - Mã hoá backup bằng AES-256 với key lưu ở vault (Hashicorp Vault hoặc OS keyring), không lưu cùng file. - Off-site backup: gửi backup mã hoá lên kho lưu trữ khác (NAS Học viện, không cùng máy chủ ứng dụng).

Hiện tại trong scope đồ án, em mới làm backup local + retention 15 ngày + restrict access. Cho production thật phải làm thêm encryption + off-site.”

Q.D.5 — “Em verify quyền `requireManager` để Manager chỉ thấy quân nhân thuộc cây đơn vị mình. Em verify này thế nào — middleware kiểm role hay query có filter?” Trả lời: “Em làm cả 2 lớp — đây là defense-in-depth quan trọng:

Lớp 1 — Middleware `unitFilter` (`middlewares/unitFilter.ts`): chạy sau `verifyToken`, đọc `req.user.id`, query cây đơn vị Manager phụ trách (từ field `co_quan_don_vi_id` và `don_vi_truc_thuoc_id` của tài khoản) rồi tính danh sách quân nhân thuộc cây. Gán vào `req.unitFilter = { ...unitInfo, personnelIds: [...] }`.

Lớp 2 — Service query: mọi method service nhận `unitFilter` từ controller, intersect trên `personnelIds` trong `where` clause:

```
where: {
  AND: [
    userFilter, // input filter từ user
    { id: { in: req.unitFilter.personnelIds } },
  ]
}
```

Tại sao 2 lớp? Nếu chỉ middleware: developer tương lai có thể quên gán `unitFilter` vào query, query lấy tất cả → IDOR. Service ép filter là ‘fail-secure’ — nếu thiếu `unitFilter`, query sẽ trả mảng rỗng (an toàn hơn lấy tất cả).

Em test: trong `tests/authz/` có test case Manager A query quân nhân của đơn vị B → 0 row trả về, dù endpoint không có middleware role check nào khác.

Hạn chế em thừa nhận: Admin route không apply `unitFilter` (Admin xem tất cả) — đúng business rule nhưng có nghĩa là: nếu Admin bị compromise, attacker thấy toàn bộ data. Mitigation: Admin có MFA bắt buộc (chưa implement, hướng phát triển).”

Q.E. Mẫu trả lời “không biết”

Q.E.1 — “Em có biết Postgres `vacuum` chạy thế nào khi `updateMany` batch lớn không?” Trả lời mẫu khi không chắc: “Em hiểu cơ bản: Postgres MVCC giữ row cũ sau UPDATE để các transaction cũ đọc được, `vacuum` (hoặc `auto-vacuum`) sau đó dọn dead tuple. Với `updateMany` 300 row, sẽ tạo 300 dead tuple, `autovacuum` sẽ chạy khi vượt ngưỡng `autovacuum_vacuum_scale_factor`.

Tuy nhiên em **không tự tin về tuning chi tiết** — chưa cấu hình `autovacuum` threshold cho app này, dùng default Postgres. Trong đồ án em chưa gặp vấn đề performance từ bloat. Nếu thầy cô có gợi ý cụ thể em rất muốn nghe để tìm hiểu thêm.”

Nguyên tắc trả lời “không biết”: 1. **Thừa nhận thẳng** — không vòng vo, không giả vờ. 2. **Show what you DO know** — kể phần kiến thức nền tảng có liên quan. 3. **Đặt câu hỏi ngược lại** — “Thầy cô có gợi ý cụ thể không?” — biến từ chỗ bị dồn thành cơ hội học. Hội đồng đánh giá cao sinh viên tò mò hơn sinh viên giả vờ biết.